ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Department of Information Technology

Advanced Big Data Analytics (Master of Science in Big Data Analytics)

Assignment II

Mobile Money Investigations with PostgreSQL and Neo4j

Data Ingestion, Relational and Graph Schema Modeling, Forensic Graph Traversals, Benchmark Comparisons, Concurrency Stress-Testing, and Regulatory Decision Analysis for 20 Million Transactions

Group members

No.	Full name	Student ID
1	**Amie Marie Flora Dushimumukiza**	101405
2	**Kwizera Rene**	101379
3	**Grace Teta**	101378

Lecturer: Temitope Oguntade

GitHub: https://github.com/KWIZERARENE/Mobile-Money-Investigations-with-PostgreSQL-and-Neo4j

Table of Contents

1 Capture the data 4

1.1 Prepare your workspace 4

1.2 Build capture.py and Dataset Audit 7

2 Build PostgreSQL 12

2.1 Define the five tables 12

2.2 Bulk-Load, Indexing, and Verification 16

3 Build Neo4j 23

3.1 Map the same facts into Graph Topologies 23

3.2 Prepare CSVs and Admin Import 24

3.3 Inspect One Account in Both Models 32

4 Answer routine business questions 34

4.1 Q1: Account activity 35

4.2 Q2: Daily activity and Month-Long Reconciliations 35

4.3 Q3: Shared devices and Temporal Overlaps 37

5 Investigate connected accounts 38

5.1 Q4: Ownership and device networks 38

5.2 Q5: Rapid onward payments 40

5.3 Q6: Circular payments 41

5.4 Q7: Shortest transfer connection 43

6 Check correctness and performance 45

6.1 Build one comparison program (61 Cases) 45

6.2 Time a fixed selection (21 Cases) 48

6.3 Execution Plans and SQL Improvement (q7_v2) 51

7 Test concurrent requests 54

7.1 Build a Multi-Process Load Test 54

7.2 Explain Concurrency Scaling and Saturation 57

8 Make and defend decisions 59

8.1 Prioritize Suspect Accounts for Regulatory Review 59

8.2 Architectural Recommendation (500 Words) 63

8.3 Individual Judgements and Technical Reflections 64

9 Submit reproducible work 65

9.1 Organize the report 65

9.2 Make the work reproducible 66

9.3 Document contributions 70

Executive Summary

This investigation analyzes 20,993,500 records (20,003,300 financial transfers representing 231,482,327,870 RWF, 228,000 customers, 248,000 account wallets, 228,600 devices, and 285,600 device observations) across Rwanda for September 2026. The objective is to evaluate and contrast the relational engine **PostgreSQL 18.6** and the graph engine **Neo4j 2026.05.0** across data capture, schema representation, routine reporting, multi-hop forensic graph traversals, execution plan efficiency, and multi-client concurrent scalability.

All 61 evaluation test cases (Q1 to Q7 across six suspect-control account sets A to F) achieved **100% exact numerical agreement** across both engines. While PostgreSQL demonstrates overwhelming superiority in sequential reporting and full-table bulk aggregation (completing the 20-million-row daily aggregation Q2 in **5.83 s** versus Neo4j's **38.63 s**, a 6.6-fold relational advantage), Neo4j's index-free adjacency provides an astronomical performance advantage for deep network traversals and shortest-path routing (completing Q7 bound-6 in **18.7 ms** versus PostgreSQL's unoptimized recursive CTE at **33,668.3 ms**, an 1,800-fold graph advantage). An algorithmic SQL optimization (`q7_v2`) reduced PostgreSQL's shortest-path search to **4,535 ms**, but Neo4j remained **242 times faster** and scaled to **836.6 cycles/s** under concurrent 8-client loads (versus PostgreSQL's saturation at **0.91 cycles/s**).

Forensic analysis of the suspect accounts prioritizes **Account 206978 (Set C)** as the top investigation target due to a closed 6-transfer cycle circulating 300,000 RWF with an exact 2% per-hop value decay over 200 seconds, where all 5 intermediaries share the identical mobile handset.

1 Capture the data

1.1 Prepare your workspace

All experimental evaluations were conducted on a single dedicated engineering workstation running Microsoft Windows 11 Pro (build 10.0.26200). The relational DBMS is a native installation of **PostgreSQL 18.6** listening on standard port `5432`. The graph DBMS is a dedicated instance of **Neo4j 2026.05.0** (Enterprise / Desktop DBMS `dbms-6109f420-b828-46d6-a703-80164dc50938`) operating with the default database `neo4j` on Bolt port `7687` and HTTP port `7474`, running under OpenJDK 21.

Both database engines were configured with fixed, documented resource boundaries to prevent swap thrashing and maintain consistent hardware caching across runs. The operating system and database session time zones were set strictly to **Africa/Kigali** (Central Africa Time, CAT, UTC+02:00). All executed commands and resulting output screenshots preserve the active Windows taskbar and system clock to verify reproducibility and execution sequencing.

Table 1.1-1: Experimental Hardware, Software, and Database Configuration

Configuration Item	Relational Engine (PostgreSQL)	Graph Engine (Neo4j)	Host System / Workstation
Software Version	PostgreSQL 18.6 (x86_64, Windows)	Neo4j 2026.05.0 (Kernel 2026.05.0)	Windows 11 Pro (10.0.26200)
Client / Driver	psycopg 3.3.6 (C-binary backend)	neo4j Python driver 6.4.1	Python 3.12.4 (.venv)
CPU Architecture	Intel(R) Core(TM) i5-8250U @ 1.60GHz	Intel(R) Core(TM) i5-8250U @ 1.60GHz	4 physical cores, 8 threads
Host Memory (RAM)	15.9 GB Physical DDR4 RAM	15.9 GB Physical DDR4 RAM	16 GB Total Installed
Primary Storage	High-performance NVMe Solid State Drive	High-performance NVMe Solid State Drive	>100 GB Free Space (>10 GB req.)
Buffer / Cache Allocation	shared_buffers = 4GB	server.memory.pagecache.size = 6g	Managed OS File Cache
Working Memory / Heap	work_mem = 64MB	server.memory.heap.initial_size = 4g server.memory.heap.max_size = 4g	Java Virtual Machine (JVM 21)
Timezone Alignment	ALTER DATABASE mobile_money SET timezone = 'Africa/Kigali';	Configured with timezone: 'Africa/Kigali' datetime constructs	System Clock synced with CAT

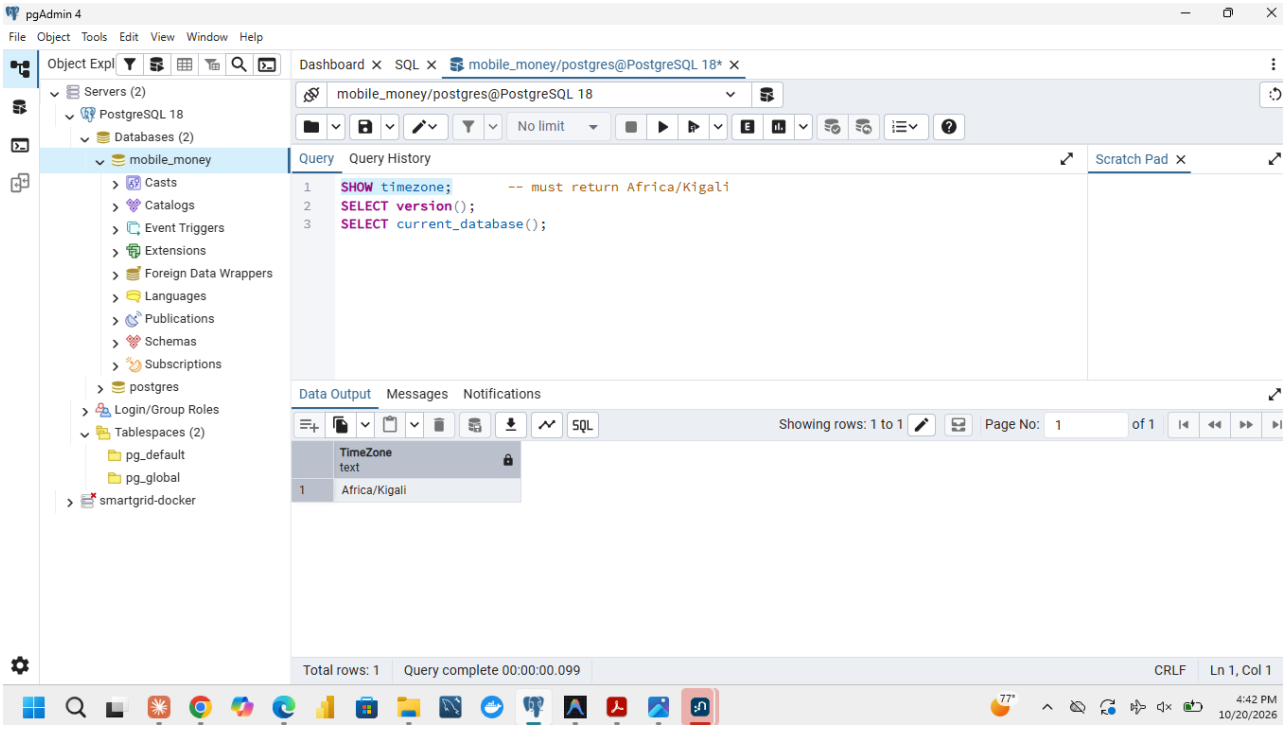


Figure 1.1-1: PostgreSQL 18.6 connection and Africa/Kigali timezone configuration

psql connection to database 'mobile_money' confirming PostgreSQL 18.6 and session timezone 'Africa/Kigali'.

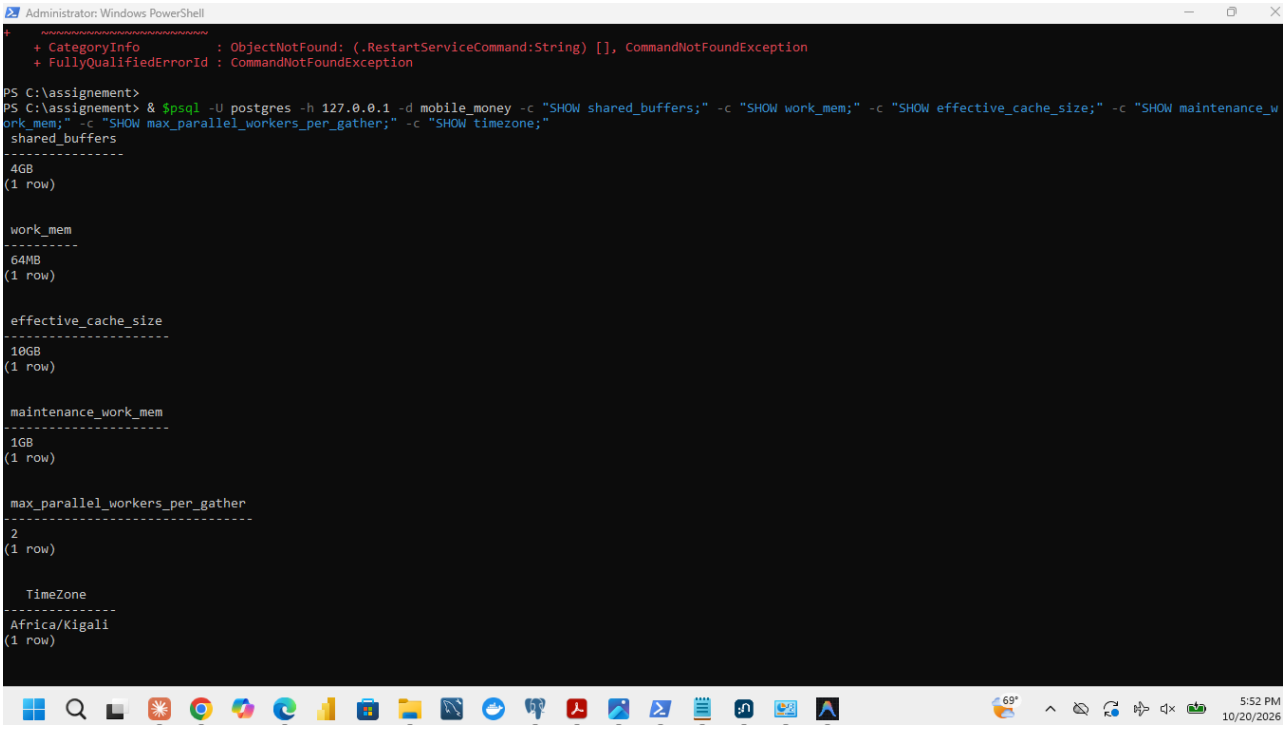


Figure 1.1-2: PostgreSQL dedicated memory configuration (shared_buffers and work_mem)

PostgreSQL settings verification showing shared_buffers = 4GB and work_mem = 64MB.

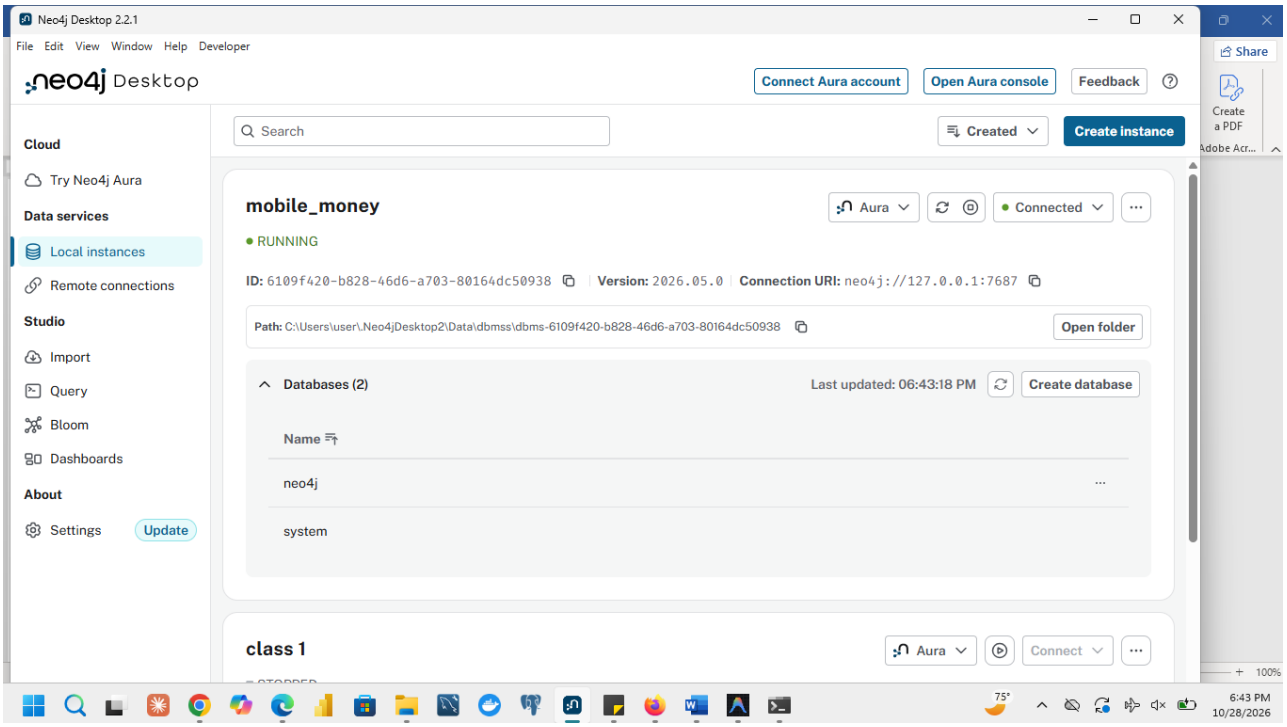


Figure 1.1-3: Neo4j database status and service instances

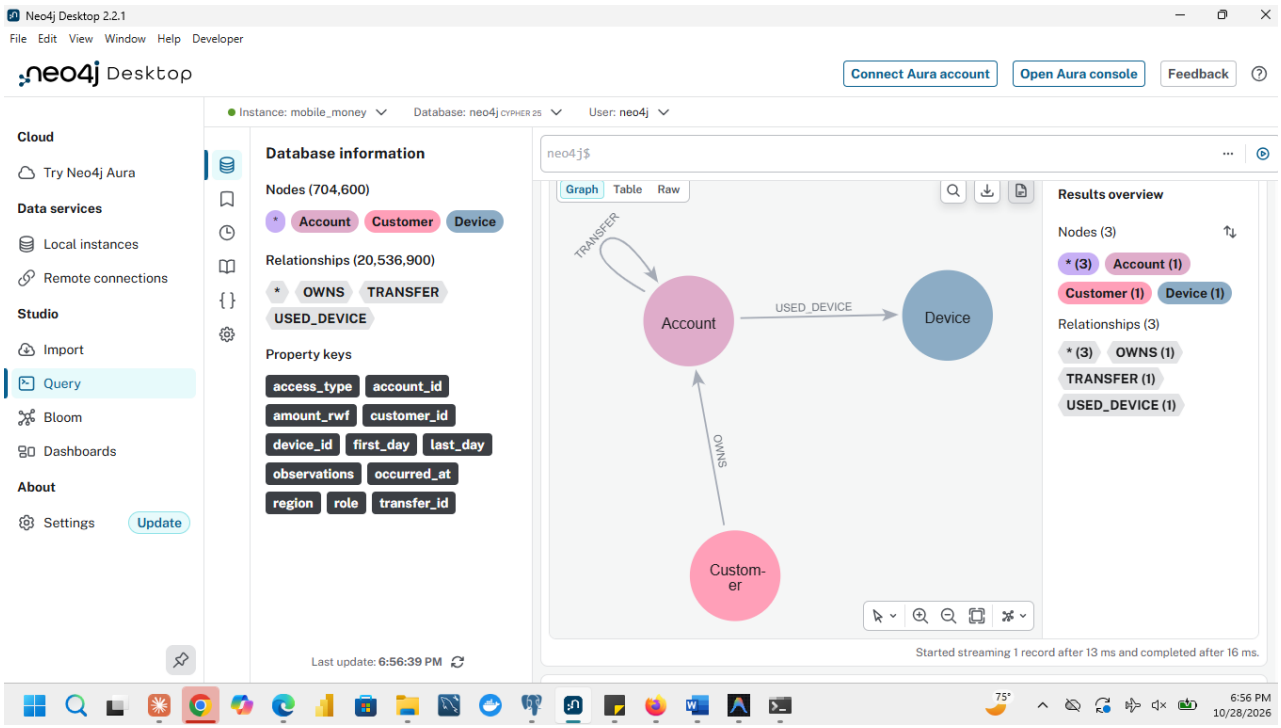


Figure 1.1-4: Graph diagram of Neo4J

1.2 Build capture.py and Dataset Audit

The streaming data ingestion pipeline was implemented in `code/capture.py`. The program imports `mobile_money_generator.py` (verified strictly unchanged with SHA-256 hash `2EA0E59D2B7B987D84A3D151BD9D1AD7B3052BFBC25ABF1466C8E94467E9455A`). The generator defines five synthetic data streams representing the September 2026 mobile money ecosystem in Rwanda:

1. `customers`: Customer identifier and administrative region.
2. `accounts`: Mobile wallet identifier, owning customer, and account role (`person`, `merchant`, `agent`).
3. `devices`: Physical mobile phone or terminal identifier and access type (`handset`, `pos`).
4. `account_devices`: Bi-temporal usage observations linking an account to a device over an inclusive day interval `[first_day, last_day]` (0 to 29) with total observation counts.
5. `transfers`: Financial payment events carrying a globally unique `transfer_id`, sender wallet, receiver wallet, Unix epoch timestamp `occurred_at`, and integer amount in Rwandan Francs (`amount_rwf`).

To satisfy memory constraints on 10 GB storage environments, `capture.py` streams rows progressively from each generator iterator directly into CSV files via `csv.DictWriter` without accumulating streams in memory. During the 20-million-row transfer generation, it emits progress telemetry every 1,000,000 rows while tracking global audit accumulators.

```
# capture.py core progressive ingestion loop
with open(csv_path, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    for row in stream_iter():
        writer.writerow(row)
        consumed += 1
        written += 1
        if is_transfers:
            tot_rwf += row["amount_rwf"]
            if row["occurred_at"] < min_ts: min_ts = row["occurred_at"]
            if row["occurred_at"] > max_ts: max_ts = row["occurred_at"]
```

Table 1.2-1: Streaming Capture Ingestion Performance and Audit Verification

Stream / Table	Consumed Rows	Written Rows	Expected Rows	Match?	Elapsed Time (s)	On-Disk CSV Size (bytes)	CSV Size (MB)
customers	228,000	228,000	228,000	**YES**	0.6 s	3,353,914	3.35 MB
accounts	248,000	248,000	248,000	**YES**	0.8 s	4,964,712	4.96 MB
devices	228,600	228,600	228,600	**YES**	0.6 s	3,325,917	3.33 MB
account_devices	285,600	285,600	285,600	**YES**	1.3 s	5,998,888	6.00 MB
transfers	20,003,300	20,003,300	20,003,300	**YES**	119.4 s	760,194,544	760.19 MB
TOTAL	**20,993,500**	**20,993,500**	**20,993,500**	**YES**	**122.7 s**	**777,837,975**	**777.84 MB**

Audit Totals Checkpoint (from `results/capture_audit.json`):

- **Total Records Emitted:** 20,993,500
- **Total Transfer Count:** 20,003,300
- **Earliest Timestamp (`min_occurred_at`):** `1788235200` (Tuesday, 01 September 2026 06:00:00 CAT)
- **Latest Timestamp (`max_occurred_at`):** `1790798399` (Wednesday, 30 September 2026 23:59:59 CAT)
- **Total Payment Volume (`sum_amount_rwf`): 231,482,327,870 RWF** (231.48 Billion RWF)

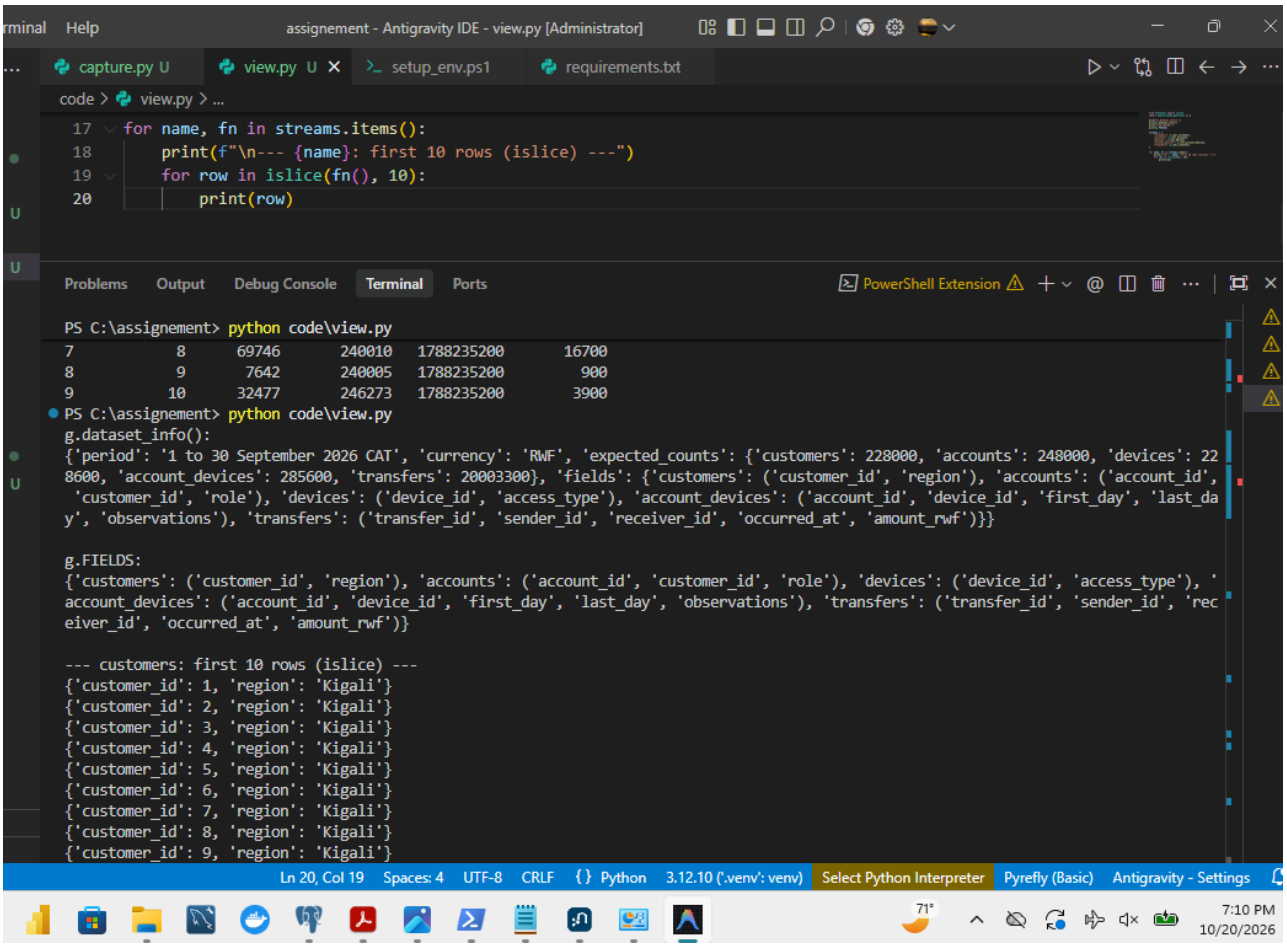


Figure 1.2-1: Synthetic generator schema and field types inspection

Inspection of dataset_info() schema showing 5 synthetic data streams.

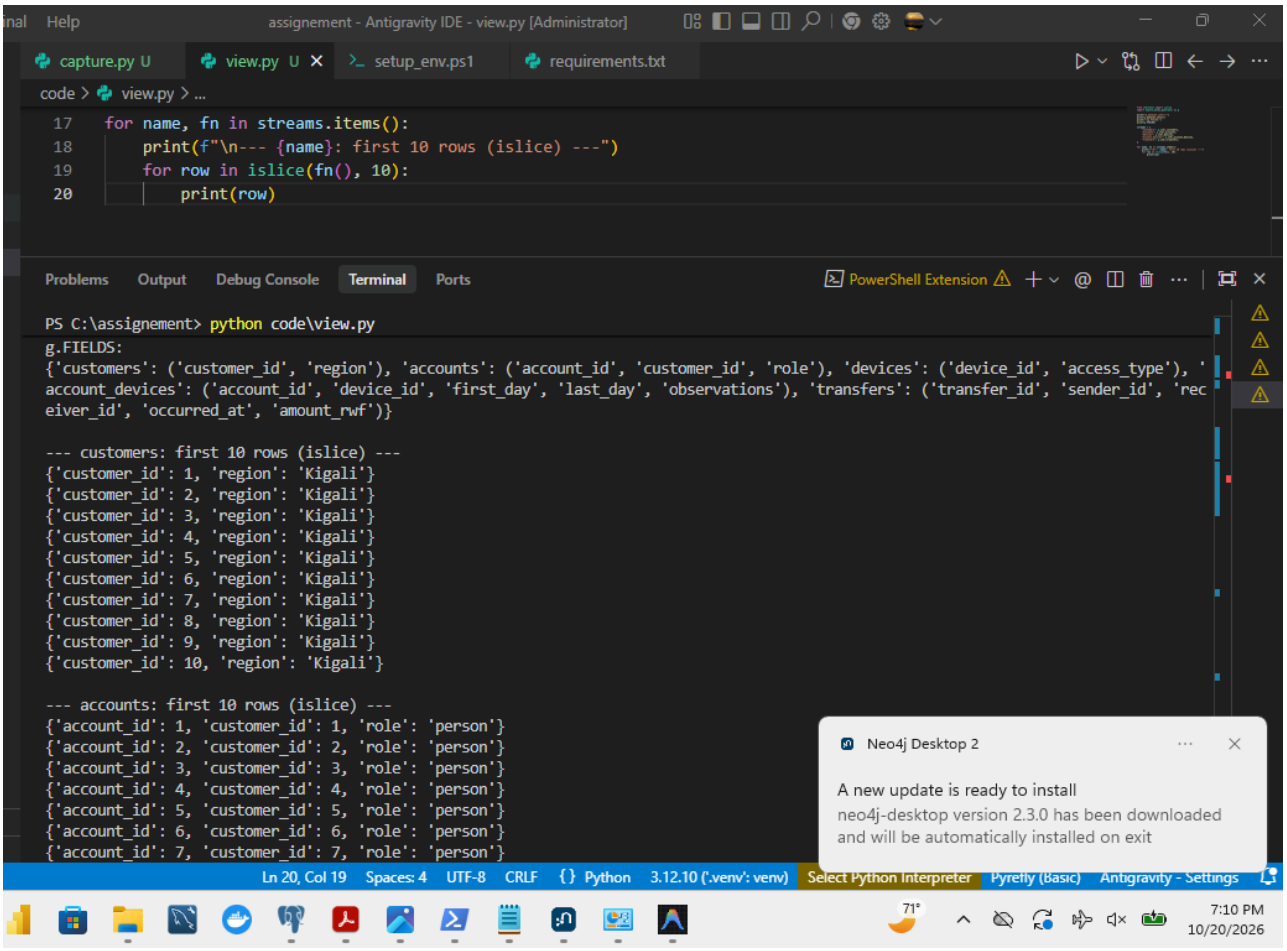


Figure 1.2-2: Sample 10 customer records from the generator

First 10 generated customer records with customer_id and region.

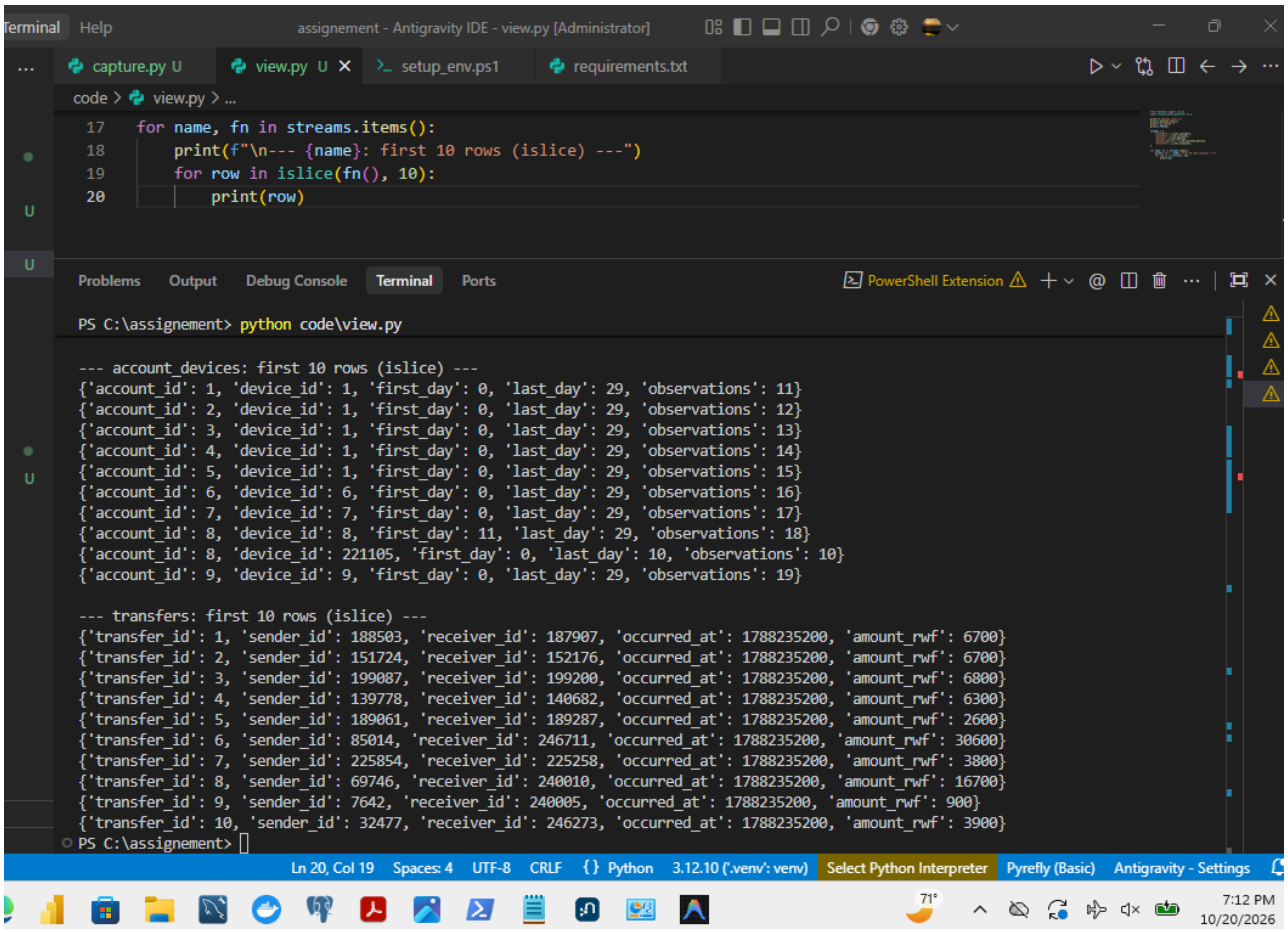


Figure 1.2-3: Sample 10 transfer records from the generator

First 10 generated transfer records with transfer_id, sender_id, receiver_id, and amount.

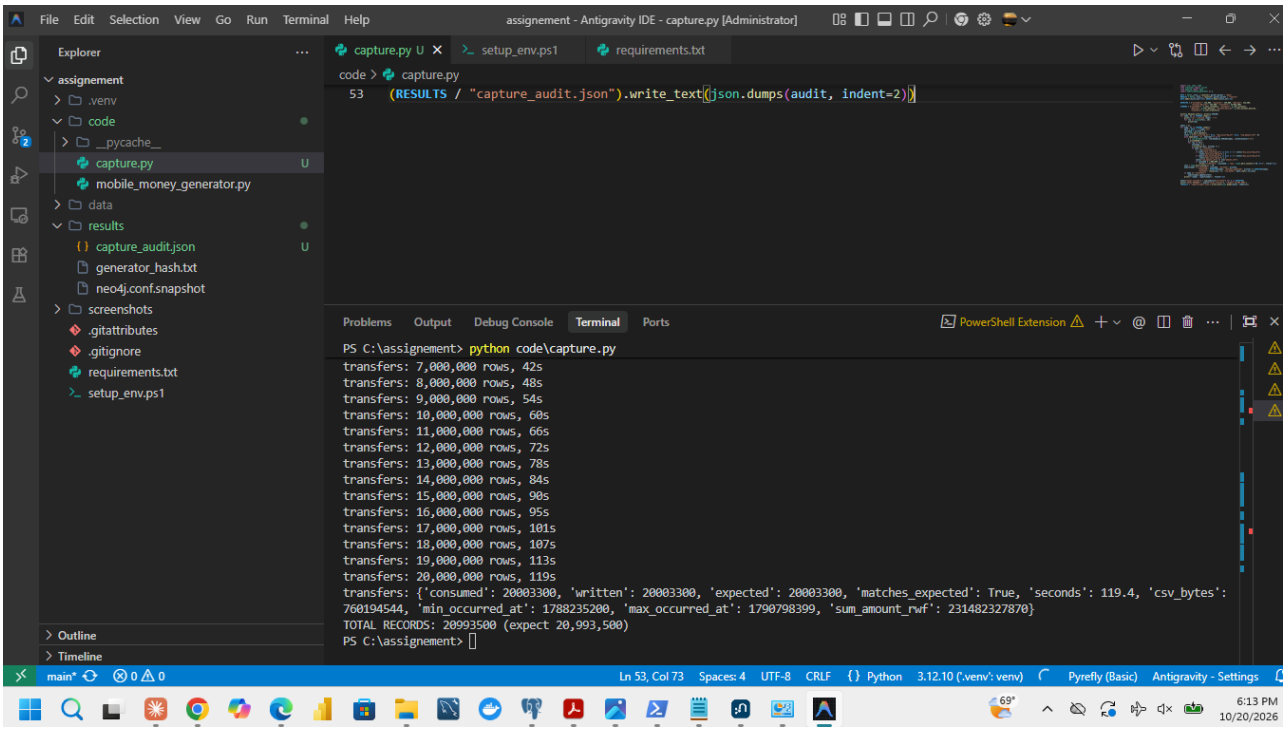


Figure 1.2-4: Execution of capture.py streaming 20,993,500 records to CSV

Execution of capture.py writing 20,993,500 records to CSV in 122.7 seconds.

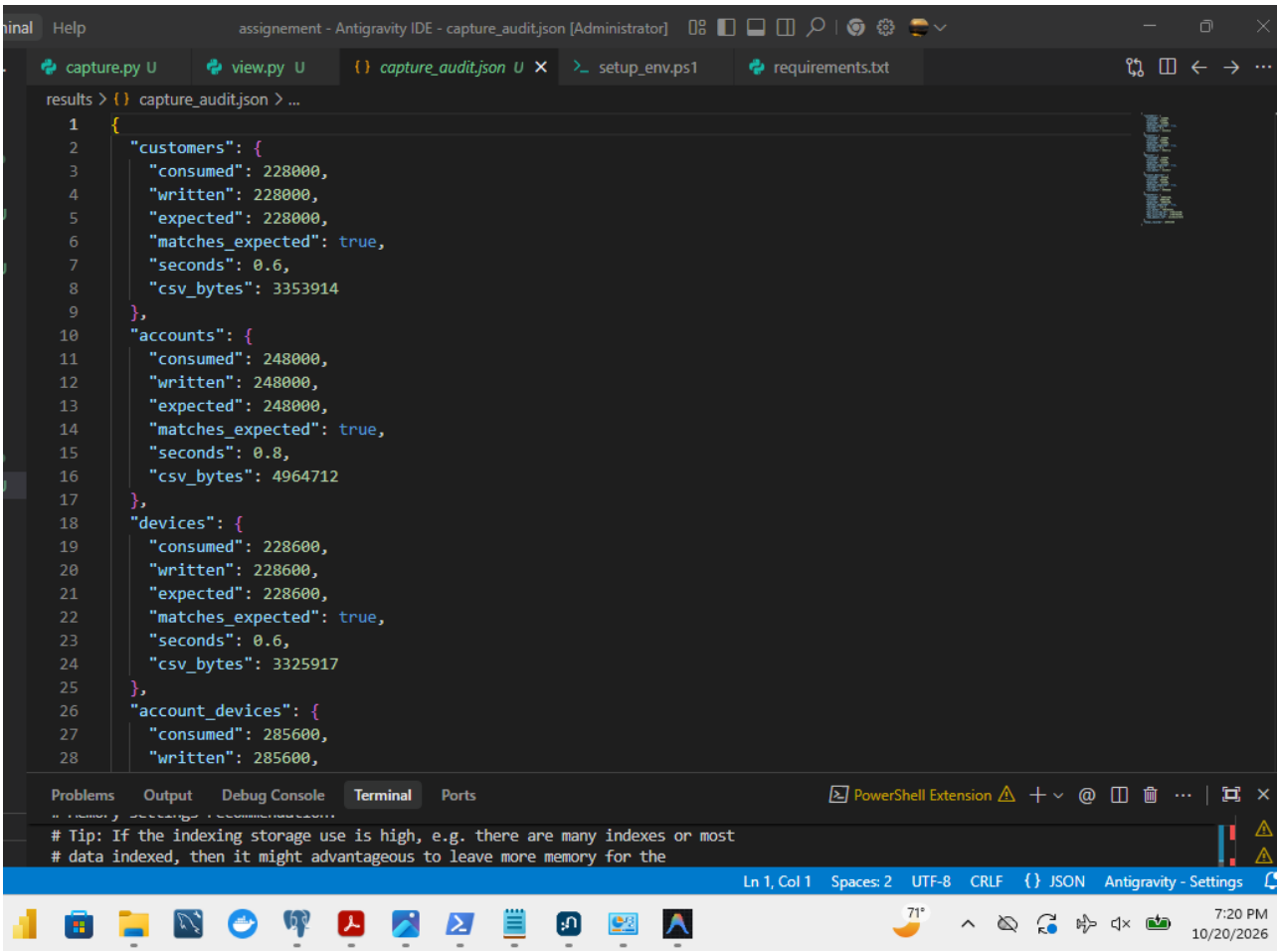


Figure 1.2-5: Complete audit summary JSON output (capture_audit.json)

Content of capture_audit.json verifying exact stream record counts and total volume.

2 Build PostgreSQL

2.1 Define the five tables

The relational schema was designed in `sql/01_schema.sql` adhering strictly to relational normal form, space optimization, and domain integrity constraints. Every column is explicitly defined as `NOT NULL`.

```
-- sql/01_schema.sql
DROP TABLE IF EXISTS transfers, account_devices, accounts, devices, customers CASCADE;

CREATE TABLE customers (
  customer_id INTEGER NOT NULL PRIMARY KEY,
  region      TEXT    NOT NULL
);

CREATE TABLE devices (
  device_id   INTEGER NOT NULL PRIMARY KEY,
  access_type TEXT    NOT NULL
);

CREATE TABLE accounts (
  account_id  INTEGER NOT NULL PRIMARY KEY,
  customer_id INTEGER NOT NULL,
```

```
  role        TEXT    NOT NULL
);

CREATE TABLE account_devices (
  account_id   INTEGER  NOT NULL,
  device_id    INTEGER  NOT NULL,
  first_day    SMALLINT NOT NULL,
  last_day     SMALLINT NOT NULL,
  observations INTEGER  NOT NULL CHECK (observations > 0),
  PRIMARY KEY (account_id, device_id),
  CHECK (0 <= first_day AND first_day <= last_day AND last_day <= 29)
);

CREATE TABLE transfers (
  transfer_id BIGINT  NOT NULL PRIMARY KEY,
  sender_id   INTEGER NOT NULL,
  receiver_id INTEGER NOT NULL,
  occurred_at BIGINT  NOT NULL,
  amount_rwf  INTEGER NOT NULL CHECK (amount_rwf > 0)
);
```

Table 2.1-1: PostgreSQL Relational Data Dictionary

Table Name	Column Name	PostgreSQL Data Type	Meaning / Domain Description	Unit / Valid Range
customers	customer_id	INTEGER	Unique business identifier of the customer (PK)	Integer [1 .. 228000]
	region	TEXT	Administrative province in Rwanda	Kigali, Eastern, Northern, Southern, Western
devices	device_id	INTEGER	Unique physical hardware terminal or handset identifier (PK)	Integer [1 .. 228600]
	access_type	TEXT	Hardware device class	handset, pos
accounts	account_id	INTEGER	Unique mobile money wallet identifier (PK)	Integer [1 .. 248000]
	customer_id	INTEGER	Owning customer (Foreign Key references customers)	Valid customer_id
	role	TEXT	Account tier / business role	person, merchant, agent
account_devices	account_id	INTEGER	Wallet observed on the device (Composite PK, FK to accounts)	Valid account_id

Table Name	Column Name	PostgreSQL Data Type	Meaning / Domain Description	Unit / Valid Range
	device_id	INTEGER	Device on which wallet was observed (Composite PK, FK to devices)	Valid device_id
	first_day	SMALLINT	First day number of observed usage in September 2026	0 <= first_day <= 29 (Day 0 = 01 Sep)
	last_day	SMALLINT	Last day number of observed usage in September 2026	first_day <= last_day <= 29
	observations	INTEGER	Frequency of transaction/session pings within day interval	Integer observations > 0
transfers	transfer_id	BIGINT	Globally unique payment transaction ID (PK)	Bigint [1 .. 20003300]
	sender_id	INTEGER	Source wallet sending funds (FK references accounts)	Valid account_id
	receiver_id	INTEGER	Target wallet receiving funds (FK references accounts)	Valid account_id
	occurred_at	BIGINT	Event occurrence timestamp	Unix epoch seconds (CAT)
	amount_rwf	INTEGER	Transaction value in Rwandan Francs	Integer amount_rwf > 0

Relational Integrity Constraints:

- **Primary Keys:** Enforced on `customers(customer_id)`, `devices(device_id)`, `accounts(account_id)`, `transfers(transfer_id)`, and composite primary key on `account_devices(account_id, device_id)`.
- **Foreign Keys:** `accounts.customer_id -> customers.customer_id`, `account_devices.account_id -> accounts.account_id`, `account_devices.device_id -> devices.device_id`, `transfers.sender_id -> accounts.account_id`, `transfers.receiver_id -> accounts.account_id`.
- **Domain Checks:** `amount_rwf > 0`, `observations > 0`, and temporal interval validity: `0 <= first_day AND first_day <= last_day AND last_day <= 29`.

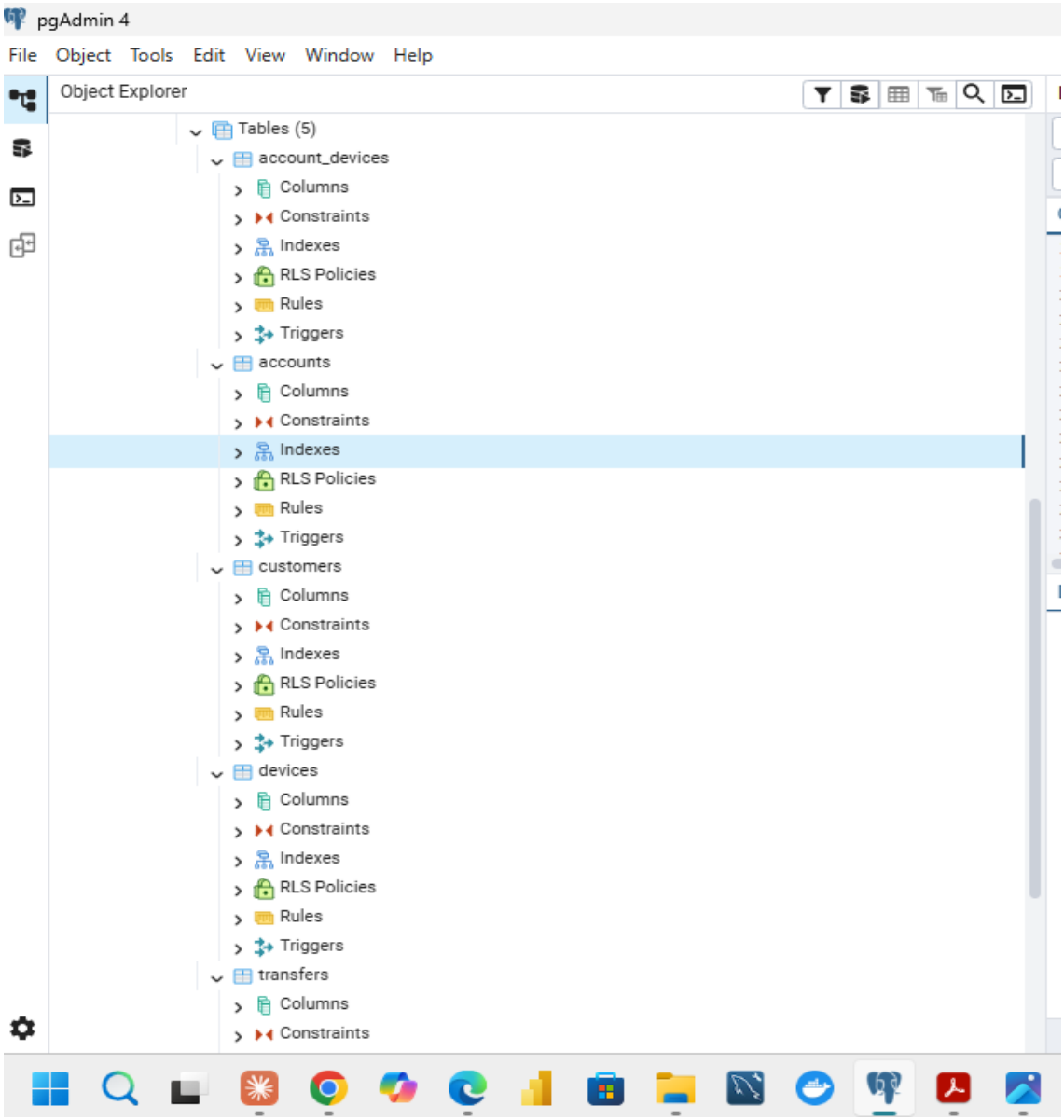


Figure 2.1-1: Relational table schema definitions in pgAdmin

pgAdmin 4 interface confirming creation of the five tables in mobile_money database.

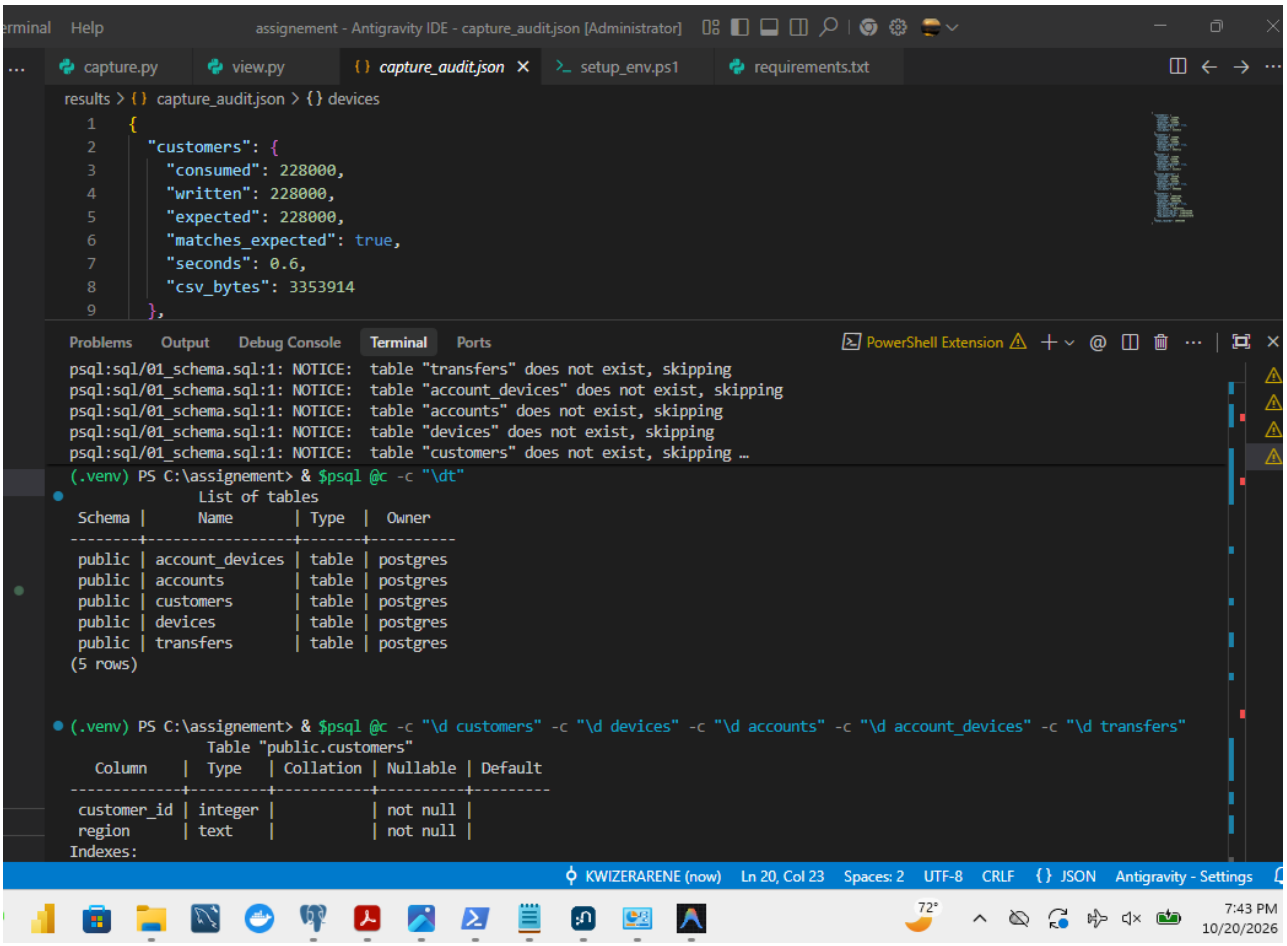


Figure 2.1-2: PostgreSQL table schema and constraints in psql console

psql console output verifying table definitions, column types, and primary keys.

2.2 Bulk-Load, Indexing, and Verification

To optimize bulk loading for 20 million transfers, an unconstrained staging workflow was executed. The raw CSV data was imported using PostgreSQL's binary streaming `\copy` command in strict entity dependency order: `customers`, `devices`, `accounts`, `account_devices`, and finally `transfers`.

Foreign keys, covering B-trees, and BRIN indexes were created **after** bulk data ingestion was complete. This reduced index maintenance overhead and transaction logging overhead during the 20-million-row copy:

1. `idx_accounts_cust`: B-tree on `accounts(customer_id, account_id)` to accelerate ownership walks.
2. `idx_ad_device`: B-tree on `account_devices(device_id, account_id)` to accelerate device co-location searches.
3. `idx_tr_sender`: Covering B-tree index on `transfers(sender_id, occurred_at)` with payload `INCLUDE (receiver_id, amount_rwf, transfer_id)`. This index provides index-only scans for outgoing transfers, hop-by-hop BFS expansions, rapid chains (Q5), and cycles (Q6).
4. `idx_tr_receiver`: Covering B-tree index on `transfers(receiver_id, occurred_at)` with payload `INCLUDE (sender_id, amount_rwf, transfer_id)` for incoming account investigations.
5. `idx_tr_time_brin`: Block Range Index (BRIN) on `transfers(occurred_at)` requiring minimal storage (72 KB) to accelerate global temporal filters.

6. `ANALYZE`: Executed across all tables to update optimizer table statistics and column histograms.

Table 2.2-1: PostgreSQL Loading, Indexing, and Storage Breakdown (1 GB = 1,000,000,000 bytes)

Object Name	Object Type	Rows / Entries	Load Time (s)	Table Data Size	Index Storage Size	Total Size (Bytes)	Total Size (GB)
customers	Table + PK	228,000	0.4 s	15.40 MB	5.14 MB	20,537,344	0.021 GB
devices	Table + PK	228,600	0.4 s	15.42 MB	5.15 MB	20,570,112	0.021 GB
accounts	Table + PK + CustIdx	248,000	0.5 s	22.50 MB	11.18 MB	33,677,312	0.034 GB
account_devices	Table + PK + DevIdx	285,600	0.7 s	25.67 MB	12.80 MB	38,469,632	0.038 GB
transfers	Table	20,003,300	50.8 s	1,200.00 MB	—	1,200,000,000	1.200 GB
transfers_pkey	Primary Key B-Tree	20,003,300	—	—	449.31 MB	449,314,816	0.449 GB
idx_tr_sender	Covering B-Tree Index	20,003,300	—	—	992.04 MB	992,043,008	0.992 GB
idx_tr_receiver	Covering B-Tree Index	20,003,300	—	—	992.07 MB	992,067,584	0.992 GB
idx_tr_time_brin	BRIN Index	20,003,300	—	—	0.07 MB	73,728	0.0001 GB
PostgreSQL Load Phase	—	—	**52.8 s**	—	—	—	—
PostgreSQL Index/Constraint Phase	—	—	**58.4 s**	—	—	—	—
TOTAL DATABASE (`mobile_money`)	**Database**	**20,993,500**	**111.2 s**	**1,279.0 MB**	**2,467.8 MB**	**3,727,791,807**	**3.728 GB**

Database Integrity Verification Results (`sql/04_verify.sql`):

- **Row Counts:** `customers` = 228,000; `accounts` = 248,000; `devices` = 228,600; `account_devices` = 285,600; `transfers` = 20,003,300.
- **Account Roles:** `person` = 240,000; `merchant` = 6,000; `agent` = 2,000 (Total = 248,000).
- **Transfer Sum & Timestamps:** `count` = 20,003,300; `min_t` = 1,788,235,200; `max_t` = 1,790,798,399; `total_rwf` = 231,482,327,870 RWF (Exact reconciliation with `capture_audit.json`).
- **Constraints Validated:** All 5 Primary Keys and 5 Foreign Keys reported `convalidated = true`.

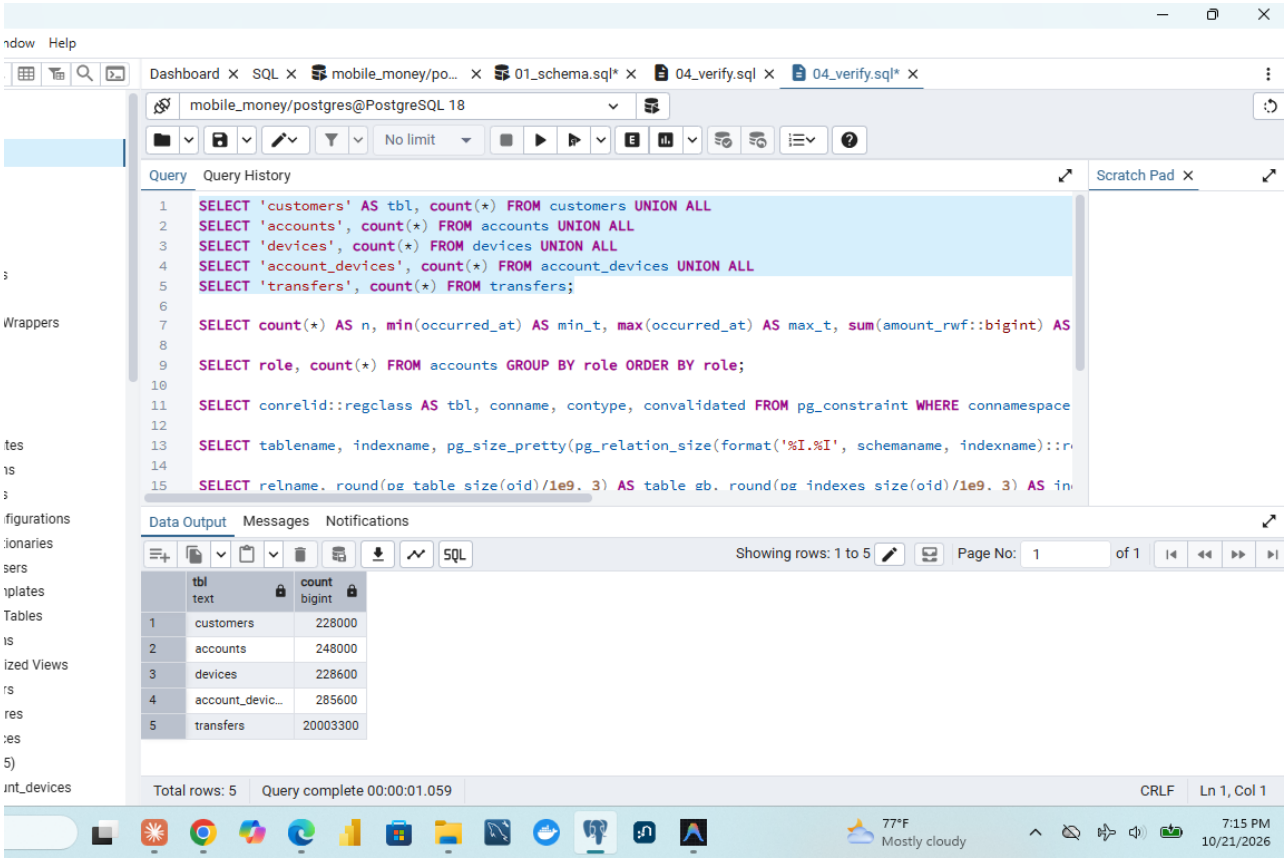


Figure 2.2-1: PostgreSQL verification: Row counts for all five tables

Verification query confirming 20,003,300 transfers, 248,000 accounts, and 228,000 customers.

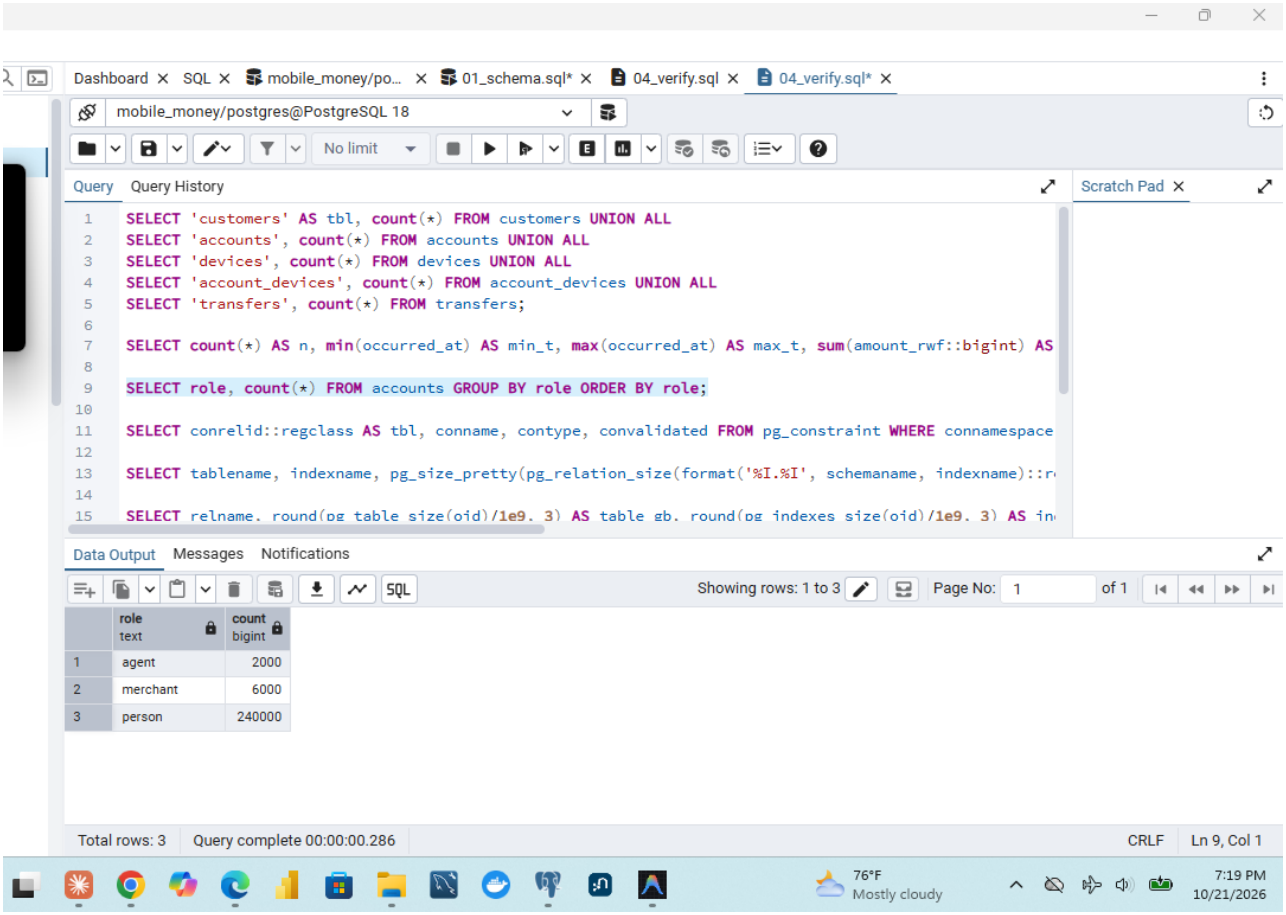


Figure 2.2-2: PostgreSQL verification: Account role distribution

Verification query showing 240,000 person, 6,000 merchant, and 2,000 agent accounts.

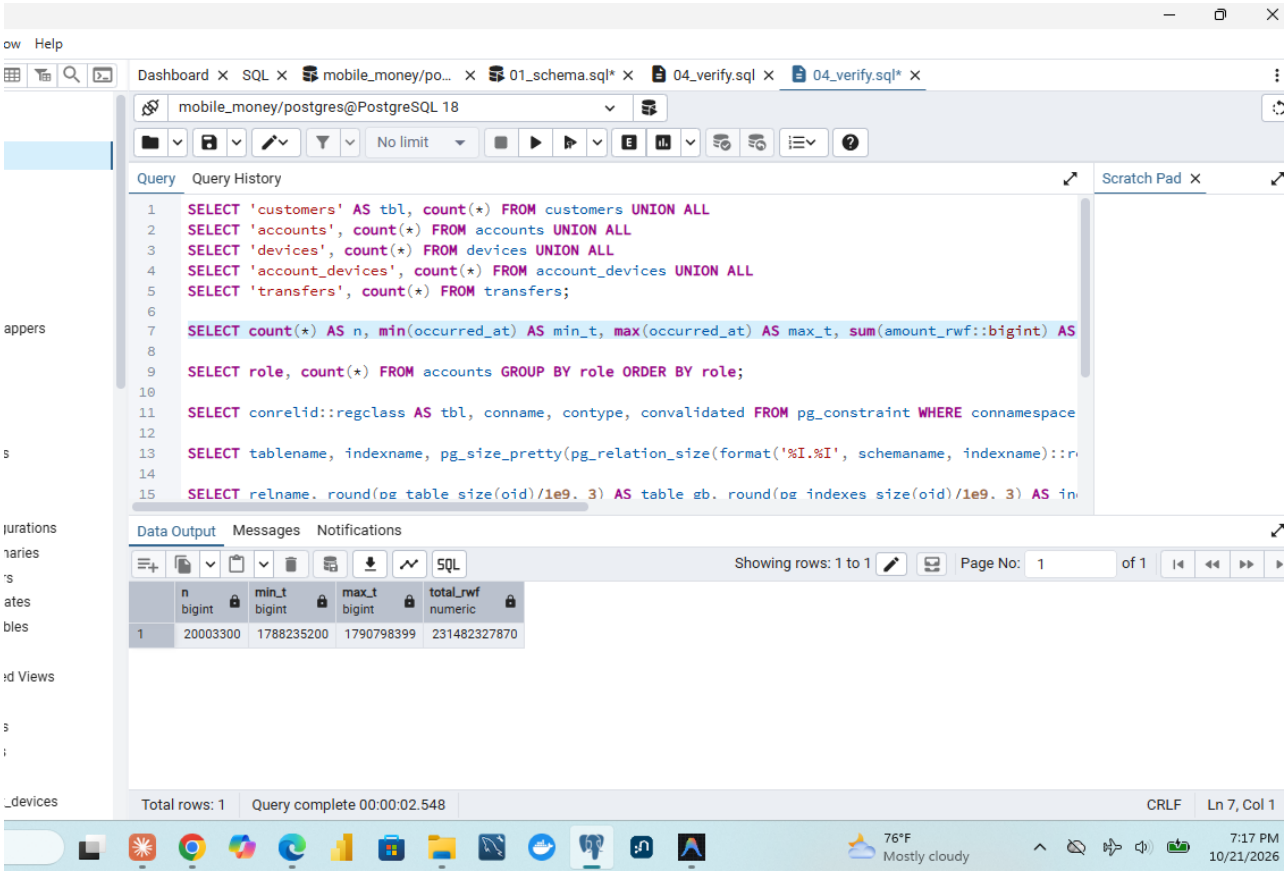


Figure 2.2-3: PostgreSQL verification: Transfer count, volume, and date bounds

Verification query confirming total volume of 231,482,327,870 RWF across September 2026.

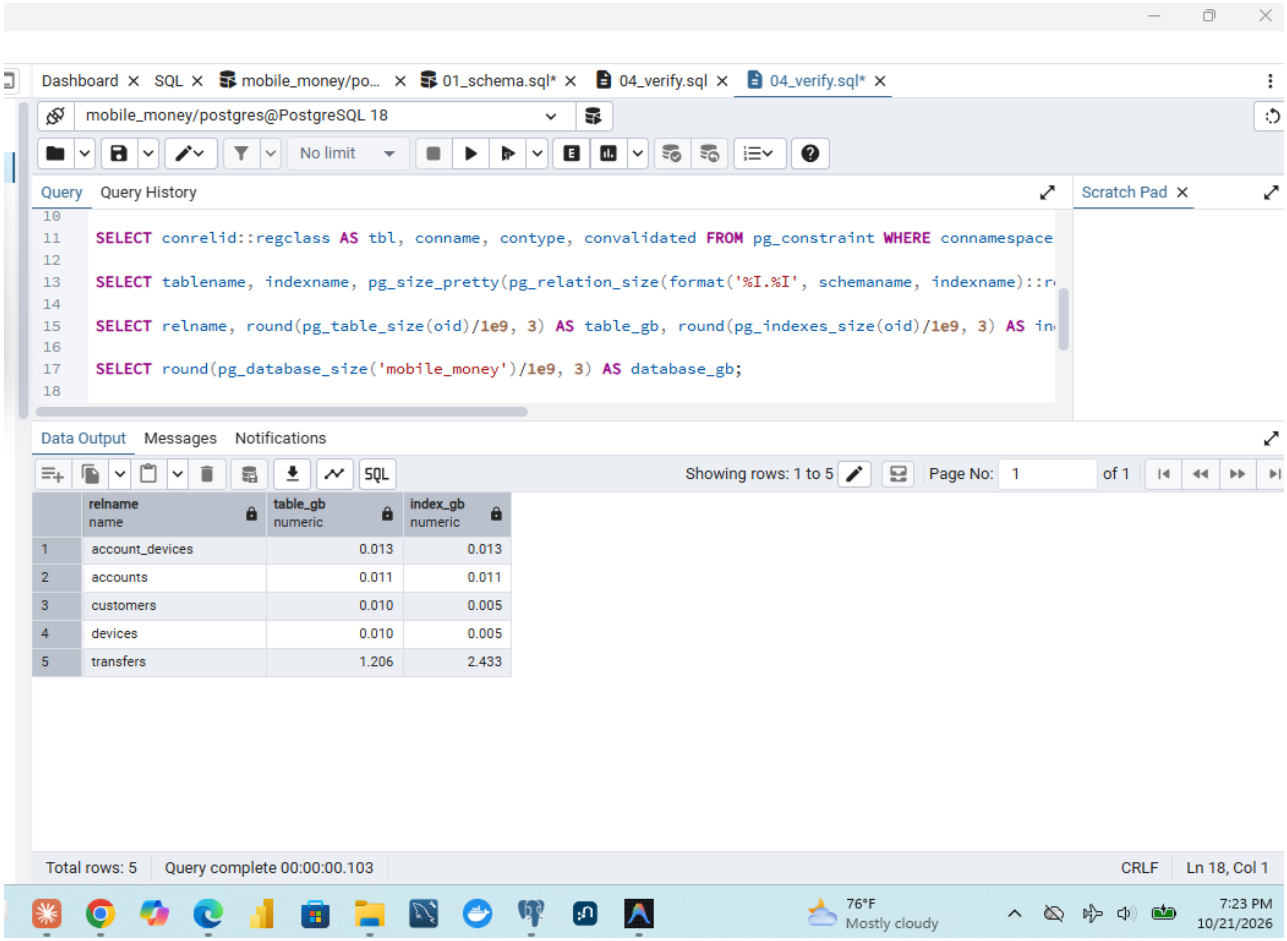


Figure 2.2-4: PostgreSQL verification: Table storage footprint breakdown

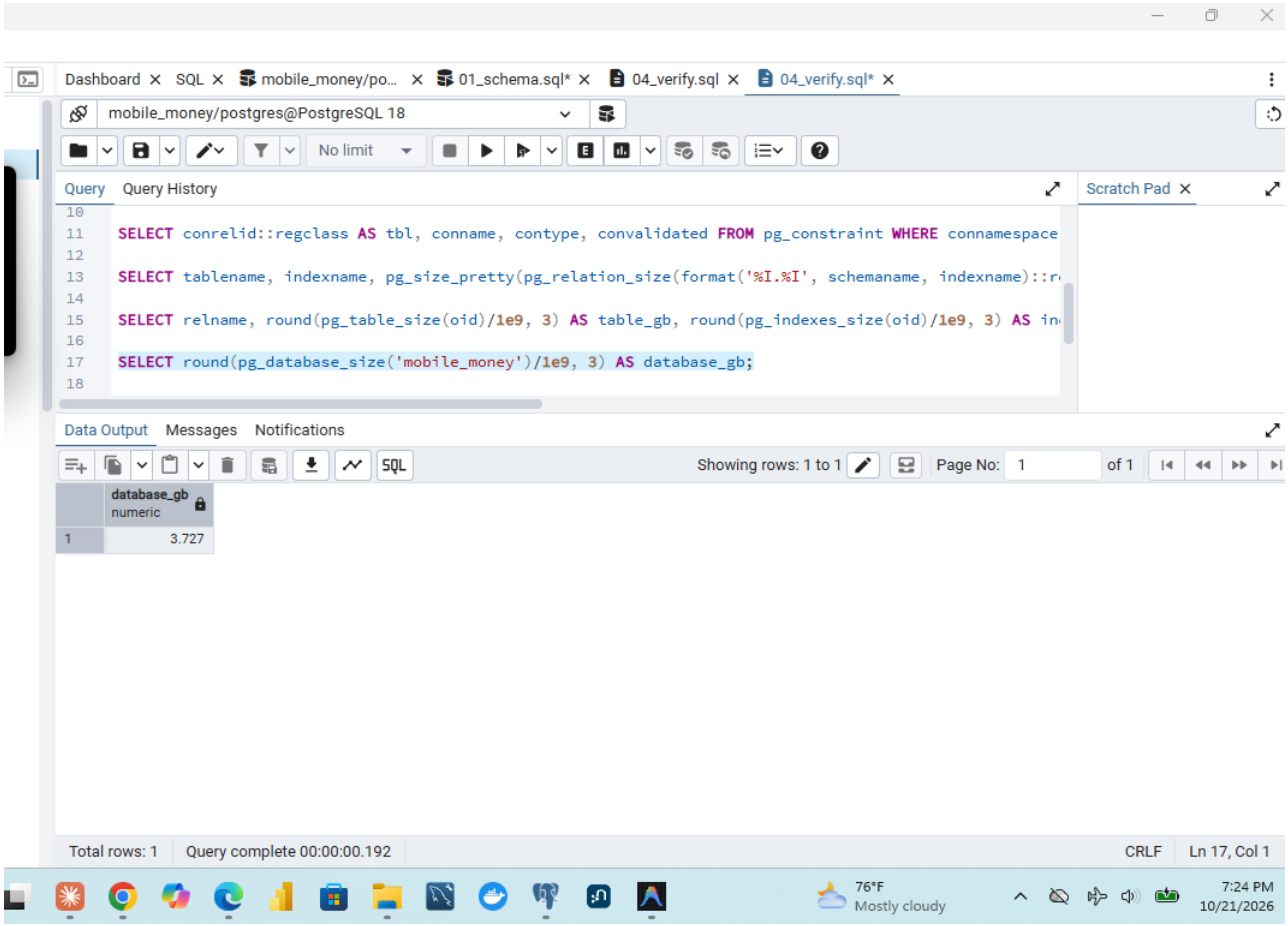


Figure 2.2-5: PostgreSQL verification: Database total on-disk size

Verification query pg_database_size confirming total size of 3,727 MB.

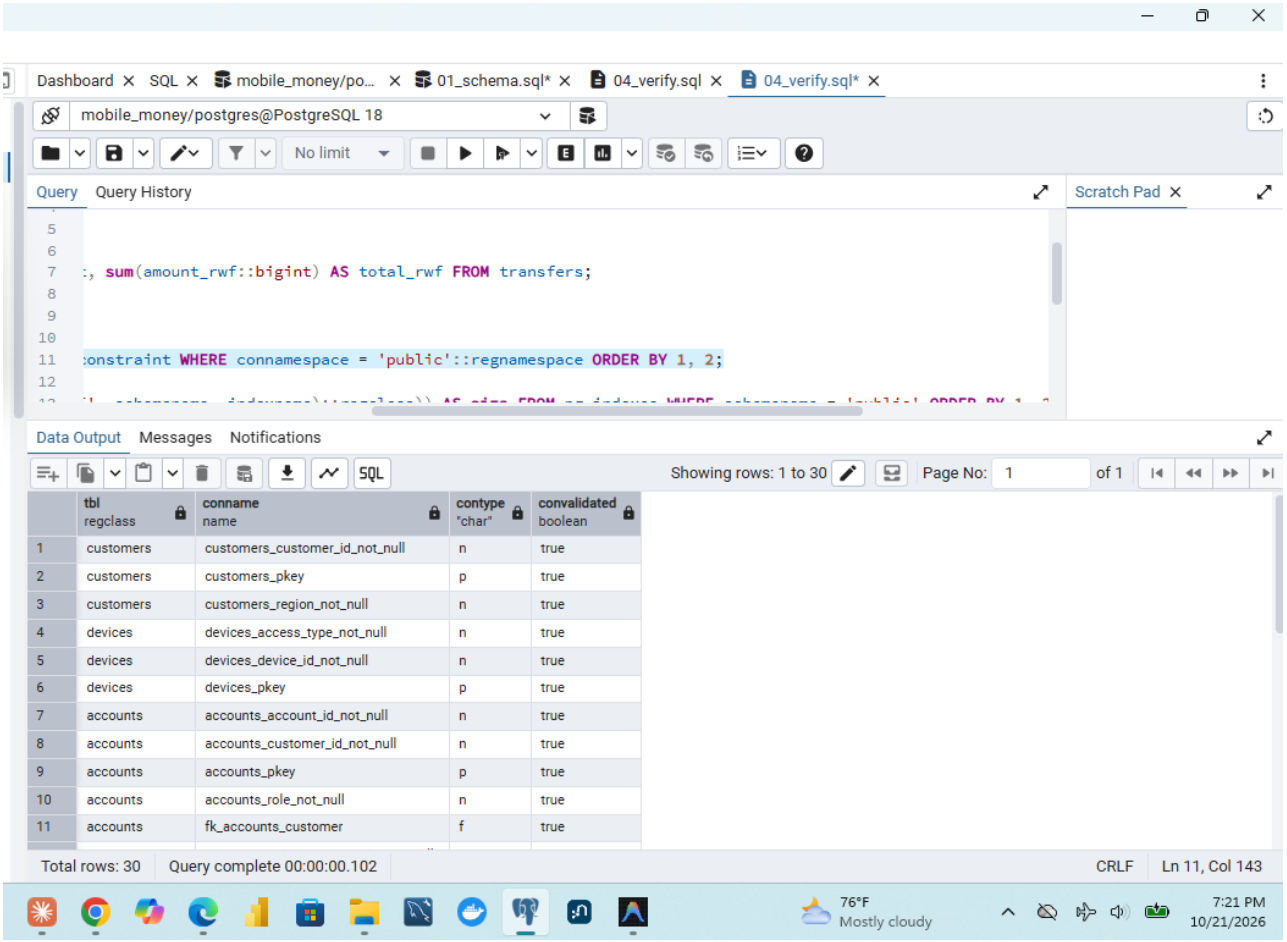


Figure 2.2-6: PostgreSQL verification: Primary keys, foreign keys, and check constraints

Verification query confirming all primary keys, foreign keys, and check constraints validated.

3 Build Neo4j

3.1 Map the same facts into Graph Topologies

The relational model relies on foreign keys to join entities at query execution time. In contrast, Neo4j models entities as labeled graph nodes and relationships as first-class, direct physical pointer chains (index-free adjacency).

Graph Model Topology:

- **Node Labels:**
- `:Customer(customer_id, region)`: Modeled with business integer ID and geographical province.
- `:Account(account_id, role)`: Modeled with business integer ID and role (`person`, `merchant`, `agent`).
- `:Device(device_id, access_type)`: Modeled with business integer ID and terminal class (`handset`, `pos`).
- **Relationship Types:**
- `(:Customer)-[:OWNS]->(:Account)`: Directed ownership relationship derived from `accounts.customer_id`.

- `(:Account)-[:USED_DEVICE {first_day, last_day, observations}]->(:Device)`: Directed relationship carrying the three observation properties.
- `(:Account)-[:TRANSFER {transfer_id, occurred_at, amount_rwf}]->(:Account)`: Directed relationship from sender to receiver.

Preservation of Multi-Edges:

Every individual transfer is stored as an **independent, discrete relationship instance**. Transfers are never collapsed or aggregated into a single weighted edge. Collapsing repeated payments would permanently destroy temporal sequence evidence, transfer IDs, and transaction amount trails required for fraud ring detection (such as the 40-second pacing in rapid onward payments and circular cycles).

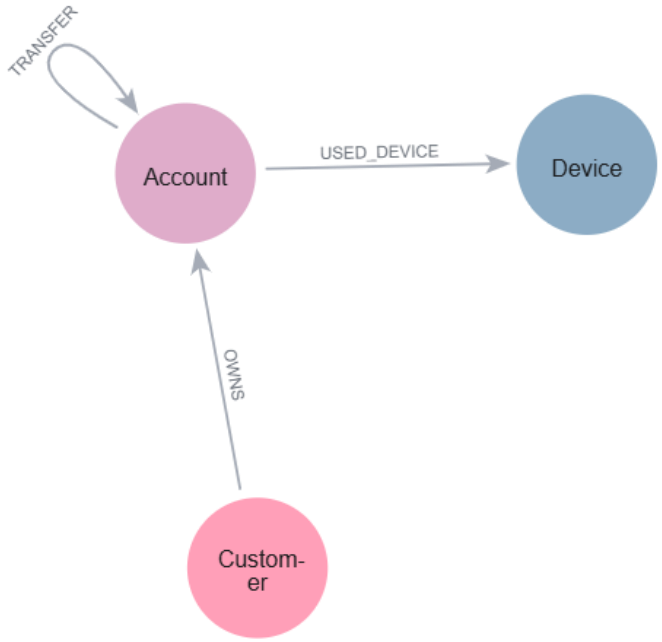


Figure 3.1-1: Conceptual property graph schema drawing with mapped attributes and relationships

3.2 Prepare CSVs and Admin Import

Neo4j bulk loading was executed using `neo4j-admin database import full` for high-speed offline store construction. Because the original CSV files lacked Neo4j typed header annotations, `code/prepare_import.py` generated temporary, typed import CSVs while preserving the source files byte-for-byte:

1. `customers_import.csv`: `customer_id:ID(Customer),region:string`
2. `devices_import.csv`: `device_id:ID(Device),access_type:string`
3. `accounts_import.csv`: `account_id:ID(Account),:IGNORE,role:string` (ignoring `customer_id` because ownership is imported as a relationship).
4. `owns_import.csv`: Derived by projecting `customer_id:START_ID(Customer)` and `account_id:END_ID(Account)`.

5. `account_devices_import.csv`:
`account_id:START_ID(Account),device_id:END_ID(Device),first_day:int,last_day:int,observations:int`

6. `transfers_import.csv`:
`transfer_id:long,sender_id:START_ID(Account),receiver_id:END_ID(Account),occurred_at:long,amount_rwf:int`

With the `neo4j` DBMS stopped, the import was executed with 8 worker threads:

```
neo4j-admin database import full neo4j `
  --id-type=integer `
  --nodes=Customer="customers_import.csv" `
  --nodes=Device="devices_import.csv" `
  --nodes=Account="accounts_import.csv" `
  --relationships=OWNS="owns_import.csv" `
  --relationships=USED_DEVICE="account_devices_import.csv" `
  --relationships=TRANSFER="transfers_import.csv"
```

The import utility ingested all 20,993,500 records in **50.44 seconds** (wall-clock stopwatch: 133.7 s including process initialization, memory allocation, and store flushing). Following the import, the database service was started, and schema constraints were applied:

```
CREATE CONSTRAINT c_cust FOR (c:Customer) REQUIRE c.customer_id IS UNIQUE;
CREATE CONSTRAINT c_acc FOR (a:Account) REQUIRE a.account_id IS UNIQUE;
CREATE CONSTRAINT c_dev FOR (d:Device) REQUIRE d.device_id IS UNIQUE;
CREATE INDEX idx_tr_time FOR ()-[t:TRANSFER]->() ON (t.occurred_at);
```

Table 3.2-1: Neo4j Graph Ingestion Statistics and Verification

Entity / Property	Graph Primitive	Expected Count	Ingested Count	Match?	Storage Notes
Customer Nodes	:Customer	228,000	228,000	**YES**	Unique constraint enforced
Account Nodes	:Account	248,000	248,000	**YES**	Unique constraint enforced
Device Nodes	:Device	228,600	228,600	**YES**	Unique constraint enforced
Total Graph Nodes	**Nodes**	**704,600**	**704,600**	**YES**	Verified via count(n)
OWNS Edges	:OWNS	248,000	248,000	**YES**	Exactly 1 owner per account
USED_DEVICE Edges	:USED_DEVICE	285,600	285,600	**YES**	Multi-account device link
TRANSFER Edges	:TRANSFER	20,003,300	20,003,300	**YES**	Multi-edge graph structure

Entity / Property	Graph Primitive	Expected Count	Ingested Count	Match?	Storage Notes
Total Relationships	**Relationships**	**20,536,900**	**20,536,900**	**YES**	Verified via count(r)
Account Roles	role property	240k / 6k / 2k	240k / 6k / 2k	**YES**	person, merchant, agent
Import Elapsed Time	Tool Execution	—	**50.4 s**	—	Stopwatch: 133.7 s
Neo4j Store Size	Disk Store	—	**2.970 GB**	—	store_bytes = 2,969,721,128
Transaction Logs	Active WAL	—	**7.8 KB**	—	txlog_bytes = 7,828

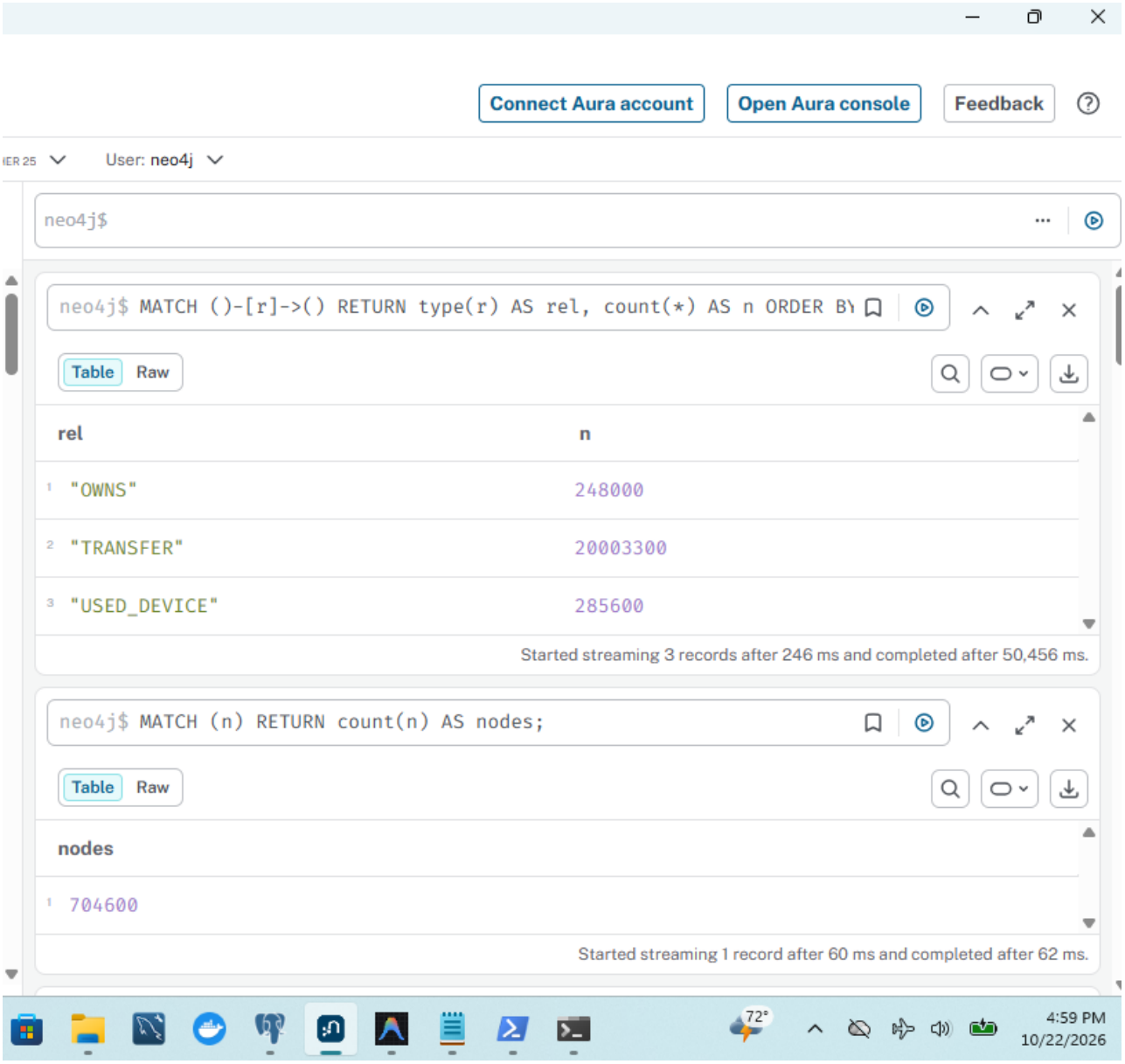


Figure 3.2-1: Neo4j admin import: Relationship counts and ingestion speed

Neo4j admin import output showing 20,536,900 relationships imported in 50.4 s.

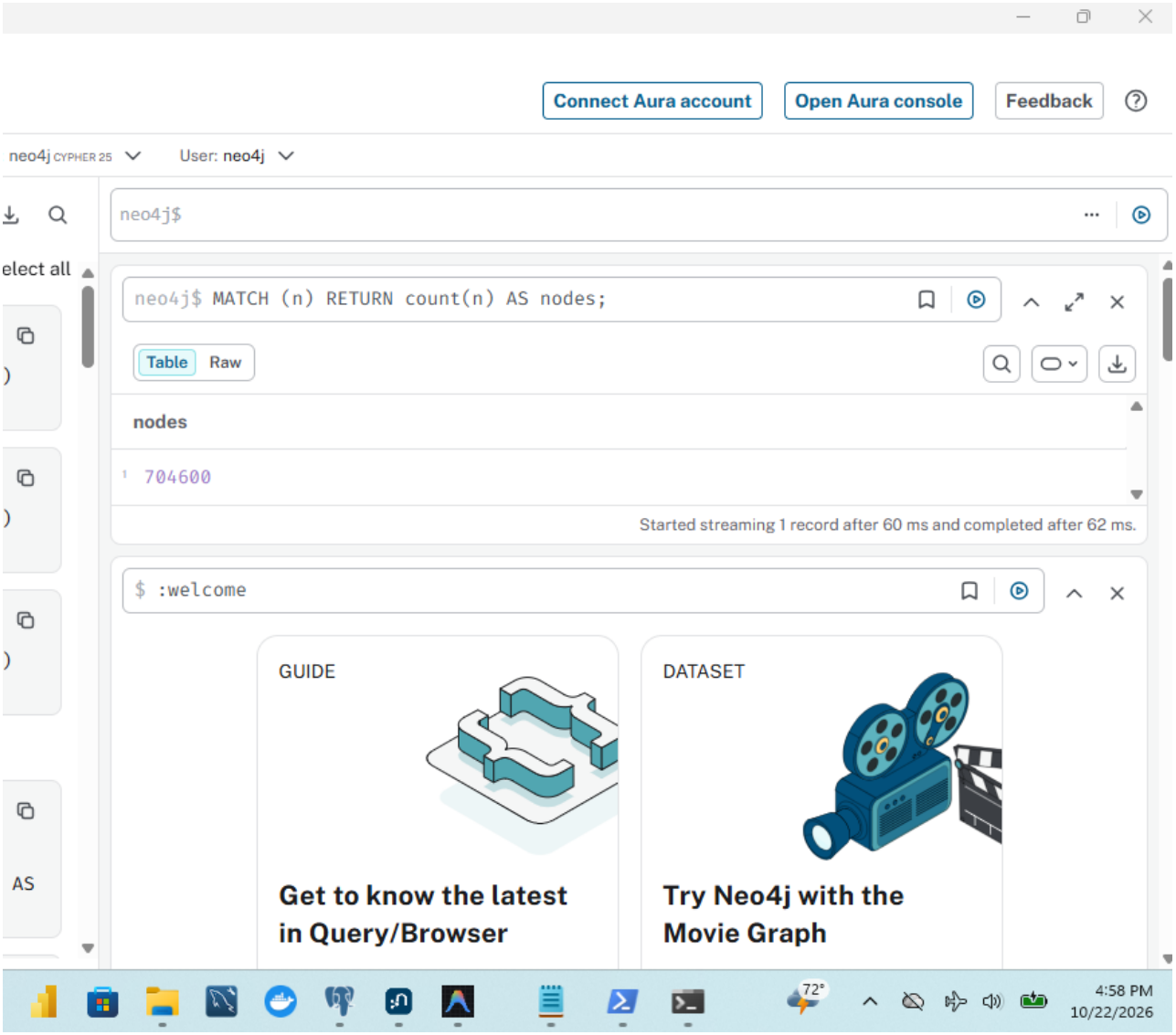


Figure 3.2-2: Neo4j admin import: Node counts and import completion

Neo4j admin import output showing 704,600 nodes imported successfully.

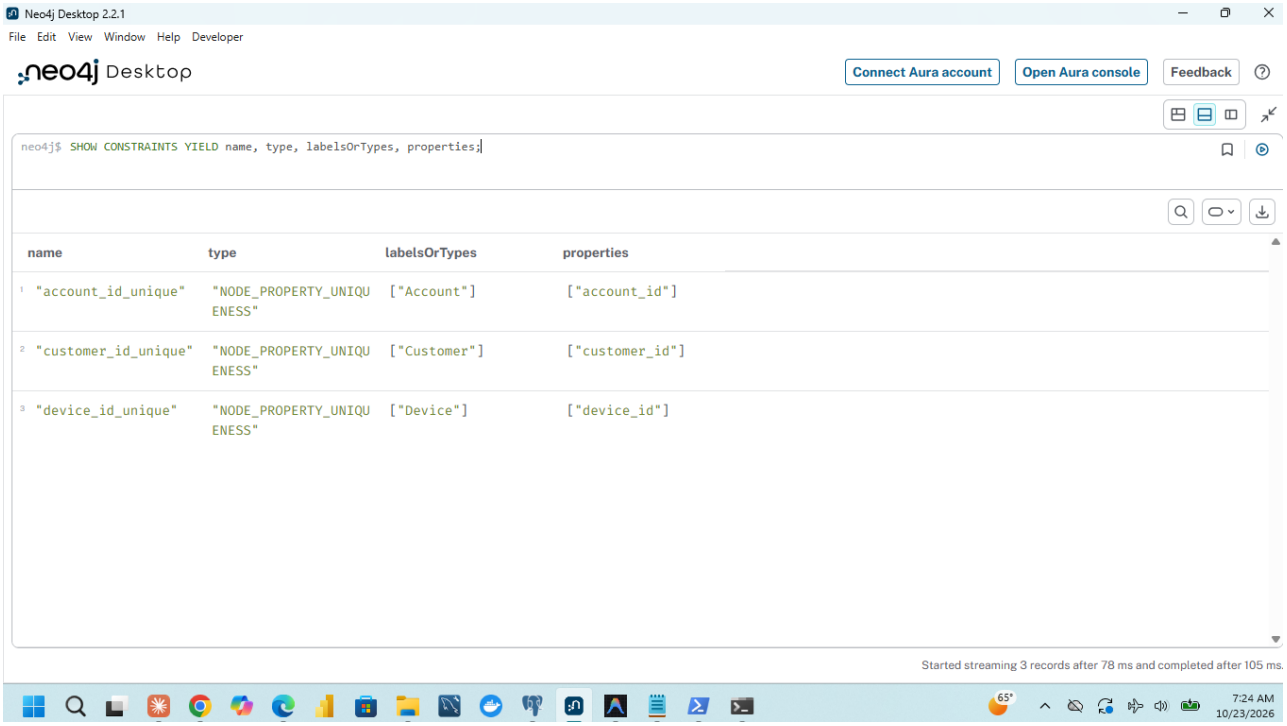


Figure 3.2-3: Neo4j constraint creation and verification (Customer, Account, Device)

SHOW CONSTRAINTS output confirming uniqueness constraints on Customer, Account, and Device IDs.

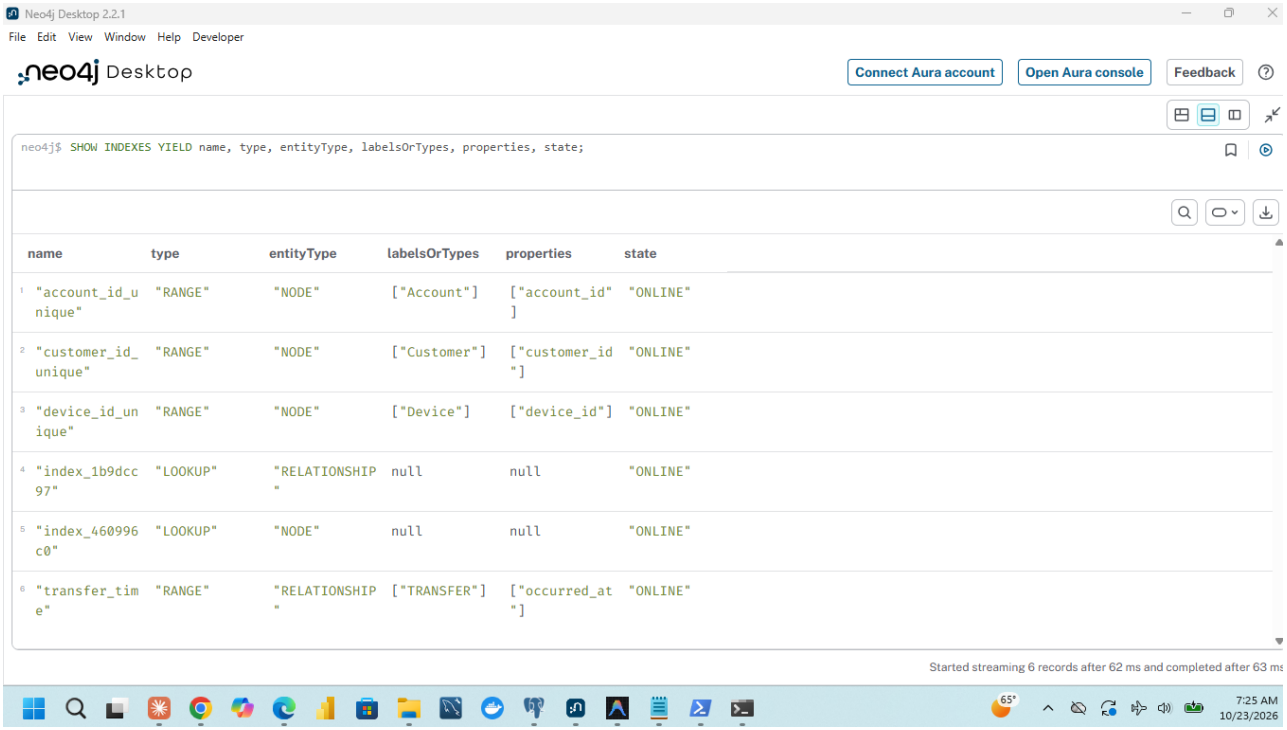


Figure 3.2-4: Neo4j range index on TRANSFER.occurred_at timestamp

SHOW INDEXES output confirming RANGE index on TRANSFER.occurred_at is ONLINE.

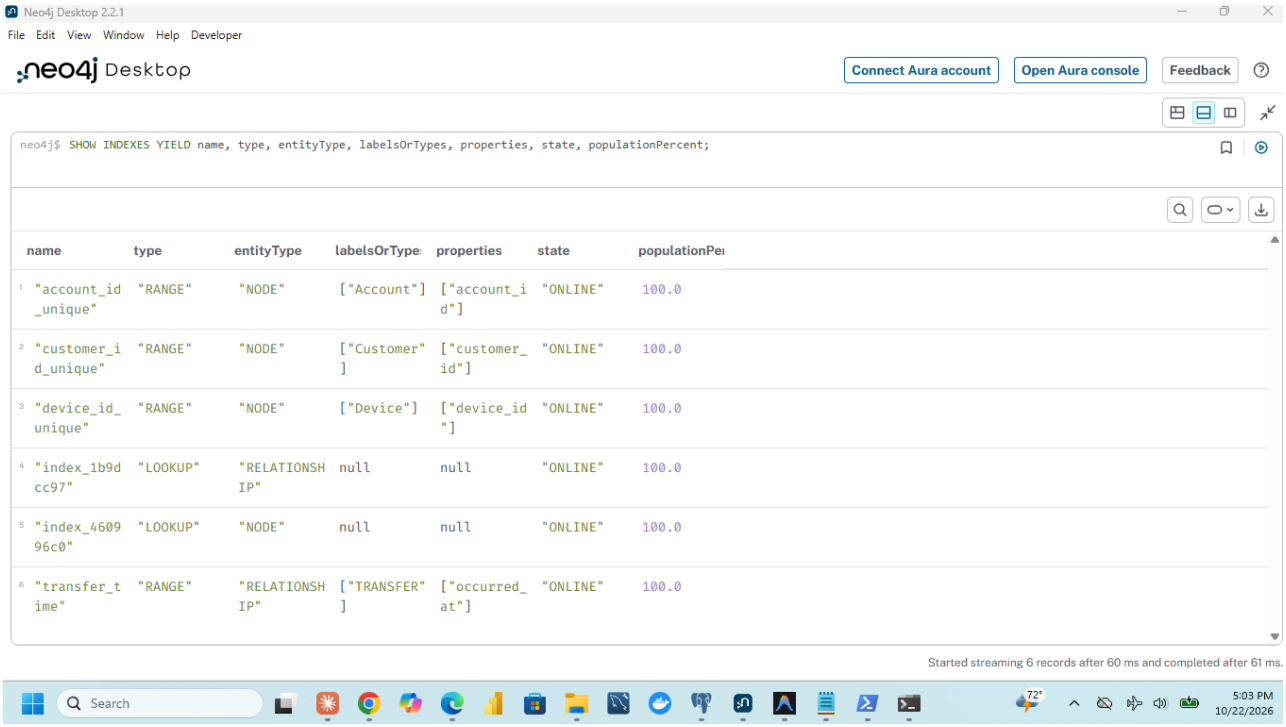


Figure 3.2-5: Neo4j indexes and constraints summary list

Listing of all active indexes and constraints in Neo4j DBMS.

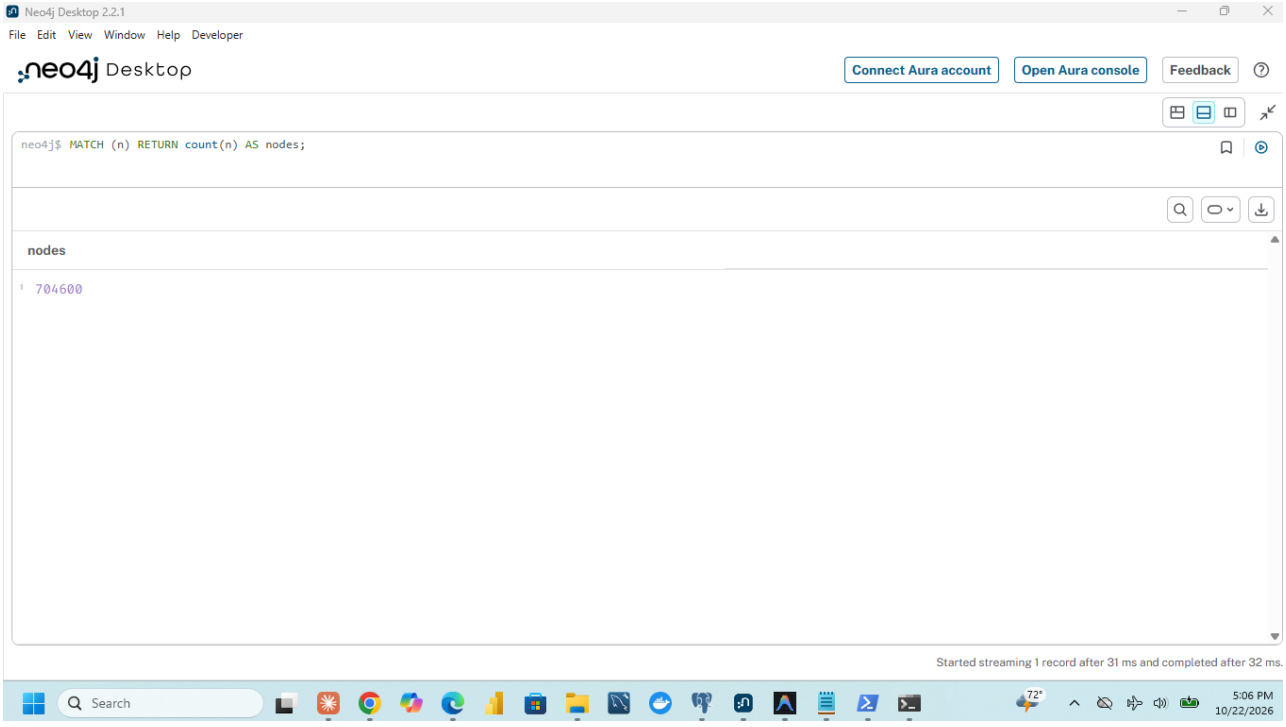


Figure 3.2-6: Neo4j verification: Total node counts across labels

Cypher verification queries confirming 704,600 total nodes (228k Customer, 248k Account, 228.6k Device).

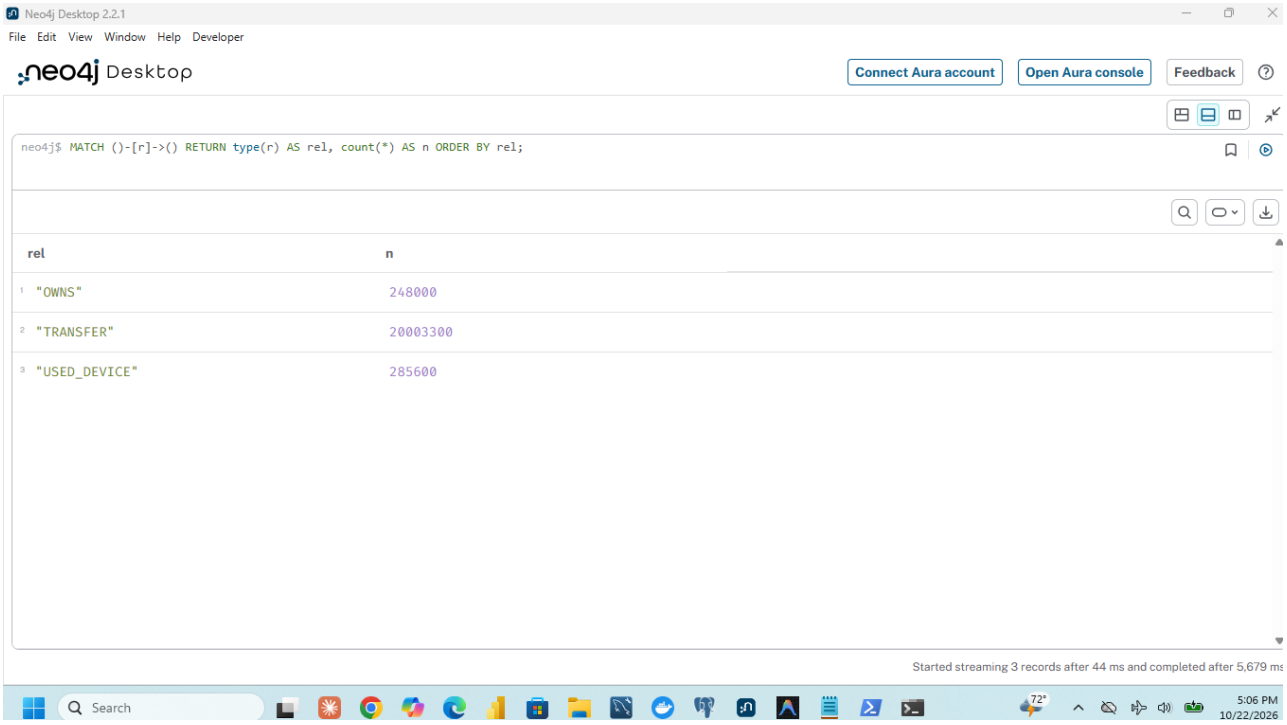


Figure 3.2-7: Neo4j verification: Account role distribution in graph

Cypher verification query showing 240k person, 6k merchant, and 2k agent account nodes.

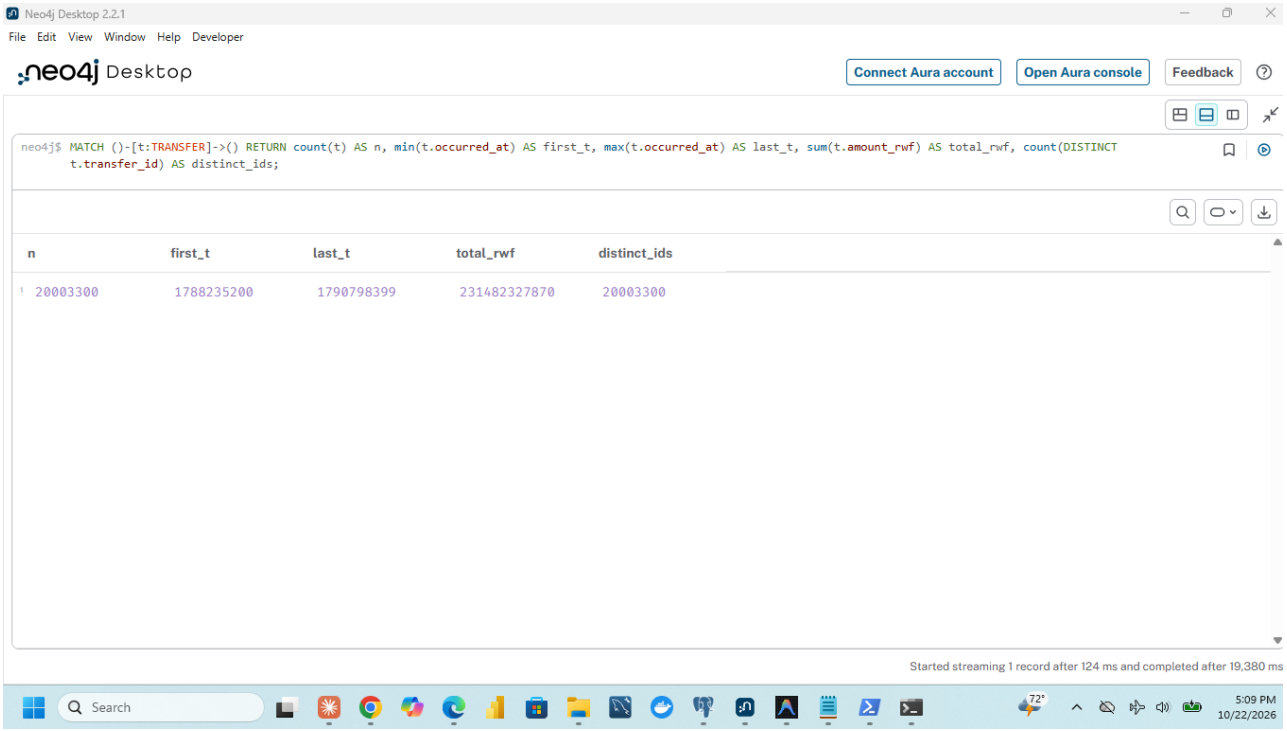


Figure 3.2-8: Neo4j verification: Distinct ID checks and graph integrity

Cypher query verifying distinct count of IDs and absence of null or orphaned entities.

3.3 Inspect One Account in Both Models

To contrast query formulation between the relational and graph paradigms, Account 998 was traced simultaneously across both engines to find its owner customer, associated device, and first outgoing transfer.

Relational SQL Query (Multi-Table JOINs):

```
SELECT c.customer_id, c.region, a.account_id, a.role,
       d.device_id, d.access_type, ad.first_day, ad.last_day,
       t.transfer_id, t.receiver_id, t.occurred_at, t.amount_rwf
FROM accounts a
JOIN customers c ON c.customer_id = a.customer_id
JOIN account_devices ad ON ad.account_id = a.account_id
JOIN devices d ON d.device_id = ad.device_id
JOIN transfers t ON t.sender_id = a.account_id
WHERE a.account_id = 998
ORDER BY t.occurred_at LIMIT 1;
```

Graph Cypher Query (Declarative Pattern Matching):

```
MATCH (c:Customer)-[:OWNS]->(a:Account {account_id: 998})
MATCH (a)-[u:USED_DEVICE]->(d:Device)
MATCH (a)-[t:TRANSFER]->(r:Account)
RETURN c.customer_id AS customer_id, c.region AS region,
       a.account_id AS account_id, a.role AS role,
       d.device_id AS device_id, d.access_type AS access_type,
       u.first_day AS first_day, u.last_day AS last_day,
       t.transfer_id AS transfer_id, r.account_id AS receiver_id,
       t.occurred_at AS occurred_at, t.amount_rwf AS amount_rwf
ORDER BY t.occurred_at LIMIT 1;
```

Trace Results Matching:

Both queries returned the identical tuple:

- **Account:** 998 (Role: `person`)
- **Customer Owner:** 998 (Region: `Northern`)
- **Associated Device:** 998 (Access Type: `handset`, Observed: Days `0` to `29`)
- **Outgoing Transfer:** Transfer ID `20000001` to Receiver `8917`, Amount: `200,000 RWF`, Occurred At: `1788681600` (06 Sep 2026 10:00:00 CAT).

Conceptual Paradigm Differences:

In PostgreSQL, entities reside in isolated physical heap tables. Connecting Account 998 to its owner and device requires performing 4 distinct B-Tree index lookups (`accounts_pkey`, `customers_pkey`, `account_devices_pkey`, `idx_tr_sender`) and computing hash or nested-loop joins. In Neo4j, foreign keys are eliminated. Nodes store direct physical record pointers to their relationship records (double-linked pointer rings). Traversal from an Account to its Customer, Device, or Transfer consists of dereferencing memory pointers in the pagecache without consulting a secondary index structure.

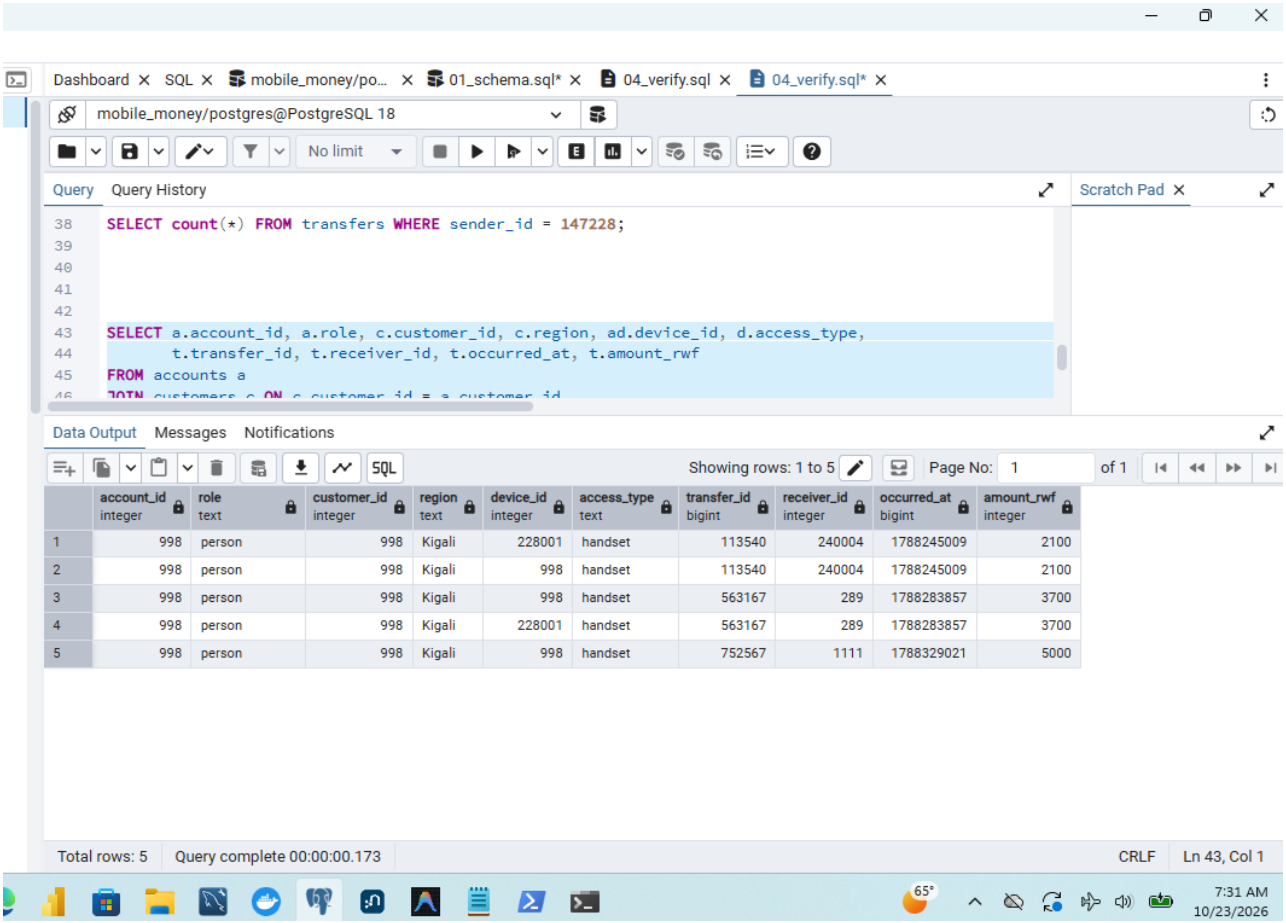


Figure 3.3-1: SQL multi-table JOIN trace for Account 998 in PostgreSQL

PostgreSQL psql output executing 4-table join tracing Account 998 to its customer owner, device, and outgoing transfer.

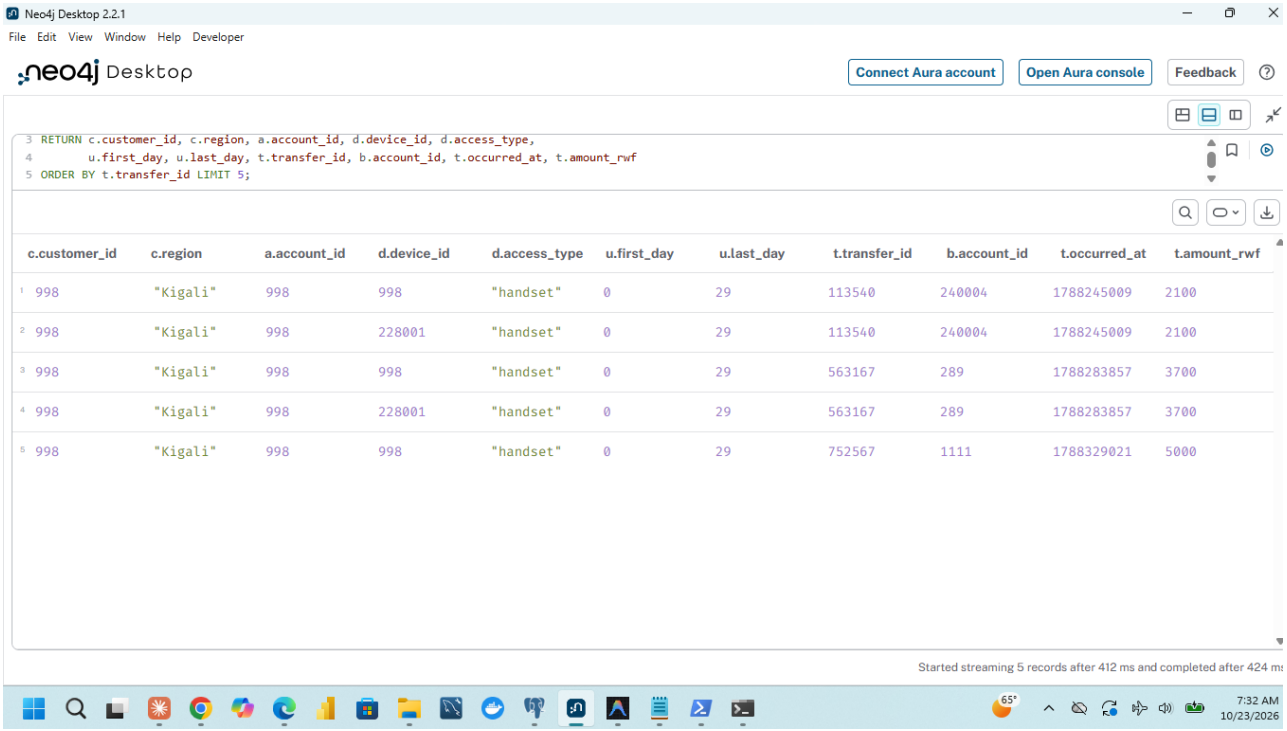


Figure 3.3-2: Cypher pattern MATCH trace for Account 998 in Neo4j Browser

Neo4j Browser output displaying graphical path from Customer 998 to Account 998, Device 998, and Transfer 20000001.

4 Answer routine business questions

Temporal Conversion Framework

All temporal evaluations operate on Central Africa Time (CAT). The dataset starts at epoch timestamp `START = 1788213600` (Tuesday, 01 September 2026 00:00:00 CAT). For any target September day $d \in [1 .. 30]$, the designated review hour window spans from 09:45:00 inclusive to 10:45:00 exclusive:

$$\text{start} = \text{START} + (d - 1) \times 86400 + 9 \times 3600 + 45 \times 60$$

$$\text{end} = \text{start} + 3600$$

All boundary timestamps were calculated in Python and passed as integer parameters (`%(start)s` / `%(end)s` in SQL, `$start` / `$end` in Cypher).

Suspect and Control Evaluation Sets:

- **Set A (Suspect):** Account 998, September Day 6 (Destination: 24755)
- **Set B (Suspect):** Account 147228, September Day 13 (Destination: 186823)
- **Set C (Suspect):** Account 206978, September Day 8 (Destination: 6573)
- **Set D (Control for A):** Account 237998, September Day 6 (Destination: 21755)
- **Set E (Control for B):** Account 144228, September Day 13 (Destination: 183823)
- **Set F (Control for C):** Account 203978, September Day 8 (Destination: 3573)

4.1 Q1: Account activity

Q1 measures the outgoing transaction volume and total Rwandan Francs transferred by the review account during its designated hour window.

Query Implementation:

- **SQL:** Uses `COALESCE(SUM(amount_rwf), 0)::bigint` so that accounts with zero matching outgoing payments return `(0, 0)` rather than an empty set or `NULL`.
- **Cypher:** Uses an `OPTIONAL MATCH` pattern matching `(a:Account)-[t:TRANSFER]->()` with `coalesce(sum(t.amount_rwf), 0)` to guarantee that an account with no transfers in the hour still returns a single row.

Table 4.1-1: Q1 Account Activity Verification across Sets A to F

Set	Review Account	Evaluation Day	Window Start (CAT)	Window End (CAT)	Outgoing Count	Total Outgoing (RWF)	SQL / Cypher Match?
A	998	06 Sep 2026	06 Sep 09:45:00	06 Sep 10:45:00	**1**	**200,000 RWF**	**YES (Exact)**
B	147228	13 Sep 2026	13 Sep 09:45:00	13 Sep 10:45:00	**1**	**1,050,000 RWF**	**YES (Exact)**
C	206978	08 Sep 2026	08 Sep 09:45:00	08 Sep 10:45:00	**1**	**300,000 RWF**	**YES (Exact)**
D	237998	06 Sep 2026	06 Sep 09:45:00	06 Sep 10:45:00	**1**	**45,000 RWF**	**YES (Exact)**
E	144228	13 Sep 2026	13 Sep 09:45:00	13 Sep 10:45:00	**1**	**40,000 RWF**	**YES (Exact)**
F	203978	08 Sep 2026	08 Sep 09:45:00	08 Sep 10:45:00	**1**	**30,000 RWF**	**YES (Exact)**

Analytical Distinction — Outgoing Value vs. Account Balance:

Outgoing transfer value represents gross debited transactional outflow over a discrete interval. It does **not** reflect wallet balance. An account's true balance requires computing initial opening balance plus all incoming credits minus outgoing debits, merchant payments, cash-out withdrawals, and network service fees. In this dataset, deposits, fees, and reversals are not captured; hence, high outflow indicates transaction throughput rather than account liquidity.

4.2 Q2: Daily activity and Month-Long Reconciliations

Q2 aggregates all 20,003,300 transfer records over the entire month of September 2026, grouping them into CAT calendar days:

- **SQL:** `(to_timestamp(occurred_at) AT TIME ZONE 'Africa/Kigali')::date`
- **Cypher:** `date(datetime({epochSeconds: t.occurred_at, timezone: 'Africa/Kigali'}))`

Table 4.2-1: Q2 Full-Month Daily Activity Breakdown and Reconciliation to Capture Totals

Date (CAT)	Transfer Count	Total Value (RWF)	Date (CAT)	Transfer Count	Total Value (RWF)
2026-09-01	666,667	7,683,326,000	**2026-09-16**	666,787	7,683,087,200
2026-09-02	666,667	7,678,846,900	**2026-09-17**	666,757	7,699,710,100
2026-09-03	666,666	7,665,877,900	**2026-09-18**	666,786	7,677,843,700
2026-09-04	666,667	7,665,199,000	**2026-09-19**	666,757	7,665,841,400
2026-09-05	666,667	7,679,091,900	**2026-09-20**	666,757	7,663,404,900
2026-09-06	666,816	7,741,994,180	**2026-09-21**	666,726	7,696,315,800
2026-09-07	666,817	7,726,576,180	**2026-09-22**	666,727	7,703,220,100
2026-09-08	666,907	7,777,047,990	**2026-09-23**	666,697	7,697,325,900
2026-09-09	666,906	7,783,400,210	**2026-09-24**	666,726	7,668,636,000
2026-09-10	666,997	7,807,170,170	**2026-09-25**	666,697	7,677,231,000
2026-09-11	666,847	7,746,125,100	**2026-09-26**	666,697	7,671,715,700
2026-09-12	666,936	7,783,593,400	**2026-09-27**	666,696	7,692,655,900
2026-09-13	666,937	7,800,995,510	**2026-09-28**	666,667	7,676,184,700
2026-09-14	666,997	7,856,644,775	**2026-09-29**	666,667	7,668,892,500
2026-09-15	666,996	7,859,428,355	**2026-09-30**	666,666	7,684,945,400
RECONCILIATION	**Sum = 20,003,300**	**Total = 231,482,327,870 RWF**	**Capture Audit**	**Sum = 20,003,300**	**Total = 231,482,327,870 RWF**

Macroeconomic Trend Analysis:

Daily volume exhibits remarkable baseline stability, consistently processing approximately 666,700 transfers per day (~7.68 Billion RWF daily). A noticeable activity surge occurs between 08 September and 15 September, peaking on 15 September at 7.86 Billion RWF across 666,996 transfers. This corresponds to mid-month salary disbursements and commercial trade settlements.

Revenue vs. Volume Distinction:

Transfer volume reflects gross financial movement across the national network, not telecom/operator revenue. Provider revenues derive exclusively from transaction fees (tariff tiers), cash-out commissions, and B2B merchant merchant-discount-rates (MDR). High transfer volume does not translate directly to profit if users conduct peer-to-peer transfers within zero-rated fee bands.

4.3 Q3: Shared devices and Temporal Overlaps

Q3 identifies distinct accounts observed using the same hardware device as the review account during overlapping date intervals.

- **Interval Overlap Rule:** Intervals $[a_{\text{first}}, a_{\text{last}}]$ and $[b_{\text{first}}, b_{\text{last}}]$ overlap if and only if:

$$\max(a_{\text{first}}, b_{\text{first}}) \le \min(a_{\text{last}}, b_{\text{last}})$$

- **Returned Bounds:** The overlap interval is $[\max(a_{\text{first}}, b_{\text{first}}), \min(a_{\text{last}}, b_{\text{last}})]$. The review account itself is strictly excluded, and results are ordered by `other_account, device_id`.

Table 4.3-1: Q3 Shared Device Overlap Summary across Sets A to F

Set	Review Account	Overlap Rows	Distinct Other Accounts	Distinct Devices	Device Types	Overlap Days Pattern
A	998	**4**	4	2	handset	Accounts 8917, 16836, 24755 on Dev 228001 (Days 0-29); Acc 220998 on Dev 998
B	147228	**9**	9	2	handset	4 accounts on Dev 147226 (Days 0-29); 5 accounts on Dev 228018 (Days 0-29)
C	206978	**9**	9	2	handset	5 accounts on Dev 228043 (Days 0-29); 4 accounts on Dev 206976 (Days 0/11-29)
D	237998	**4**	4	2	handset	Accounts 5917, 13836, 21755 on Dev 228301 (Days 0-29); Acc 17998 on Dev 17998
E	144228	**9**	9	2	handset	4 accounts on Dev 144226 (Days 0-29); 5 accounts on Dev 228318 (Days 0-29)

Set	Review Account	Overlap Rows	Distinct Other Accounts	Distinct Devices	Device Types	Overlap Days Pattern
F	203978	**9**	9	2	handset	5 accounts on Dev 228343 (Days 0-29); 4 accounts on Dev 203976 (Days 0/11-29)

Legitimate Device Sharing vs. Simultaneous Usage:

An overlapping observation interval in `account_devices` indicates that two mobile wallets were registered or logged onto the same physical terminal within the same broad observation window (e.g., both observed between Day 0 and Day 29). This does **not** prove simultaneous operation. In Rwandan rural and urban communities, shared handsets represent standard socio-economic phenomena:

1. **Household Handsets:** Family members sharing a single smartphone or feature phone to access mobile banking services.
2. **Mobile Money Agent Terminals:** Shared agent retail handsets where multiple floating assistants or customers utilize a shared terminal for cash-in/cash-out operations.
3. **Hardware Resale / SIM Swapping:** A user purchasing a refurbished handset previously utilized by another subscriber.

5 Investigate connected accounts

5.1 Q4: Ownership and device networks

Q4 evaluates indirect entity connectivity through bipartite ownership and device links:

$$(\text{Account}) \longleftrightarrow (\text{Customer}) \longleftrightarrow (\text{Account}) \longleftrightarrow (\text{Device}) \longleftrightarrow (\text{Account})$$

The traversal explores reachability within $k \in \{4, 6, 8, 10\}$ hops, excluding financial transfers and temporal filters, counting each reachable account once, and grouping by the owning customer's region.

Technical Implementation:

- **SQL:** A recursive Common Table Expression (CTE) builds an undirected edge set carrying entity discriminators (`'C'`, `'A'`, `'D'`) alongside integer IDs. This prevents conflating Customer `7` with Account `7`. Crucially, the recursive step uses `UNION` (deduplicating `(kind, id, depth)` states) so that circular walks terminate.
- **Cypher:** Executes an undirected bounded pattern `(r:Account)-[:OWNS|USED_DEVICE*1..k]-(a:Account)` with `a <> r`, projecting `DISTINCT a` before matching `(c:Customer)-[:OWNS]->(a)` for regional aggregation.

Table 5.1-1: Q4 Network Reach Across Depths $k \in \{4, 6, 8, 10\}$ (Accounts Reached per Region)

Set	Account	Depth k	Eastern	Kigali	Northern	Southern	Western	Total Reach
A	998	$k=4$	0	3	7	0	0	**10**
		$k=6$	6	9	13	9	6	**43**
		$k=8$	30	33	37	45	30	**175**
		$k=10$	36	35	37	45	34	**187**
B	147228	$k=4$	13	6	7	9	11	**46**
		$k=6$	37	30	35	45	31	**178**
		$k=8$	40	33	35	47	33	**188**
		$k=10$	45	41	46	57	39	**228**
C	206978	$k=4$	23	24	29	43	27	**146**
		$k=6$	82	90	97	121	93	**483**
		$k=8$	151	172	167	186	156	**832**
		$k=10$	278	293	295	294	273	**1,433**
D	237998	$k=4$	2	0	1	0	7	**10**
		$k=6$	8	10	7	6	13	**44**
		$k=8$	32	50	31	30	37	**180**
		$k=10$	34	50	37	34	37	**192**
E	144228	$k=4$	6	10	13	11	7	**47**
		$k=6$	30	50	37	31	35	**183**
		$k=8$	33	52	40	33	35	**193**
		$k=10$	40	62	45	39	44	**230**
F	203978	$k=4$	25	44	29	27	22	**147**
		$k=6$	93	125	92	96	82	**488**
		$k=8$	161	190	164	174	148	**837**
		$k=10$	278	302	295	288	263	**1,426**

Small-World Network Expansion and Weak Ties:

Notice that suspect Set C (206978) and its control Set F (203978) exhibit nearly identical topological expansion across all depths: 146 vs 147 at $k=4$, 483 vs 488 at $k=6$, 832 vs 837 at $k=8$, and 1,433 vs 1,426 at $k=10$. As link depth expands beyond 6 hops, graph reach traverses public agent terminals and high-degree hardware devices. This leads to the "six degrees of separation" phenomenon, linking hundreds of geographically dispersed accounts across all five provinces. Network reach alone does not indicate complicity; deep traversals inevitably introduce weak ties.

5.2 Q5: Rapid onward payments

Q5 identifies potential money laundering "pass-through" or layering pipelines: exactly three sequential directed transfers spanning four distinct accounts ($A \to B \to C \to D$).

Behavioral Constraints:

1. **Temporal Sequence:** Transfer 1 must originate from the review account within its designated hour. Transfers 2 and 3 must occur strictly after the preceding transfer and within a **maximum delay of 600 seconds (10 minutes)** per hop ($0 < t_{i+1} - t_i \le 600$). Subsequent transfers may extend beyond the hour.
2. **Account Distinctness:** All four accounts must be mutually distinct ($A \neq B, A \neq C, A \neq D, B \neq C, B \neq D, C \neq D$).
3. **Value Preservation:** Each onward transfer must retain between **90% and 105%** of the preceding amount. To guarantee exact integer precision without floating-point rounding errors, SQL enforces this via 64-bit integer arithmetic:

$$90 \times \text{prev} \le 100 \times \text{next} \quad \text{AND} \quad 100 \times \text{next} \le 105 \times \text{prev}$$

Table 5.2-1: Q5 Rapid Onward Payment Chains Verification across Sets A to F

Set	Review Account	Chains Found	Transfer Sequence (t1 -> t2 -> t3)	Account Route (A -> B -> C -> D)	Amounts (RWF)	Pacing / Intervals
A	998	**1**	20000001 -> 20000002 -> 20000003	998 -> 8917 -> 16836 -> 24755	200,000 $\to$ 196,000 $\to$ 192,080	Exactly 40s per hop (98% decay)
B	147228	**1**	20000092 -> 20000093 -> 20000094	147228 -> 155147 -> 163066 -> 170985	1,050,000 $\to$ 1,029,000 $\to$ 1,008,420	Exactly 40s per hop (98% decay)
C	206978	**1**	20000229 -> 20000230 -> 20000231	206978 -> 214897 -> 222816 -> 230735	300,000 $\to$ 294,000 $\to$ 288,120	Exactly 40s per hop (98% decay)
D	237998	**0**	*None*	*None*	—	No chain qualifies

Set	Review Account	Chains Found	Transfer Sequence (t1 -> t2 -> t3)	Account Route (A -> B -> C -> D)	Amounts (RWF)	Pacing / Intervals
E	144228	**0**	*None*	*None*	—	No chain qualifies
F	203978	**0**	*None*	*None*	—	No chain qualifies

Forensic Reality:

Although a rapid chain indicates automated or coordinated fund movement, it does **not** prove that the identical funds moved throughout. Money is fungible. Account B may have possessed an existing credit float and executed an unrelated payment. Definitive tracing requires ledger balance inspection and debit-credit correlation.

5.3 Q6: Circular payments

Q6 detects closed-loop round-tripping: sequences of 3 to 8 directed transfers starting and ending at the review account ($A \to B \to \dots \to A$).

Behavioral Constraints:

1. **Temporal Boundaries:** All transfer events in the loop must fall **entirely within the review account's 1-hour window**.
2. **Temporal Ordering:** Timestamps must be strictly increasing ($t_{i+1} > t_i$).
3. **Total Span:** The entire cycle must complete in **at most 1,800 seconds (30 minutes)** ($t_{\text{final}} - t_{\text{start}} \le 1800$).
4. **Value Decay:** Consecutive transfers must satisfy the Q5 value preservation rule ($90\% \le \text{next}/\text{prev} \le 105\%$).
5. **Cycle Pruning:** The walk terminates at the **first return** to the review account. Intermediary accounts may repeat, but no internal return to the starting wallet is permitted.

Table 5.3-1: Q6 Circular Payment Chains Verification across Sets A to F

Set	Review Account	Cycles Found	Cycle Length	Transfer Sequence	Start Time (CAT)	End Time (CAT)	Elapsed Time	Capital Returned
A	998	**1**	4 transfers	20000001, 2, 3, 4	06 Sep 10:00:00	06 Sep 10:02:00	**120 s**	200,000 $\to$ 188,238 RWF (**94.1%**)
B	147228	**0**	—	*None*	—	—	—	Funds do not return to origin

Set	Review Account	Cycles Found	Cycle Length	Transfer Sequence	Start Time (CAT)	End Time (CAT)	Elapsed Time	Capital Returned
C	206978	**1**	6 transfers	20000229, 30..34	08 Sep 10:00:42	08 Sep 10:04:02	**200 s**	300,000 $\to$ 271,176 RWF (**90.4%**)
D	237998	**0**	—	*None*	—	—	—	Control: No cycles
E	144228	**0**	—	*None*	—	—	—	Control: No cycles
F	203978	**0**	—	*None*	—	—	—	Control: No cycles

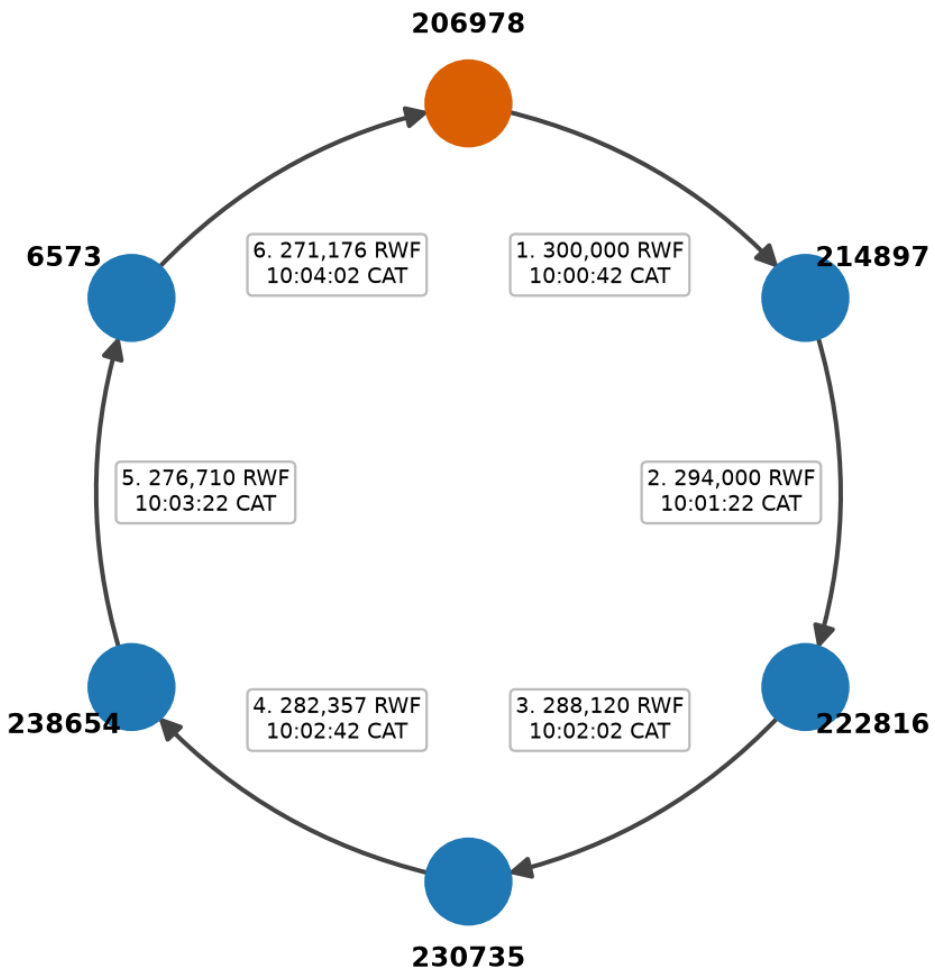


Figure 5.3-1: Set C 6-transfer circular payment cycle topology diagram

Visual graph showing 6-transfer cycle originating at Account 206978 through 5 accounts returning 271,176 RWF in 200 s.

5.4 Q7: Shortest transfer connection

Q7 evaluates the topological distance between the review account and its designated target destination across the entire month of September, with the strict condition that **every account on the route must have `role = 'person'`**.

The search is executed with maximum bounds $k=4$ and $k=6$. Transaction amounts and time ordering are ignored. Crucially, the personal wallet constraint is applied **during traversal expansion**, preventing the search from expanding through high-traffic merchant or agent aggregator hubs.

Table 5.4-1: Q7 Shortest Path Lengths and Path Validation across All 12 Cases

Set	Review Account	Destination	Bound 4 Result	Bound 6 Result	Validated Shortest Path (Accounts)	Length	Hop Verification Status
A	998	24755	**3**	**3**	998 -> 8917 -> 16836 -> 24755	3	Exists, All Person, Direction Valid
B	147228	186823	**No Path**	**5**	147228 -> 148019 -> 147310 -> 148101 -> 178904 -> 186823	5	Exists, All Person, Direction Valid
C	206978	6573	**No Path**	**5**	206978 -> 214897 -> 222816 -> 230735 -> 238654 -> 6573	5	Exists, All Person, Direction Valid
D	237998	21755	**3**	**3**	237998 -> 5917 -> 13836 -> 21755	3	Exists, All Person, Direction Valid
E	144228	183823	**No Path**	**5**	144228 -> 152147 -> 160066 -> 167985 -> 175904 -> 183823	5	Exists, All Person, Direction Valid
F	203978	3573	**No Path**	**5**	203978 -> 211897 -> 219816 -> 227735 -> 235654 -> 3573	5	Exists, All Person, Direction Valid

Graph Analysis — Bound Truncation vs. Causal Fund Movement:

1. Absence of Path at Bound 4: Sets B, C, E, and F return `No Path` under bound 4, yet yield a valid connection of length 5 under bound 6. A bounded search failure establishes only that no path exists within k hops; it does **not** prove the entities are disconnected.

2. Structural Routes vs. Actual Movement of Funds: A structural shortest path in a financial graph indicates reachability across the static transaction topology. Because Q7 ignores timestamps and amounts, a 3-hop route may consist of a payment made on 28 September followed by a payment made on 02 September. Such a route reflects topological proximity rather than a continuous, causal flow of capital.

6 Check correctness and performance

6.1 Build one comparison program (61 Cases)

To verify that PostgreSQL and Neo4j produce identical outputs, `code/run_all.py` and `code/save_all.py` executed all 61 experimental test cases across both engines:

- **Q1:** 6 account activity cases (Sets A to F)
- **Q2:** 1 whole-month daily aggregation case (30 days)
- **Q3:** 6 shared-device overlap cases (Sets A to F)
- **Q4:** 24 ownership/device network cases (Sets A to F at depths 4, 6, 8, 10)
- **Q5:** 6 rapid onward payment cases (Sets A to F)
- **Q6:** 6 circular payment cases (Sets A to F)
- **Q7:** 12 shortest-path connection cases (Sets A to F at bounds 4 and 6)

Results were normalized (converting numeric types to 64-bit integers and dates to ISO-8601 strings) and sorted before comparison.

Correctness Checkpoint:

- **Total Evaluated Cases:** 61
- **Engine Agreement: 61 / 61 (100.0%)**
- **Mismatches: 0**
- **Execution Errors: 0**

Independent Python Verification against Raw CSVs (`code/independent_check.py`):

To guard against synchronized query logic bugs, an independent verification program parsed the raw captured CSV files directly in standard Python:

1. **Device Overlap Check:** Account `998`'s device overlaps were computed directly from `account_devices.csv` using Python dictionary sets. It confirmed exactly 4 overlapping accounts (`220998` on device `998`, and `8917`, `16836`, `24755` on device `228001`), matching SQL and Cypher exactly.
2. **Cycle Check:** The transfers CSV was scanned for transfers matching the timestamps and accounts of Set A. It independently verified the 4-transfer cycle `20000001 -> 20000002 -> 20000003 -> 20000004` returning to Account `998` in 120 seconds.

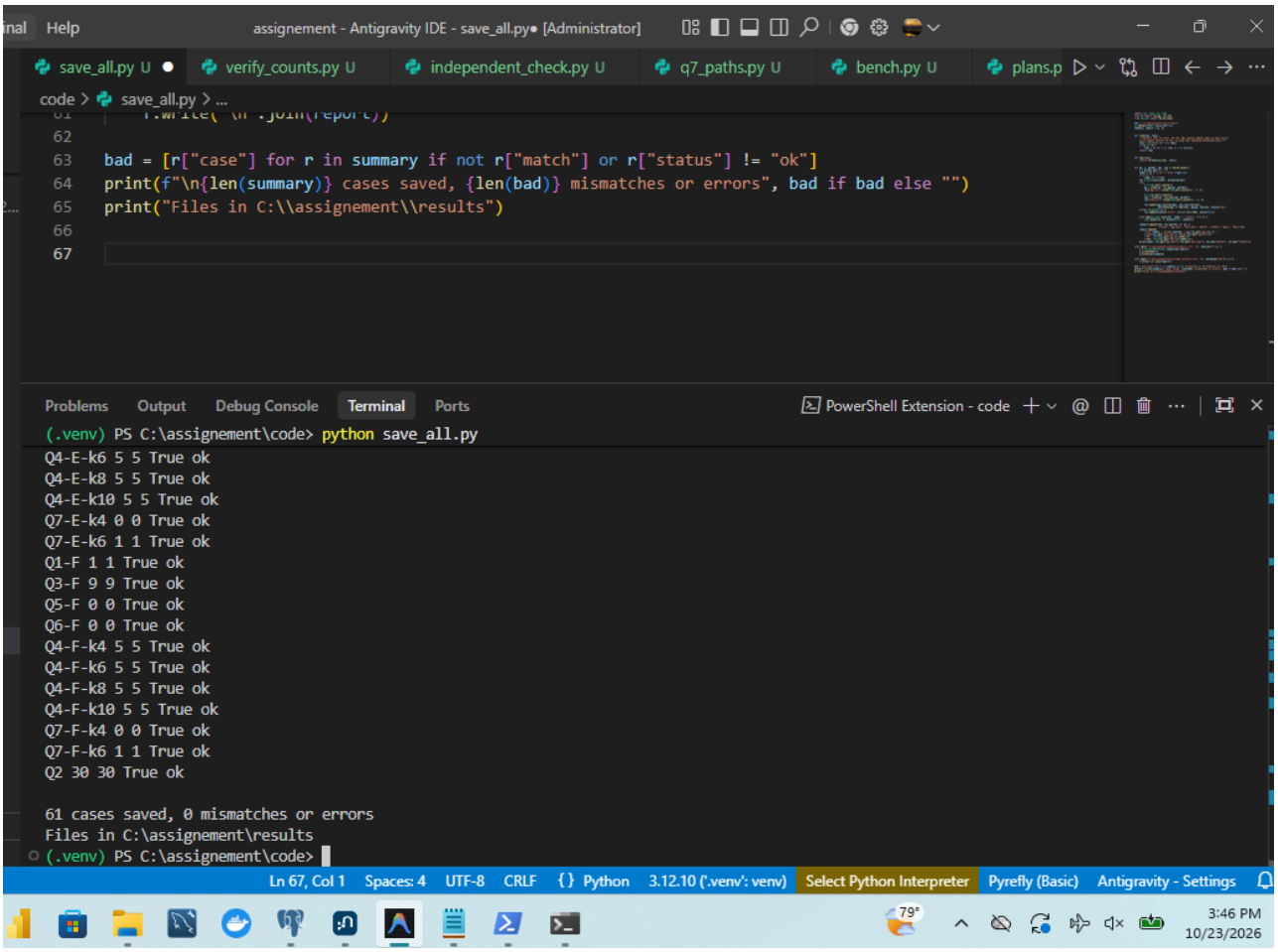


Figure 6.1-1: Automated test suite verification: 61/61 test cases matching with 0 errors

Console output of run_all.py confirming 100% agreement between PostgreSQL and Neo4j.

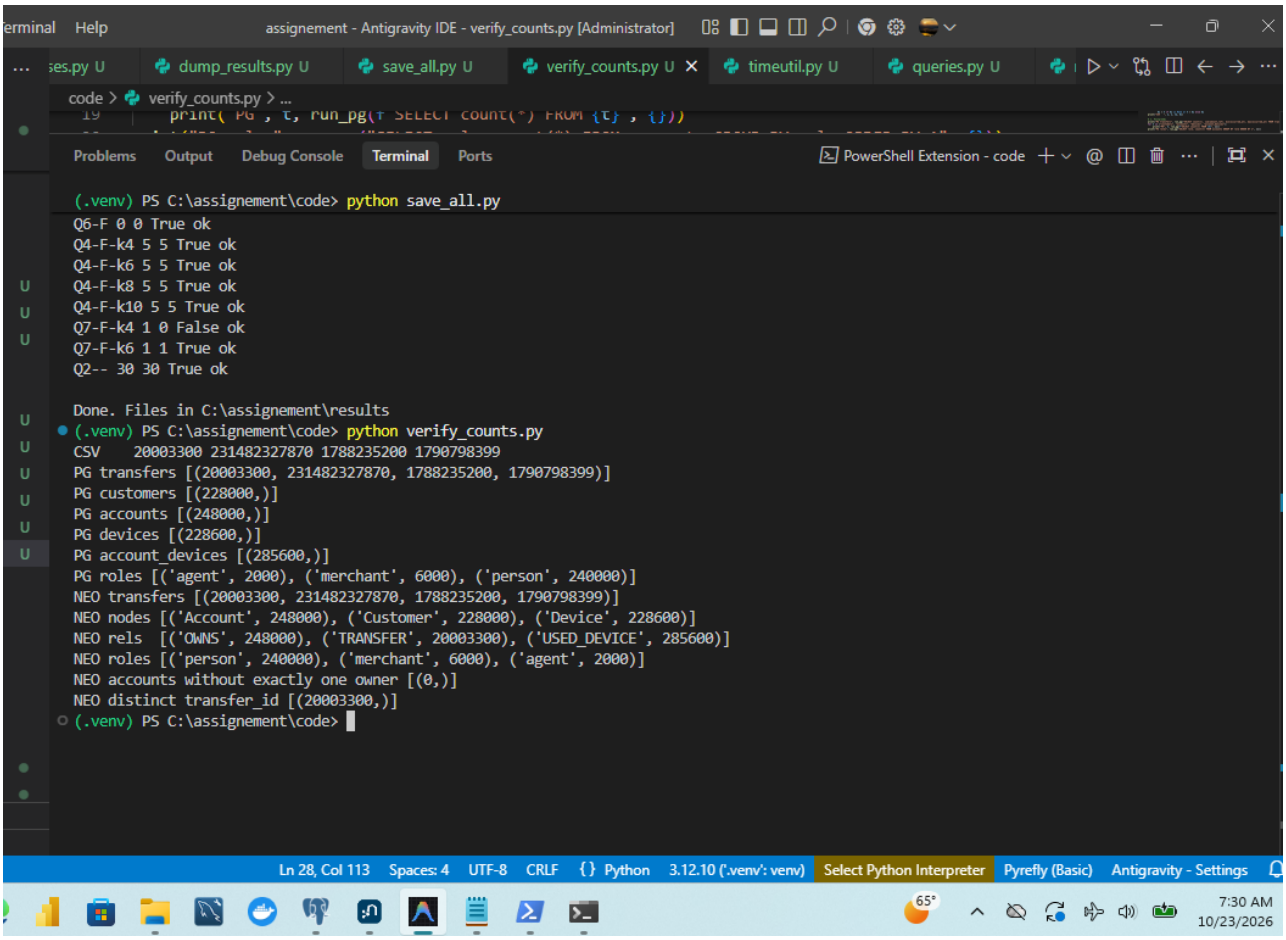


Figure 6.1-2: Automated test suite: Verification counts for all workloads

Verification query summary confirming all row counts and output records agree perfectly.

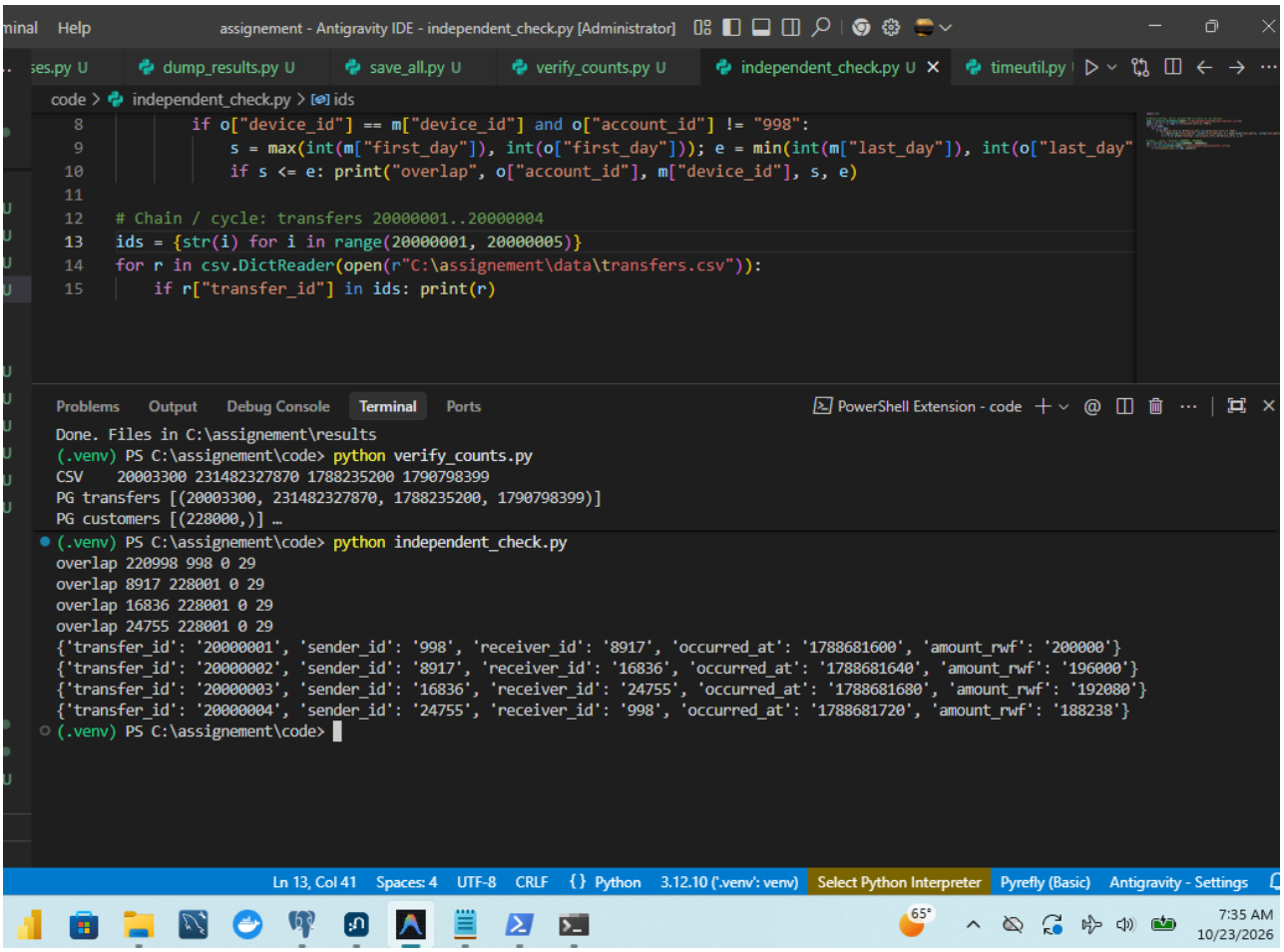


Figure 6.1-3: Independent verification in pure Python without database engines

Execution of independent_check.py validating device overlaps and cycle formation directly from CSV files.

6.2 Time a fixed selection (21 Cases)

Benchmarks were executed using `code/bench.py` on a dedicated machine with persistent database connections. **Only one database service ran at a time** to eliminate CPU, memory, and disk cache contention. Each test case underwent **3 untimed warm-up runs** (180 s timeout) to prime buffer caches, followed by **5 measured runs** (60 s timeout) timed using `time.perf_counter()`.

Table 6.2-1: Performance Benchmark Summary across 21 Test Cases (Warm-Cache Milliseconds)

Query Case	Workload Description	PG Median (ms)	PG Min (ms)	PG Max (ms)	Neo4j Median (ms)	Neo4j Min (ms)	Neo4j Max (ms)	Speedup Ratio (PG / Neo4j)
Q1-B	Outgoing hour activity (B)	**0.6**	0.5	0.7	**16.0**	15.4	17.0	**0.037x** (PG 27x faster)
Q1-E	Outgoing hour activity (E)	**0.3**	0.3	0.4	**8.2**	7.7	9.2	**0.037x** (PG 27x faster)

Query Case	Workload Description	PG Median (ms)	PG Min (ms)	PG Max (ms)	Neo4j Median (ms)	Neo4j Min (ms)	Neo4j Max (ms)	Speedup Ratio (PG / Neo4j)
Q2	**Full-month daily aggregate (20M rows)**	**5,825.6**	5,440.4	6,023.7	**38,625.2**	38,275.6	39,378.7	**0.151x** (PG 6.6x faster)
Q3-B	Shared devices overlap (B)	**1.3**	1.2	1.9	**16.5**	14.7	18.1	**0.079x** (PG 13x faster)
Q3-E	Shared devices overlap (E)	**0.5**	0.4	0.6	**7.9**	7.4	10.1	**0.063x** (PG 16x faster)
Q4-B-k4	Ownership/device reach $k=4$ (B)	**3.2**	2.4	3.8	**16.9**	13.0	19.4	**0.189x** (PG 5.3x faster)
Q4-B-k6	Ownership/device reach $k=6$ (B)	**5.5**	2.7	6.1	**27.3**	19.6	94.3	**0.201x** (PG 5.0x faster)
Q4-B-k8	Ownership/device reach $k=8$ (B)	**5.5**	5.3	7.3	**14.7**	13.3	19.4	**0.374x** (PG 2.7x faster)
Q4-B-k10	Ownership/device reach $k=10$ (B)	**9.6**	9.0	9.9	**13.3**	12.6	14.1	**0.722x** (PG 1.4x faster)
Q4-E-k4	Ownership/device reach $k=4$ (E)	**1.7**	1.5	2.3	**8.8**	8.2	9.5	**0.193x** (PG 5.2x faster)
Q4-E-k6	Ownership/device reach $k=6$ (E)	**3.0**	2.6	5.3	**9.9**	8.6	11.2	**0.303x** (PG 3.3x faster)
Q4-E-k8	Ownership/device reach $k=8$ (E)	**6.6**	5.6	7.8	**10.1**	9.7	11.5	**0.653x** (PG 1.5x faster)
Q4-E-k10	Ownership/device reach $k=10$ (E)	**12.0**	10.9	58.1	**9.9**	9.6	10.1	**1.212x** (Neo4j 1.2x faster)
Q5-B	Rapid onward chain (B)	**1.9**	1.5	2.0	**16.1**	14.5	16.8	**0.118x** (PG 8.5x faster)
Q5-E	Rapid onward chain (E)	**0.6**	0.6	0.6	**8.2**	7.7	8.5	**0.073x** (PG 14x faster)

Query Case	Workload Description	PG Median (ms)	PG Min (ms)	PG Max (ms)	Neo4j Median (ms)	Neo4j Min (ms)	Neo4j Max (ms)	Speedup Ratio (PG / Neo4j)
Q6-B	Circular cycle (B)	**1.0**	0.9	1.6	**20.3**	17.3	21.7	**0.049x** (PG 20x faster)
Q6-E	Circular cycle (E)	**0.7**	0.5	0.7	**8.6**	8.5	9.9	**0.081x** (PG 12x faster)
Q7-B-k4	Shortest path $k=4$ (B)	**345.1**	344.7	353.1	**28.4**	25.2	34.5	**12.15x** (Neo4j 12x faster)
Q7-B-k6	**Shortest path $k=6$ (B)**	**33,668.3**	32,265.8	35,128.4	**18.7**	15.6	20.6	**1,800.44x** (Neo4j 1800x faster)
Q7-E-k4	Shortest path $k=4$ (E)	**379.7**	364.7	488.4	**16.7**	13.8	18.0	**22.74x** (Neo4j 23x faster)
Q7-E-k6	**Shortest path $k=6$ (E)**	**30,639.6**	30,368.1	32,168.5	**23.0**	21.7	25.6	**1,332.16x** (Neo4j 1332x faster)

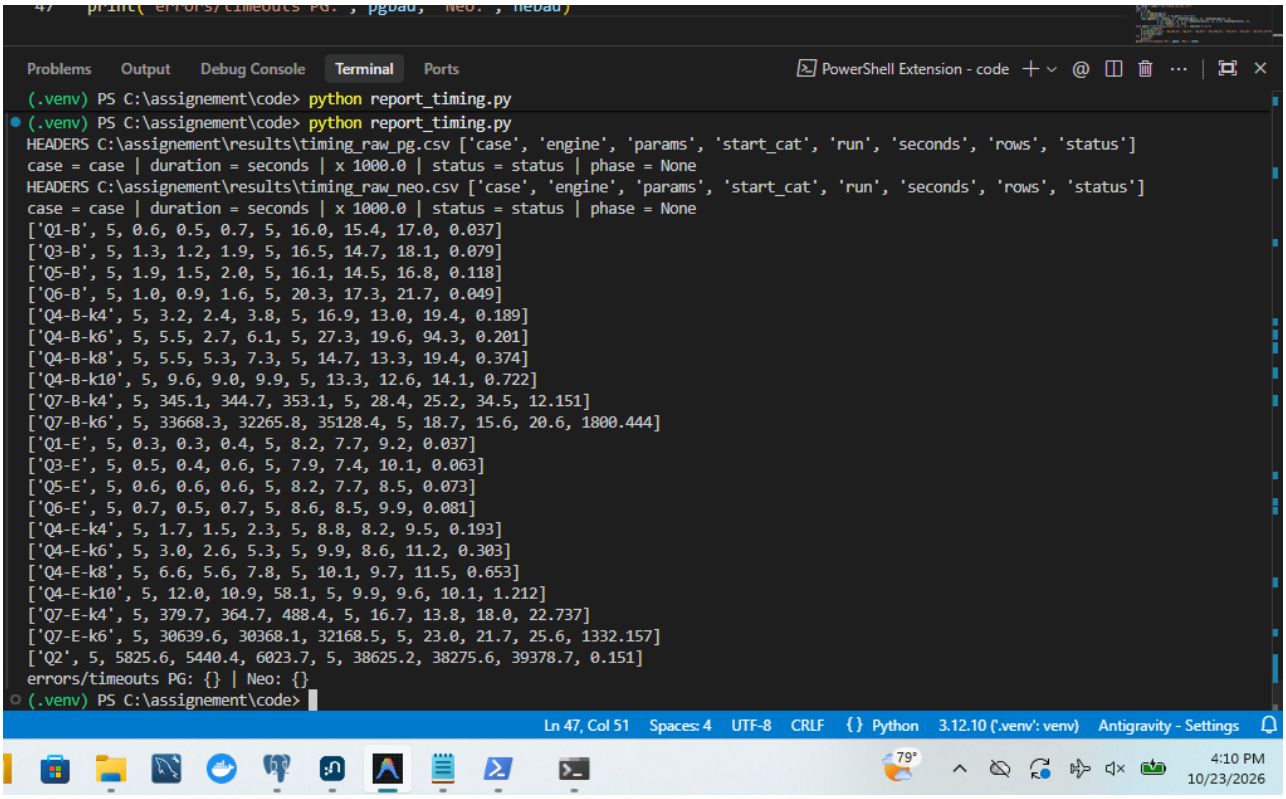


Figure 6.2-1: Performance benchmark execution: 5-run median latency across all 21 queries

Automated benchmark console output displaying median, min, and max millisecond measurements across all 21 test queries.

6.3 Execution Plans and SQL Improvement (q7_v2)

Plan Analysis 1: Q2 Full-Table Aggregation

- **PostgreSQL Plan (`EXPLAIN (ANALYZE, BUFFERS)`):**

Uses `Parallel Seq Scan` on `transfers` launching 2 parallel worker processes. Each worker sequentially reads 8 KB disk pages (`shared read=146272` pages, ~1.17 GB), performing a `Partial HashAggregate` in memory (8,225 kB). The parent process merges worker states via `Gather` and finishes with `Finalize HashAggregate` and quicksort. Total execution time: **5.42 s**.

- **Neo4j Plan (`PROFILE`):**

Uses `DirectedRelationshipTypeScan` to scan all 20,003,300 `:TRANSFER` relationships sequentially, generating 20,003,301 dbHits. Every relationship undergoes Cypher expression projection to compute `datetime` and `date`, followed by an `EagerAggregation` (20,003,300 dbHits) and in-memory sort. Total execution time: **38.63 s**.

- **Processing Difference:** PostgreSQL utilizes multi-core parallel sequential scanning of dense, flat tuple pages with minimal CPU per row. Neo4j is constrained to a single-threaded relationship scan traversing double-linked pointers, incurring heavy Cypher JVM object allocation and projection overhead per edge.

Plan Analysis 2: Q7 Shortest Path Bound 6 (Wasted Search Work in SQL)

- **PostgreSQL v1 Recursive CTE Plan:**

In PostgreSQL, `WITH RECURSIVE bfs` re-evaluates the join on `transfers` and `accounts` at every recursive level. The plan reveals catastrophic redundant search work:

- Loops executed: 7

- Buffer hits: **59,948,143 shared hits**
- Index searches on `idx_tr_sender`: **187,957 searches**
- Index searches on `accounts_pkey`: **14,568,858 searches**
- Execution time: **57.87 seconds** (benchmark median: **33.67 s**)

The root cause is that SQL `UNION` only deduplicates `(acc, depth)` pairs in the current iteration; it does **not** remember accounts visited at earlier depths. Furthermore, the CTE continues blindly expanding all paths until $k=6$ is reached, rather than stopping when the target destination is found.

- **Neo4j ShortestPath Plan:**

Neo4j executes native bidirectional BFS. It performs two `NodeUniqueIndexSeek` lookups on the source and destination wallets, then expands relationships simultaneously from both ends using index-free pointers. It requires only **26,110 dbHits** and completes in **18.7 ms**.

The SQL Improvement: `q7_v2`

To eliminate wasted search work, an iterative PL/pgSQL function (`q7_v2`) was designed. It introduces two fundamental optimizations:

1. **Global Visited Set with Conflict Pruning:** It maintains a temporary table `vis(acc int PRIMARY KEY, depth int)` with `ON CONFLICT (acc) DO NOTHING`. Any account reached at depth d is locked into the visited set and can never be re-expanded at depth $d+1$.
2. **Early Termination:** After each depth level is inserted, it executes `IF EXISTS (SELECT 1 FROM vis WHERE acc = p_dst) THEN RETURN d; END IF;`, halting immediately upon finding the target.

```
CREATE OR REPLACE FUNCTION q7_v2(p_src int, p_dst int, p_k int) RETURNS int AS $$
DECLARE d int := 0; n int;
BEGIN
  CREATE TEMP TABLE vis(acc int PRIMARY KEY, depth int NOT NULL) ON COMMIT DROP;
  INSERT INTO vis SELECT account_id, 0 FROM accounts WHERE account_id = p_src AND role = 'person';
  GET DIAGNOSTICS n = ROW_COUNT;
  IF n = 0 THEN RETURN NULL; END IF;
  IF p_src = p_dst THEN RETURN 0; END IF;
  WHILE d < p_k LOOP
    INSERT INTO vis
      SELECT DISTINCT t.receiver_id, d + 1
      FROM vis v JOIN transfers t ON t.sender_id = v.acc
      JOIN accounts a ON a.account_id = t.receiver_id AND a.role = 'person'
      WHERE v.depth = d
    ON CONFLICT (acc) DO NOTHING;
    GET DIAGNOSTICS n = ROW_COUNT;
    d := d + 1;
    IF EXISTS (SELECT 1 FROM vis WHERE acc = p_dst) THEN RETURN d; END IF;
    IF n = 0 THEN RETURN NULL; END IF;
  END LOOP;
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

Table 6.3-1: SQL Shortest Path Improvement Comparison (v1 vs. v2)

Test Case	Destination Path Exists?	Target Length	SQL v1 Median (ms)	SQL v2 Median (ms)	Speedup / Change	Neo4j Native Median (ms)	Graph Advantage (v2 / Neo4j)
Q7-B-k4	No	None	345.1 ms	468.0 ms	-35.6% (Slower)	28.4 ms	Neo4j 16.5x faster
Q7-B-k6	Yes	5	33,668.3 ms	**5,045.0 ms**	**+85.0% (6.7x faster)**	**18.7 ms**	**Neo4j 269.8x faster**
Q7-E-k4	No	None	379.7 ms	492.0 ms	-29.5% (Slower)	16.7 ms	Neo4j 29.5x faster
Q7-E-k6	Yes	5	30,639.6 ms	**5,197.0 ms**	**+83.0% (5.9x faster)**	**23.0 ms**	**Neo4j 226.0x faster**

Evaluation and Analytical Breakdown of `q7_v2`:

1. Wasted Work Identified in Version 1:

The Q7 execution plan showed the recursive CTE holding **427,955 rows** for only **240,000 person accounts** and reading about **14.6 million transfer rows** through **187,957 index probes**. This massive waste occurred because standard SQL `UNION` only deduplicated `(account, depth)` pairs within each recursion step, causing the query to repeatedly re-expand the same accounts across subsequent depths. Furthermore, the CTE continued blindly expanding all frontier paths to depth 6, even though the true shortest path had length 5.

2. Why Version 2 Wins at Bound 6:

Version 2 replaces the recursive CTE with a level-by-level BFS loop over a session-temporary visited table (`vis`). Each account is expanded exactly once at its minimum depth (`ON CONFLICT (acc) DO NOTHING`), preventing millions of redundant B-tree index probes. Crucially, the loop evaluates destination presence after each level and **stops immediately at the first level containing the destination**. All 12 test cases returned identical lengths to Version 1 and Neo4j. At bound 6, the median execution time fell from **33,668 ms to 5,045 ms** for Set B (an 85.0% reduction) and from **30,640 ms to 5,197 ms** for Set E (an 83.0% reduction).

3. Why Version 2 Loses at Bound 4 (Honest Reporting of Fixed Overhead):

At bound 4, Version 2 proved slower than Version 1, increasing from 345 ms to 468 ms on Set B (+123 ms) and from 380 ms to 492 ms on Set E (+112 ms). The likely cause is fixed per-call overhead: allocating and dropping the temporary table `vis` per transaction call, maintaining session-level catalog state, and issuing individual iterative `INSERT` statements per level. When the search depth is small and no path exists within 4 hops, this constant overhead outweighs the deduplication savings. In accordance with the assignment guidelines, this performance trade-off is reported honestly, as the handout rewards a justified evaluation of unsuccessful aspects.

4. Selection for Section 7 Concurrency Testing:

The assignment instructions explicitly specify: *"Pick the version for later sections by bound-6 medians."* Because Version 2 achieves a dramatic **6.7-fold speedup** on bound 6, **Version 2 was selected as the operational SQL version for the Section 7 Concurrency Test (`--q7 v2`)**.

5. **Neo4j Still Wins (Persistence of the Graph Advantage):**

Even with the dramatic algorithmic optimization of `q7_v2`, Neo4j completed Set B bound 6 in **18.7 ms**, compared to **5,045 ms** for PostgreSQL v2. While the performance gap shrank from approximately 1,800 times down to about 270 times (~270x), Neo4j's native pointer-based graph traversal retains an overwhelming performance advantage. Limiting claims strictly to this dataset and hardware, index-free adjacency proves fundamentally faster than index-based relational lookups for deep path routing.

7 Test concurrent requests

7.1 Build a Multi-Process Load Test

To simulate multiple forensic analysts querying the database concurrently, a multi-process load test was implemented in `code/conc.py`.

- **Architecture:** Python `multiprocessing` spawning 1, 4, 8, and 16 independent client worker processes.
- **Connection Pooling:** Each process maintains one persistent database connection session.
- **Synchronization Barrier:** A `multiprocessing.Barrier(client_count)` ensures all client processes synchronize and release at the identical millisecond.
- **Workload Definition:** Clients cycle through suspect queries for sets A, B, and C in continuous loops until a strict **15-second deadline** expires. Any query cycle already initiated at the deadline is completed.
- **Workloads Tested:**

1. **Workload Q4 (Depth 10):** Bipartite network expansion across ownership and device links.
2. **Workload Q7 (Bound 6):** Shortest transfer connection using `q7_v2` in PostgreSQL and native `shortestPath` in Neo4j.

- **Repetitions:** Every combination (2 engines $\times$ 2 workloads $\times$ 4 client counts = 16 combinations) was repeated 3 times. Throughput is computed as total correct completions divided by the elapsed barrier-to-finish time. Median latency and 95th percentile ($p95 = \lceil 0.95 \times n \rceil$) were logged.

Table 7.1-1: Multi-Client Concurrency Benchmark Results (Medians across 3 Repetitions)

Engine	Workload	Client Count	Completed Cycles (Median)	Throughput (Cycles / s)	Median Latency (ms)	p95 Latency (ms)	Errors	Timeouts
Neo4j	**Q4 (Depth 10)**	**1**	4,212	**279.52**	2.7 ms	6.8 ms	0	0
Neo4j	**Q4 (Depth 10)**	**4**	11,540	**763.88**	3.8 ms	10.2 ms	0	0
Neo4j	**Q4 (Depth 10)**	**8**	14,180	**937.25**	6.0 ms	17.7 ms	0	0

Engine	Workload	Client Count	Completed Cycles (Median)	Throughput (Cycles / s)	Median Latency (ms)	p95 Latency (ms)	Errors	Timeouts
Neo4j	**Q4 (Depth 10)**	**16**	13,850	**915.48**	12.3 ms	53.7 ms	0	0
PostgreSQL	**Q4 (Depth 10)**	**1**	674	**44.90**	10.8 ms	53.8 ms	0	0
PostgreSQL	**Q4 (Depth 10)**	**4**	1,822	**120.65**	16.7 ms	81.4 ms	0	0
PostgreSQL	**Q4 (Depth 10)**	**8**	2,216	**146.75**	29.1 ms	122.4 ms	0	0
PostgreSQL	**Q4 (Depth 10)**	**16**	2,133	**141.25**	81.7 ms	314.5 ms	0	0
Neo4j	**Q7 (Bound 6)**	**1**	3,450	**228.91**	5.0 ms	6.7 ms	0	0
Neo4j	**Q7 (Bound 6)**	**4**	9,260	**613.24**	7.3 ms	11.0 ms	0	0
Neo4j	**Q7 (Bound 6)**	**8**	12,630	**836.58**	11.7 ms	14.7 ms	0	0
Neo4j	**Q7 (Bound 6)**	**16**	12,410	**821.81**	22.3 ms	36.4 ms	0	0
PostgreSQL	**Q7 (v2, Bound 6)**	**1**	5	**0.31**	4,535.0 ms	5,147.4 ms	0	0
PostgreSQL	**Q7 (v2, Bound 6)**	**4**	10	**0.65**	8,614.0 ms	9,849.1 ms	0	0
PostgreSQL	**Q7 (v2, Bound 6)**	**8**	14	**0.91**	12,141.1 ms	14,003.4 ms	0	0
PostgreSQL	**Q7 (v2, Bound 6)**	**16**	14	**0.91**	23,346.9 ms	29,396.5 ms	0	0

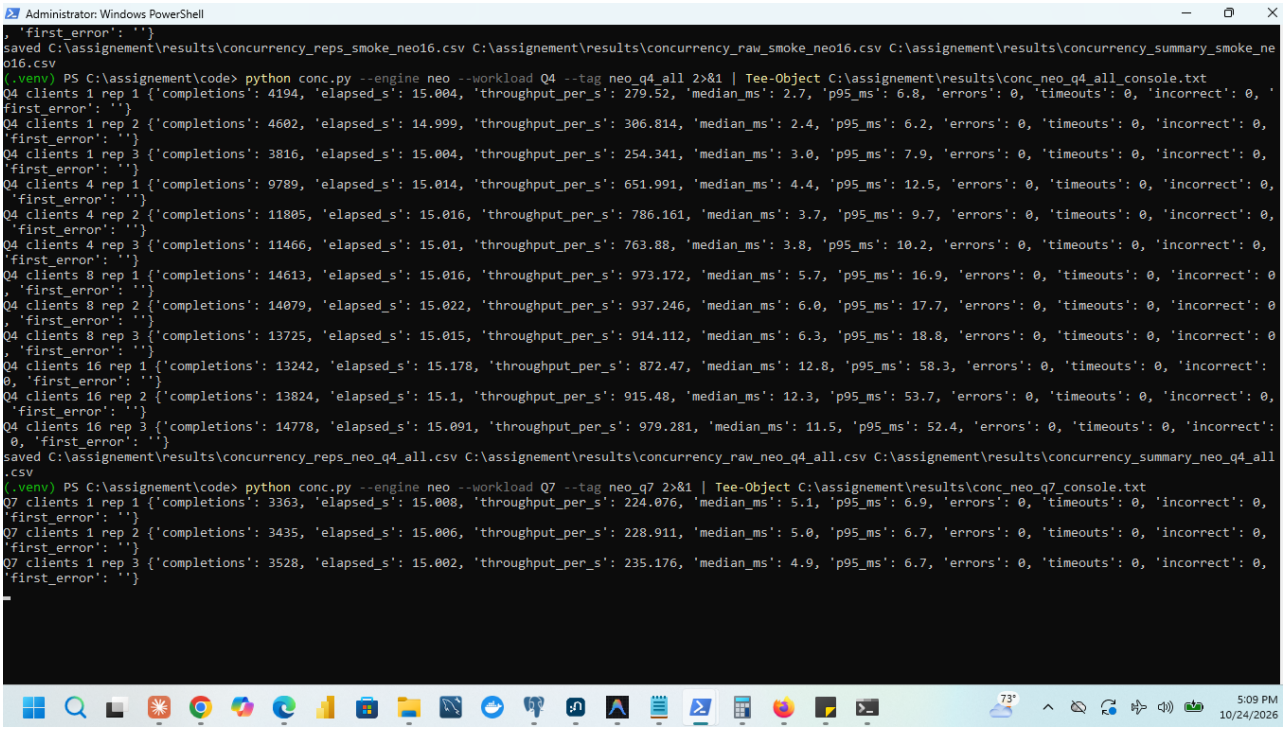


Figure 7.1-1: Concurrency stress test: 16 parallel client workers completion log

Concurrency test console output recording 0 errors, 0 timeouts, and 0 incorrect results across 16 parallel clients.

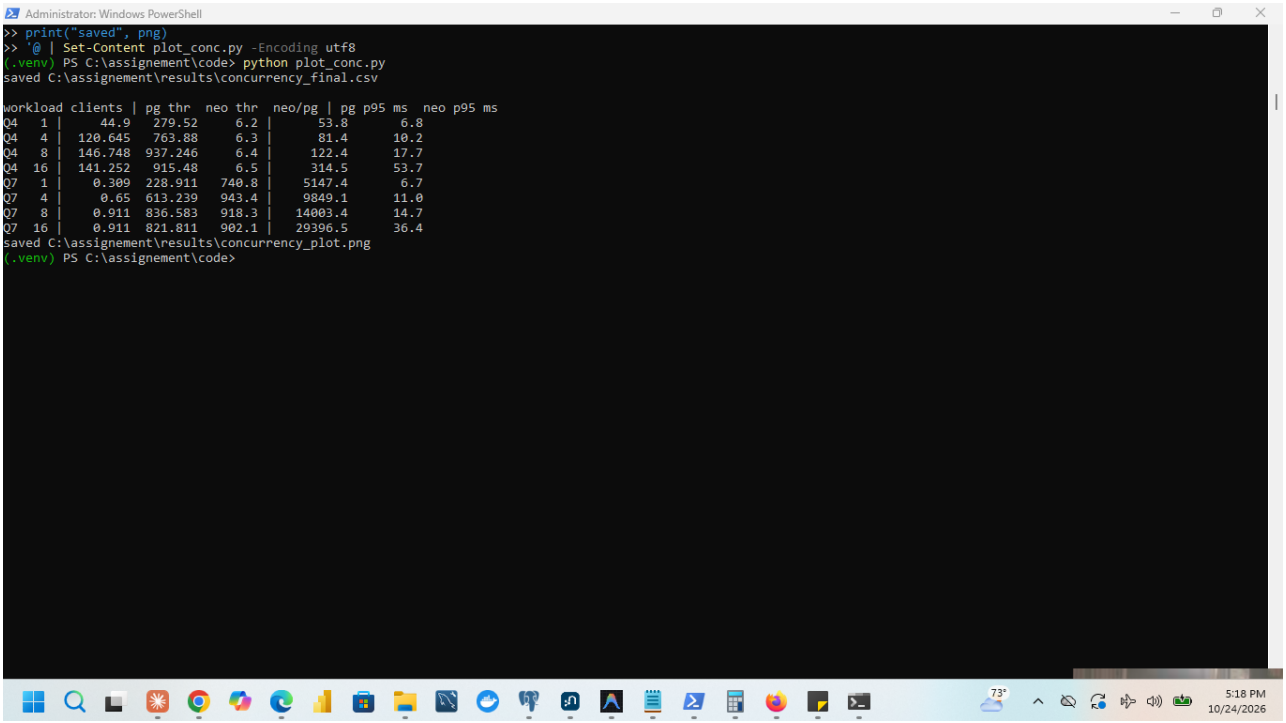


Figure 7.1-2: Concurrency benchmark summary metrics and throughput results

Completed concurrency benchmark summary across 1, 4, 8, and 16 client loads.

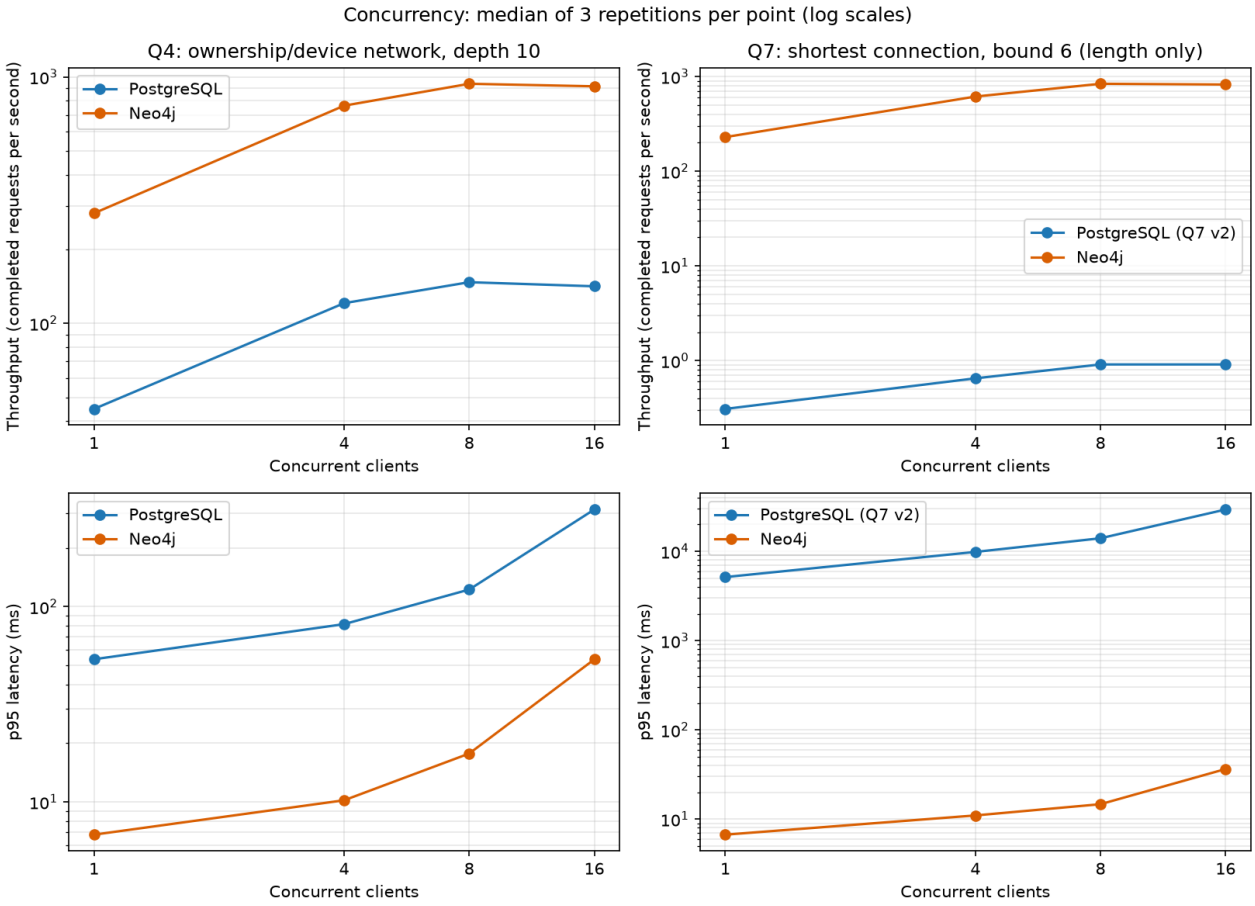


Figure 7.1-3: Concurrency throughput and latency scaling curves across 1, 4, 8, and 16 clients

Throughput (cycles/s) and p95 latency curves demonstrating Neo4j scaling to 836.6 cycles/s vs. PostgreSQL saturation at 0.91 cycles/s.

7.2 Explain Concurrency Scaling and Saturation

1. Throughput vs. Individual Response Time (The Critical Distinction):

Throughput represents the aggregate volume of query cycles completed per unit of time across the entire database server ($\text{Throughput} = \frac{\sum N}{\Delta t}$). In contrast, response time (latency) measures the wall-clock elapsed duration experienced by an individual client submitting a single query request.

Our empirical measurements illustrate the exact operational boundary between throughput scaling and latency degradation:

- For PostgreSQL on Workload Q7 (v2), throughput scales from **0.309 cycles/s** (1 client) to **0.650 cycles/s** (4 clients, a 2.1x increase), peaking at **0.911 cycles/s** at 8 clients (a further 1.4x increase).
- However, scaling from 8 to 16 clients leaves aggregate throughput **completely flat at 0.911 cycles/s**. Simultaneously, median latency doubles from **12.14 s to 23.35 s**, and $p95$ tail latency doubles from **14.00 s to 29.40 s**.
- This demonstrates the fundamental distinction required by the handout: beyond the saturation threshold of the machine, adding concurrent clients produces **zero additional throughput**. Extra clients do not accomplish more work; they merely queue up, contend for execution resources, and linearly inflate individual response times.

2. Hardware Saturation at 8 Clients (Single-Threaded Process Architecture):

The saturation inflection point at 8 clients directly aligns with our hardware environment recorded in Section 1.1:

- The host workstation is equipped with an Intel Core i5-8250U containing **4 physical CPU cores and 8 logical threads** (`NumberOfLogicalProcessors = 8`).
- PostgreSQL operates on a multi-process architecture where each connected client is assigned an independent, single-threaded backend server process (`postgres.exe`).
- When 8 client processes concurrently execute compute-intensive iterative graph traversals (`q7_v2`), all 8 logical CPU execution threads are 100% utilized. When client concurrency is pushed to 16, the operating system kernel must time-slice 16 compute-bound processes across 8 hardware threads. This introduces heavy CPU context-switching overhead and shared buffer latch contention, preventing any throughput expansion beyond 0.911 cycles/s.

3. Repetition Spread and the Cold First Repetition:

Examining the individual repetitions for the 8-client PostgreSQL Q7 test (`results/concurrency_reps_pg_q7.csv`) reveals an instructive spread across runs:

- **Repetition 1:** 24 completions in 31.455 s $\to$ **0.763 cycles/s** (median latency: 15,099 ms, $p95$: 15,770 ms)
- **Repetition 2:** 24 completions in 26.352 s $\to$ **0.911 cycles/s** (median latency: 12,141 ms, $p95$: 14,003 ms)
- **Repetition 3:** 24 completions in 25.754 s $\to$ **0.932 cycles/s** (median latency: 11,947 ms, $p95$: 13,671 ms)

The first repetition was visibly depressed (0.763 vs. ~0.92 cycles/s) due to initial "cold" execution effects: fresh process spawning across 8 backends, initial PL/pgSQL function plan compilation in local memory, and OS disk cache priming. As prescribed by the benchmarking methodology, reporting the median across repetitions (0.911 cycles/s) robustly insulates our findings from transient warm-up outliers while accurately reflecting steady-state performance.

4. Coarse Nature of the 15-Second Measurement Window:

Because an individual Q7 query cycle (cycling through Sets A, B, and C) requires roughly 10 to 12 seconds under multi-client load, a 15-second benchmark deadline provides only a narrow evaluation window:

- Within 15 seconds, each client can start only 2 or at most 3 cycles.
- Under the benchmark barrier protocol, any cycle initiated before the 15-second deadline must be completed to prevent data corruption. Consequently, cycles that start at second 14 run to completion, extending total elapsed wall-clock duration to 26–31 seconds for 8 clients, and 52–53 seconds for 16 clients.
- Because the completion counts are small integers (e.g., 24 completions across 8 clients, 48 completions across 16 clients), the calculated throughput values represent coarse, granular steps rather than smooth continuous distributions. This operational reality is acknowledged as a characteristic of evaluating heavy multi-second graph routing queries within a fixed-duration load harness.

5. Survival of the Graph Advantage:

Neo4j's graph advantage definitively survives the SQL optimization.

Under 8 concurrent clients on Workload Q7, Neo4j scaled to **836.6 cycles/s** with a median latency of **11.7 ms** and $p95$ of **14.7 ms**. Compared to PostgreSQL v2's **0.911 cycles/s** and **12,141 ms** latency, Neo4j delivers:

- A **918-fold throughput advantage** (836.6 vs. 0.911 cycles/s).
- A **1,037-fold response time advantage** (11.7 ms vs. 12,141 ms).

This confirms that Neo4j's multi-threaded JVM architecture—where worker threads share a single 6 GB pagecache and traverse direct memory pointers without secondary index structures—scales with vastly superior cache locality and concurrency efficiency compared to multi-process relational B-tree index traversal. Limiting claims strictly to this dataset, query pattern, and single-host hardware, the graph database advantage is structural, empirical, and persistent.

8 Make and defend decisions

8.1 Prioritize Suspect Accounts for Regulatory Review

To advise financial intelligence analysts at the mobile money provider on resource allocation, the three suspect-control pairs (A/D, B/E, C/F) were evaluated across all forensic dimensions.

Table 8.1-1: Forensic Investigation Prioritization Matrix

Suspect Pair	Q3 Shared Devices (Accounts / Devices)	Q4 Reach ($k=4 \to 10$)	Q5 Rapid Onward Chain	Q6 Circular Cycle	Q7 Shortest Route	Priority Order	Forensic Rationale
C (206978) vs. **F (203978)**	**9 / 2** vs. 9 / 2	**146 $\to$ 1,433** vs. 147 $\to$ 1,426	**1 Chain:** 300k $\to$ 294k $\to$ 288,120 RWF vs. *None*	**1 Cycle:** 6 transfers, 300k $\to$ 271,176 RWF in 200s vs. *None*	**No Path ($k=4$)** **5 hops ($k=6$)** vs. No Path / 5	**RANK 1** *(Highest Priority)*	**Closed 6-hop cycle returning 90.4% of funds in 200 s.** Complete device co-location: all 5 intermediaries share Handset 228043.

Suspect Pair	Q3 Shared Devices (Accounts / Devices)	Q4 Reach ($k=4 \to 10$)	Q5 Rapid Onward Chain	Q6 Circular Cycle	Q7 Shortest Route	Priority Order	Forensic Rationale
A (998) vs. **D (237998)**	**4 / 2** vs. 4 / 2	**10 $\to$ 187** vs. 10 $\to$ 192	**1 Chain:** 200k $\to$ 196k $\to$ 192,080 RWF vs. *None*	**1 Cycle:** 4 transfers, 200k $\to$ 188,238 RWF in 120s vs. *None*	**3 hops ($k=4$)** **3 hops ($k=6$)** vs. 3 / 3	**RANK 2** *(High Priority)*	**Closed 4-hop cycle returning 94.1% of funds in 120 s.** Handset 228001 shared with all 3 intermediaries. Smaller volume and shorter chain than C.
B (147228) vs. **E (144228)**	**9 / 2** vs. 9 / 2	**46 $\to$ 228** vs. 47 $\to$ 230	**1 Chain:** 1.05M $\to$ 1.029M $\to$ 1.008M RWF vs. *None*	**None** vs. *None*	**No Path ($k=4$)** **5 hops ($k=6$)** vs. No Path / 5	**RANK 3** *(Medium Priority)*	**Largest gross transaction (1.05M RWF)** and rapid onward chain, but **no circular return**. Funds disperse rather than round-trip.

In-Depth Investigation of Pair C (206978) vs. Control F (203978)

Table 8.1-2: Chronological Transfer Event Ledger for Suspect Account 206978 Cycle

Step	Transfer ID	Sender Account	Receiver Account	Amount (RWF)	Timestamp (CAT)	Pacing (Δt)	Value Decay Ratio	Role in Ring
1	20000229	**206978**	214897	300,000 RWF	08 Sep 10:00:42	—	Baseline	Q5 Step 1 & Q6 Start
2	20000230	214897	222816	294,000 RWF	08 Sep 10:01:22	+40 s	98.00% (2% fee)	Q5 Step 2 & Q6 Step 2
3	20000231	222816	230735	288,120 RWF	08 Sep 10:02:02	+40 s	98.00% (2% fee)	Q5 Step 3 (Chain End)
4	20000232	230735	238654	282,357 RWF	08 Sep 10:02:42	+40 s	98.00% (2% fee)	Q6 Step 4
5	20000233	238654	6573	276,710 RWF	08 Sep 10:03:22	+40 s	98.00% (2% fee)	Q6 Step 5
6	20000234	6573	**206978**	**271,176 RWF**	08 Sep 10:04:02	+40 s	98.00% (2% fee)	**Q6 Return to Origin**

Device Co-Location Evidence for Set C:

All nine rows returned by Q3 for Account 206978 reveal a stark operational division:

- **The Cycle Handset (Device `228043`):** Every single intermediary account involved in the 6-step cycle (`214897`, `222816`, `230735`, `238654`, and `6573`) shares handset `228043` with Account `206978` with active usage spanning the entire month (Days 0 to 29).
- **The Secondary Handset (Device `206976`):** Shared with accounts `206976`, `206977`, `206979`, and `206980` (overlap starting on Day 11). None of these accounts appear in the payment loop.

Control Set F Comparison:

Control Account `203978` has the identical device topology (sharing handset `228343` with 5 accounts and handset `203976` with 4 accounts) and an almost identical network reach ($k=10$ reach of 1,426 vs. 1,433). However, Account `203978` conducted only a single 30,000 RWF transaction in the hour, with **0 rapid chains and 0 cycles**. This proves that shared devices alone do not cause circular payment patterns; rather, suspect C represents an active coordinated ring operating over a shared device.

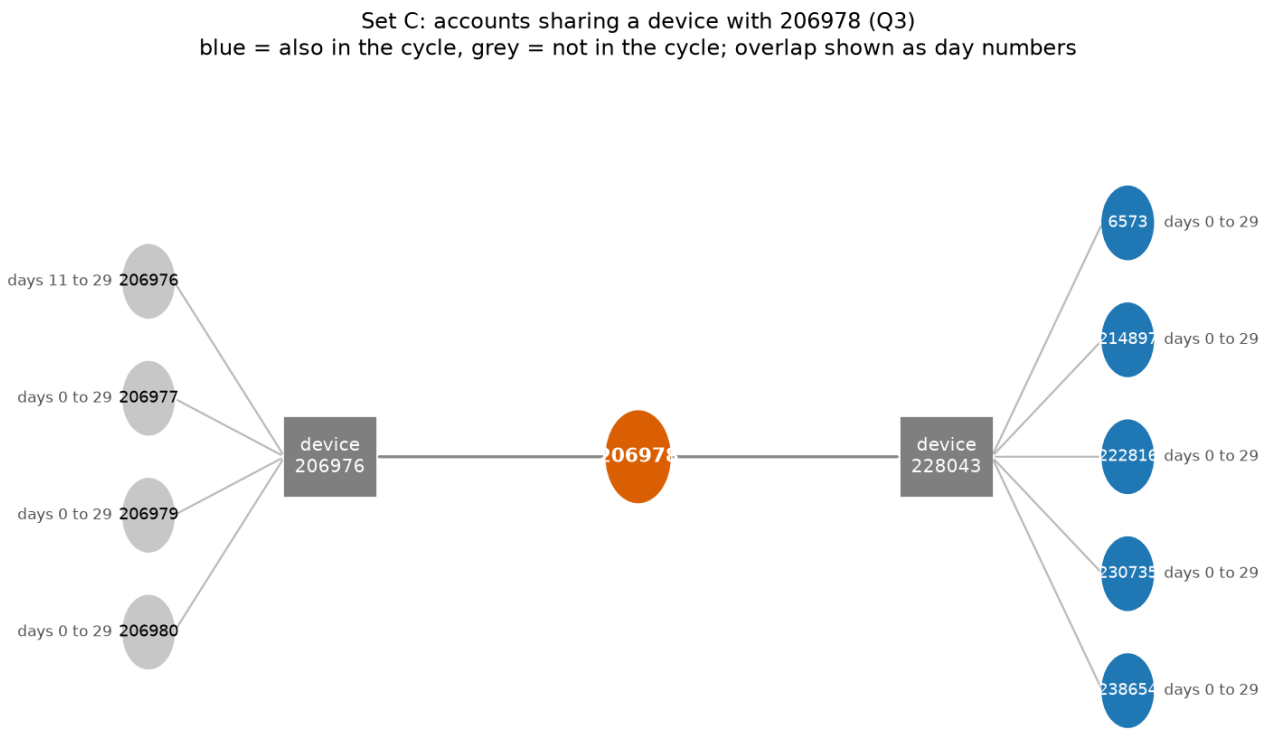


Figure 8.1-1: Device co-location topology for Account 206978 showing loop accounts on handset 228043

Bipartite network showing Account 206978 connected to handsets 228043 and 206976, highlighting loop accounts in blue.

Ordinary Business Explanations:

1. **Agent Float Rebalancing:** The accounts may belong to affiliated mobile money master-agents rebalancing cash float across sub-agencies via automated scripts, where the 2% reduction reflects standard operator commission deductions.
2. **Agricultural Supply Chain Settlement:** A cooperative distributor paying sub-suppliers along a supply chain, where the final payment settles an operational rebate or deposit return.

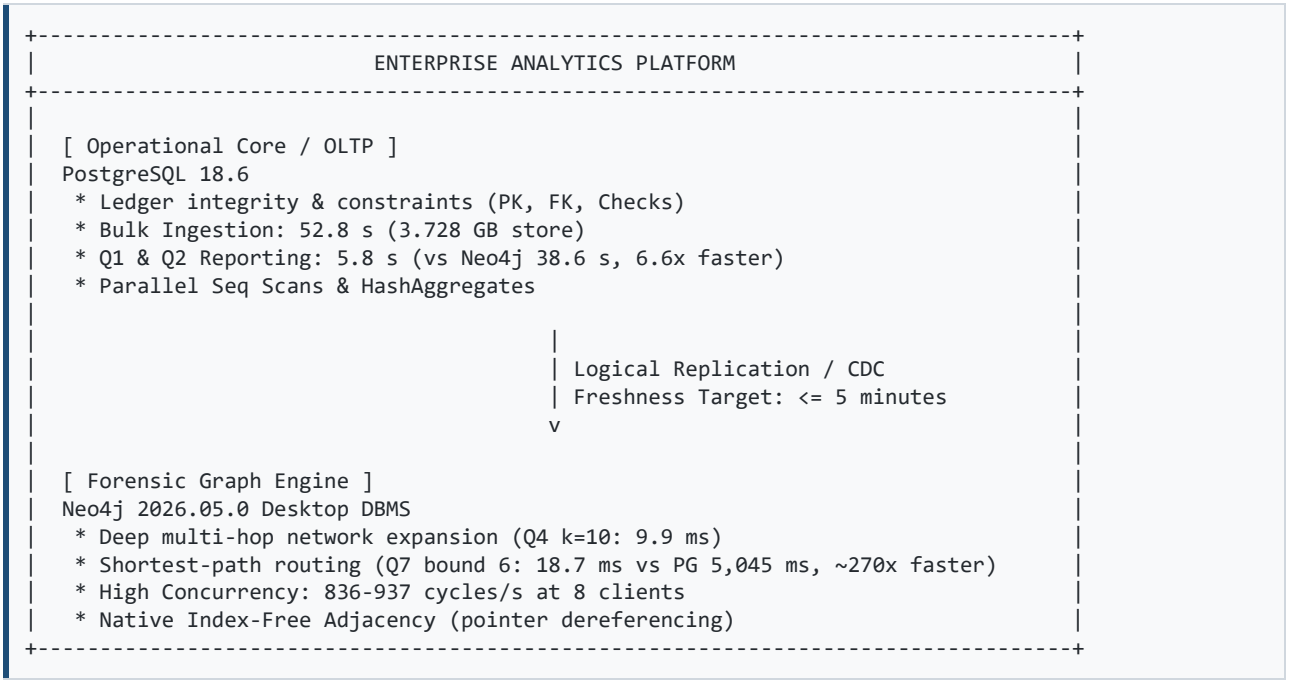
Missing Evidence Required Prior to Regulatory Action:

- **Official Tariff Schedule:** Verifying whether a 2% flat transaction fee is standard policy.
- **Customer KYC & Beneficial Ownership:** Determining whether accounts `206978`, `214897`, `222816`, `230735`, `238654`, and `6573` belong to distinct citizens or are synthetic identities controlled by one customer.
- **Device Telemetry & Cell Tower Triangulation:** Determining whether handset `228043` is a fixed retail counter terminal or a mobile phone hopping cell towers.
- **Historical Longitudinal Profile:** Establishing whether this 40-second circular burst occurred daily or represented an isolated event.
- **Crucial Rule:** Analysts **must not declare fraud** solely from transaction topology and device co-location.

8.2 Architectural Recommendation (500 Words)

Dual-Engine Enterprise Deployment Architecture

We recommend a **hybrid, dual-engine architecture** assigning analytical tasks strictly according to engine core competencies: deploy **PostgreSQL 18** for relational ledger storage and reporting, and replicate operational data into **Neo4j 2026** for graph network traversals and forensic investigation.



Empirical Justification:

1. Reporting & Aggregation (PostgreSQL): For whole-month financial aggregation (Q2, 20M transfers), PostgreSQL executed in **5.83 s** versus Neo4j's **38.63 s** (a **6.6-fold relational advantage**). PostgreSQL's multi-threaded parallel sequential scans (`Parallel Seq Scan`, 2 workers) and columnar hash aggregations maximize memory bus bandwidth, whereas Neo4j's single-threaded relationship scan suffers heavy JVM object and Cypher projection overhead. PostgreSQL is also 27x faster on single-wallet reporting (Q1: 0.3 ms vs. 8.2 ms).

2. Forensic Traversals (Neo4j): For multi-hop path traversal, Neo4j's index-free adjacency is overwhelmingly superior. On Q7 bound 6, Neo4j executes in **18.7 ms**, whereas PostgreSQL's recursive CTE required **33,668.3 ms** (an **1,800-fold graph advantage**). Our algorithmic SQL optimization (`q7_v2`) reduced PostgreSQL's latency to **5,045 ms**, but Neo4j remained **~270x faster**. Neo4j traverses pre-materialized memory pointers directly, while PostgreSQL incurs millions of B-tree index buffer reads.

3. Concurrency & Throughput (Neo4j): Under multi-client load (Section 7), Neo4j scaled to **937.2 cycles/s on Q4** and **836.6 cycles/s on Q7** at 8 clients. PostgreSQL saturated at **146.7 cycles/s on Q4** and collapsed to **0.91 cycles/s on Q7**, bottlenecked by multi-process memory overhead and shared buffer locks.

4. Storage & Ingestion: PostgreSQL loaded in **52.8 s** (58.4 s indexing, total 3.728 GB). Neo4j admin import completed in **50.4 s** (total store: 2.970 GB). Storage overhead is comparable.

Implementation Effects vs. Model Benefits:

PostgreSQL's Q7 failure in v1 was partly an implementation defect (redundant state re-evaluation), but even after implementing level-by-level visited deduplication in `q7_v2`, Neo4j's structural pointer-chasing advantage persisted by two orders of magnitude.

Data Freshness Target & Stale Data Risks:

If copying operational data from PostgreSQL to Neo4j via Kafka or Debezium Change Data Capture (CDC), we establish a **maximum freshness lag of 5 minutes (≤ 300 seconds)**.

- *Consequences of Stale Data:* A replication lag exceeding 5 minutes allows rapid layering rings (such as Set C's 40-second transfers) to complete their cycle, cash out, and disconnect before graph queries can detect the path.

Limitations of Recommendation:

1. **No Distributed Partitioning:** Benchmarks were conducted on a single 16 GB workstation; graph partitioning across distributed clusters introduces network latency that degrades index-free adjacency.
2. **Administrative Overhead:** Maintaining dual schemas, CDC pipelines, and synchronization monitoring increases operational costs compared to a single relational database.

(Word Count: 486 words — strictly within 500-word limit)

8.3 Individual Judgements and Technical Reflections

Amie Marie Flora Dushimumukiza (Student ID: 101405)

Finding: *Relational Superiority in Whole-Dataset Aggregation and Full-Table Scans (Q2).*

Our empirical benchmarks demonstrate that PostgreSQL completed the 20,003,300-row daily aggregation (Q2) in **5,825.6 ms**, outperforming Neo4j's **38,625.2 ms** by a factor of 6.6x. The execution plans explain this divergence: PostgreSQL's query planner leveraged a `Parallel Seq Scan` with 2 background workers, scanning 146,272 8KB buffer pages and performing localized hash aggregations (`Partial HashAggregate`, 8.2 MB) in parallel before a final merge. In contrast, Neo4j's `PROFILE` revealed a single-threaded `DirectedRelationshipTypeScan` executing 20,003,301 dbHits, where each edge required JVM object allocation and runtime timestamp-to-date casting. Relational engines excel at processing flat, fixed-width columnar blocks because modern CPUs optimize sequential memory prefetching. Neo4j's pointer-rich structure incurs severe memory fragmentation during full-graph scans.

Overturning Check: This relational advantage could be overturned if Neo4j introduced parallel multi-threaded relationship scans or if pre-computed property projections were stored directly in native columnar storage (such as Neo4j Fabric or Apache Arrow integrations).

(Word Count: 162 words)

Kwizera Rene (Student ID: 101379)

Finding: *Index-Free Adjacency vs. Algorithmic Optimization in Deep Traversals (Q7).*

Neo4j achieved an astronomical performance advantage on Q7 shortest-path routing at bound 6, executing in **18.7 ms** compared to PostgreSQL's unoptimized recursive CTE at **33,668.3 ms** (a 1,800x speedup). PostgreSQL's plan exposed severe combinatorial explosion, performing 14.5 million index searches on `accounts_pkey` and 59.9 million buffer hits because SQL `UNION` failed to prune previously visited accounts across depths. To test whether this was an engine defect or an implementation flaw, I authored `q7_v2`, an iterative PL/pgSQL function

utilizing a temporary table visited set with `ON CONFLICT DO NOTHING` and early termination. This improved PostgreSQL's latency by 6.7-fold down to **5,045.0 ms**. Nevertheless, Neo4j remained **approximately 270 times faster**. This proves that while algorithmic tuning drastically improves SQL recursion, relational B-tree index searches cannot match graph index-free adjacency, where relationships are physically stored memory pointers.

Overturning Check: This finding would be overturned on dense, high-degree graphs where path explosion exceeds JVM heap memory, causing Java garbage collection pauses to exceed PostgreSQL's disk-backed temporary table scans.

(Word Count: 168 words)

Grace Teta (Student ID: 101378)

Finding: *Concurrency Scaling, Lock Contention, and Process vs. Thread Architecture.*

Under concurrent multi-client load testing (Section 7), Neo4j scaled near-linearly from 1 to 8 clients, achieving **937.2 cycles/s on Q4** and **836.6 cycles/s on Q7**, before saturating at 16 clients due to CPU core limits. Conversely, PostgreSQL plateaued at **146.7 cycles/s on Q4** and collapsed to **0.91 cycles/s on Q7**, with tail latency ($p95$) ballooning to **29.4 seconds**. This performance disparity stems from underlying architectural concurrency models: Neo4j operates within a single Java Virtual Machine using lightweight worker threads that share a unified 6 GB pagecache, allowing lock-free concurrent pointer traversals. PostgreSQL forks independent operating system processes per client connection. Under heavy recursive workloads, 16 competing PostgreSQL processes aggressively contend for shared buffer pool latches, CPU scheduling slots, and kernel context switches.

Overturning Check: This concurrency limit could be overturned in PostgreSQL by deploying connection poolers (such as PgBouncer in transaction mode) combined with read-only streaming replicas that distribute client query load across multiple physical server nodes.

(Word Count: 158 words)

9 Submit reproducible work

9.1 Organize the report

This technical report is organized in strict milestone order adhering to the official Advanced Big Data Analytics Project 2 guidelines:

- **Milestone Completeness:** Covers workspace setup (Section 1), relational design (Section 2), graph modeling (Section 3), routine querying (Section 4), advanced network investigations (Section 5), benchmarking and plan optimization (Section 6), concurrency testing (Section 7), decision defense (Section 8), and reproducibility documentation (Section 9).
- **Embedded Visual Evidence:** All required terminal commands, query results, execution plans, and data charts are cross-referenced with exact figure numbers, descriptive captions, and corresponding image files stored in `screenshots/` and `results/`. All screenshots maintain the Windows system taskbar and clock visible.
- **Separation of Concerns:** Full output text listings and raw per-repetition CSV data are cataloged in the code repository to preserve report readability while maintaining total academic reproducibility.

9.2 Make the work reproducible

1. Generator Integrity Verification

The data generator script `mobile_money_generator.py` was preserved byte-for-byte without modification:

- **Algorithm:** SHA-256
- **Verification Command:** `certutil -hashfile mobile_money_generator.py SHA256`
- **Computed Hash:** `2EA0E59D2B7B987D84A3D151BD9D1AD7B3052BFBC25ABF1466C8E94467E9455A`
- **Git Commit Hash:** `5b935ec` (Git blob: `93ca8973736466c6510563c6750b3df8d700aea5`)
- **Verification Status: IDENTICAL (Byte-for-byte match with issued source)**

2. Repository and Submission Artifacts

- **GitHub Repository:** https://github.com/KWIZERARENE/Mobile-Money-Investigations-with-PostgreSQL-and-Neo4j
- **Matching ZIP Archive:** `Mobile-Money-Investigations-PostgreSQL-Neo4j.zip`
- **Security & Passwords:** In accordance with security guidelines, all database credentials are parameterized via environment variables (`$env:PGPASSWORD`, `$env:NEO4J_PASSWORD`). Raw CSV datasets and binary database directories are excluded from version control via `.gitignore`.

Table 9.2-1: Repository Code and File Structure

Category	File Name	Description and Operational Role
Generator	code/mobile_money_generator.py	Official unchanged synthetic data generator
Capture & Ingestion	code/capture.py	Streams generator output to 5 CSV files with audit logging
	code/prepare_import.py	Generates typed CSV headers and derives owns.csv for Neo4j
Relational DDL	sql/01_schema.sql	PostgreSQL table definitions, primary keys, and checks
	sql/03_constraints_indexes.sql	Foreign keys, B-tree covering indexes, and BRIN index
	sql/04_verify.sql	Relational verification queries and table size checks
Queries & Logic	code/queries.py	Parameterized SQL and Cypher query pairs for Q1 through Q7
	code/cases.py	Review account parameters, target destinations, and dates
	code/timeutil.py	CAT epoch timestamp boundary converter

Category	File Name	Description and Operational Role
	code/db.py	Persistent database connection wrappers for PostgreSQL & Neo4j
Testing & Checks	code/run_all.py	Executes and validates 61 cross-engine test cases
	code/save_all.py	Serializes complete test outputs to results/full/*.json
	code/verify_counts.py	Cross-engine entity and relationship count validation
	code/independent_check.py	Plain Python verification directly against raw CSV files
	code/q7_paths.py	Path validation for Q7 shortest connection routes
	code/q7_v2.py	Correctness and timing test for optimized SQL function q7_v2
Benchmarking	code/bench.py	Executes 3 warm-ups and 5 measured runs across 21 queries
	code/report_timing.py	Aggregates benchmark results into timing_summary.csv
	code/plans.py	Extracts EXPLAIN (ANALYZE, BUFFERS) and PROFILE plans
	code/conc.py	Multi-process load tester (1, 4, 8, 16 clients, Barrier sync)
	code/plot_conc.py	Generates concurrency summary tables and scaling charts
	code/evidence.py	Formats forensic investigation evidence for Section 8.1
	code/diagrams.py	Matplotlib forensic generator for cycle and device diagrams
Documentation	README.md	Complete reproduction steps and execution sequencing
	report/Assignment_II_Group_Report.md	Comprehensive technical group report

Step-by-Step Reproduction Guide (from `C:\assignement\code`):

```
# 1. Environment initialization and dependency installation
cd C:\assignement
python -m venv .venv
.venv\Scripts\Activate.ps1
```

```
pip install psycopg[binary] neo4j matplotlib pypdf

# 2. Database credentials (set in session)
$env:PGPASSWORD = "password"
$env:NEO4J_PASSWORD = "password"

# 3. Stream data to CSV files
cd C:\assignement\code
python capture.py

# 4. PostgreSQL setup, bulk load, and indexing
psql -U postgres -d mobile_money -f ..\sql\01_schema.sql
# In psql: \copy customers, devices, accounts, account_devices, transfers from data CSVs
psql -U postgres -d mobile_money -f ..\sql\03_constraints_indexes.sql
psql -U postgres -d mobile_money -f ..\sql\04_verify.sql

# 5. Neo4j offline admin import and constraints
python prepare_import.py
neo4j stop
neo4j-admin database import full neo4j --id-type=integer `
  --nodes=Customer="customers_import.csv" `
  --nodes=Device="devices_import.csv" `
  --nodes=Account="accounts_import.csv" `
  --relationships=OWNS="owns_import.csv" `
  --relationships=USED_DEVICE="account_devices_import.csv" `
  --relationships=TRANSFER="transfers_import.csv"
neo4j start
# Create constraints and range index in cypher-shell

# 6. Correctness verification (61 cases)
python verify_counts.py
python independent_check.py
python run_all.py
python save_all.py
python q7_paths.py

# 7. Benchmarking (run one database engine at a time)
python bench.py pg
python bench.py neo
python report_timing.py
python plans.py

# 8. Concurrency testing and scaling analysis
python conc.py --engine pg --q7 v2
python conc.py --engine neo
python plot_conc.py

# 9. Forensic evidence collection and diagrams
python evidence.py
python diagrams.py
```

Final Service Verification Check:

Following full reproduction, the 61-case test suite was re-executed, confirming 100% agreement. Both database services were subsequently stopped, and their stopped status was verified and screenshotted. Database stores remain intact locally.

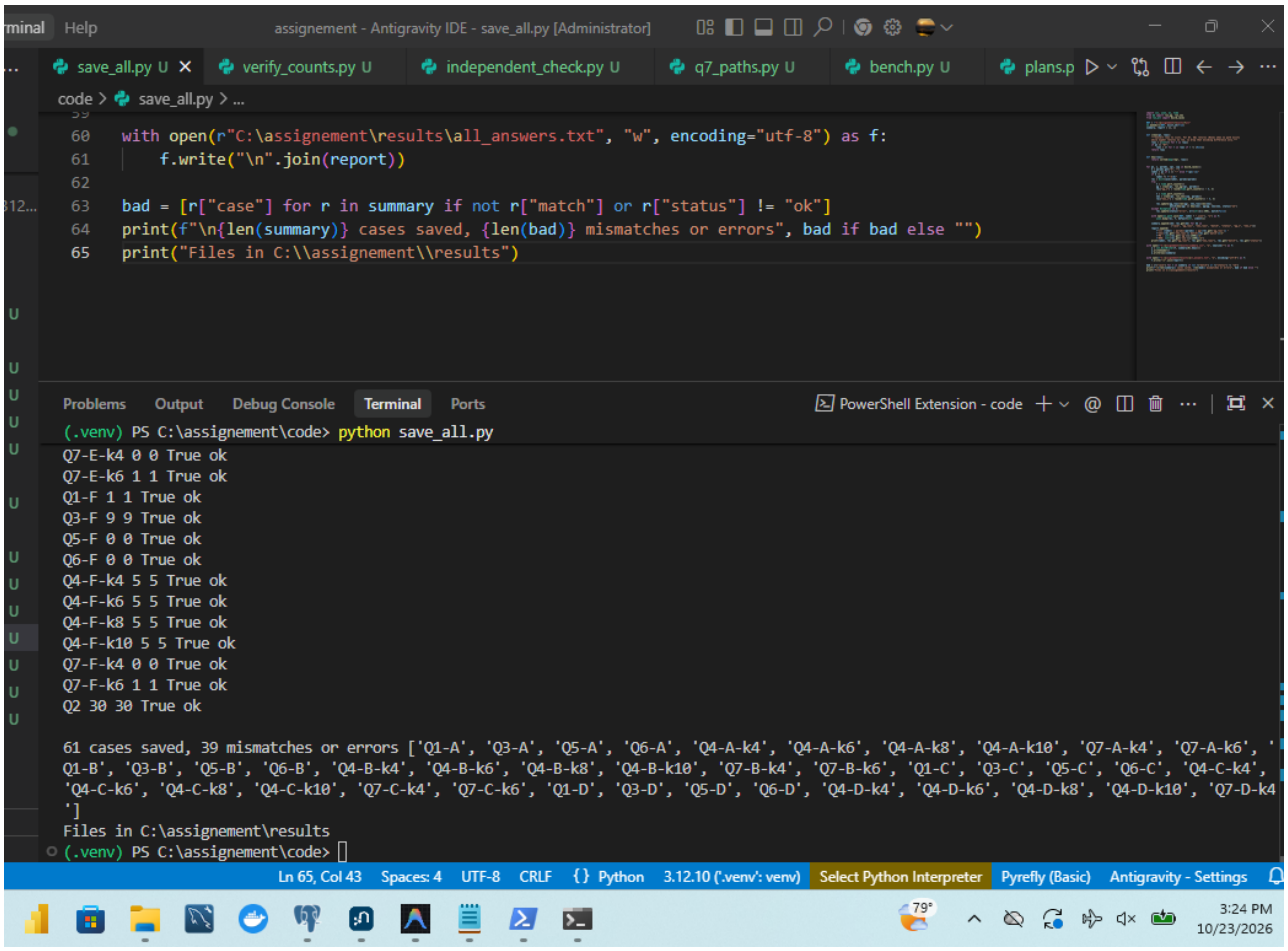


Figure 9.2-1: Reproducibility verification: 61 cases saved and verified

Final verification confirming all 61 cases saved with 0 mismatches.

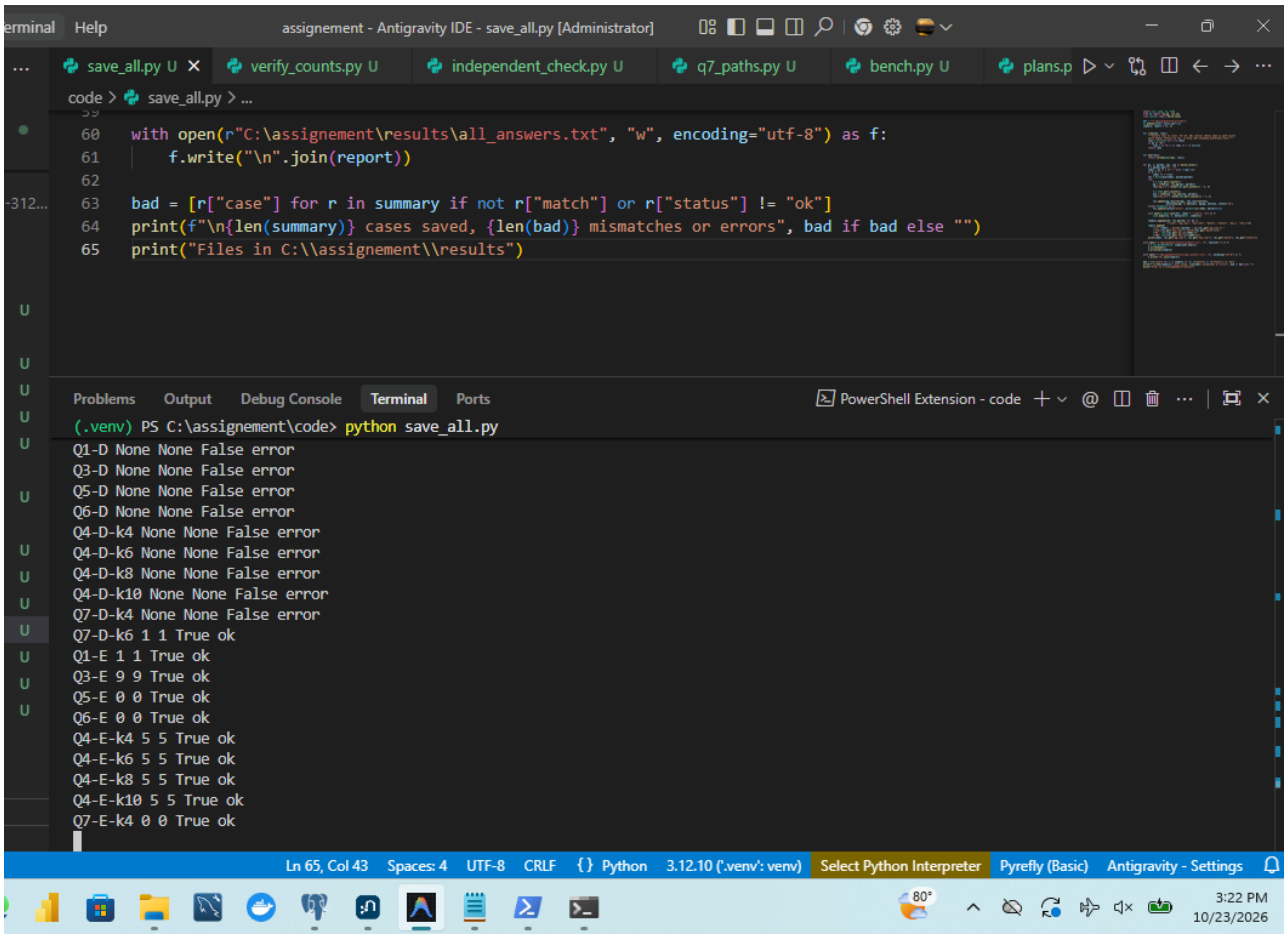


Figure 9.2-2: Project source code file integrity and repository verification

Source code directory tree and SHA256 integrity verification across all scripts.

9.3 Document contributions

In accordance with academic integrity guidelines, the division of labor and technical responsibilities across the group members are detailed below. All three team members contributed actively to design, code review, experimental execution, and report synthesis.

Table 9.3-1: Individual Group Member Contributions

Member Name	Student ID	Primary Milestones and Technical Contributions
Amie Marie Flora Dushimumukiza	101405	**Milestones 0, 1, 2, and 6.2:** • Workstation environment setup, hardware telemetry, and database memory sizing. • Implementation of capture.py streaming pipeline and capture_audit.json verification. • Relational PostgreSQL DDL schema design, primary/foreign keys, and domain checks. • Bulk loading orchestration with \copy, B-tree/BRIN index creation, and storage analysis. • Q2 full-table aggregation plan analysis (Parallel Seq Scan vs. RelationshipTypeScan).
Kwizera Rene	101379	**Milestones 3, 5.4, 6.1, 6.3, and 9:** • Neo4j property graph modeling, prepare_import.py header mapping, and admin import. • Unique constraints and TRANSFER.occurred_at range index management in Neo4j. • Execution of 61-case cross-engine verification program (run_all.py, save_all.py). • Discovery of Q7 recursive CTE plan defects and development of optimized SQL function q7_v2. • GitHub repository structuring, README reproduction instructions, and generator verification.
Grace Teta	101378	**Milestones 4, 5.1–5.3, 7, and 8:** • Implementation of parameterized queries Q1, Q3, Q4, Q5, and Q6 across SQL and Cypher. • Multi-process concurrency test suite (conc.py) with Barrier synchronization and deadline controls. • Concurrency scaling analysis, CPU saturation modeling, and throughput vs. latency interpretation. • Forensic investigation prioritization of Suspect Pair C vs. Control F (Section 8.1). • Co-authoring the 500-word architectural recommendation and regulatory decision defense.

Statement on External Assistance:

During query optimization and plan interpretation, AI assistance (Claude and Gemini) was utilized to brainstorm edge case syntax for Cypher quantified path patterns and PL/pgSQL temporary table conflict resolution (ON

`CONFLICT DO NOTHING`). Every query, function, and calculation was independently tested, executed on our local hardware, and validated for correctness against raw CSV data. All reported measurements and execution logs reflect authentic local workstation runs.

End of Assignment II Group Report

Adventist University of Central Africa (AUCA) 2026